Computational Statistics

Lecture notes with R, Lectures 1–14

Contents

Lecture 1. Introduction 6

1.1 What the course is about 6
1.1.1 Organisation and scope . . . 6

1.2 Software 7

1.3 Running R code 7

1.4 Getting help 7

1.5 Selected exercises 8

Lecture 2. Data Visualisation 10

2.1 Data visualisation with ggplot2 10
2.1.1 The mpg data frame . . . 10
2.1.2 Creating a ggplot . . . 10

2.2 Aesthetic mappings 11

2.3 Facets 11

2.4 Geometric objects 12

2.5 Statistical transformations 13

2.6 Position adjustments 14

2.7 Coordinate systems 15

2.8 The layered grammar of graphics 16

2.9 Graphics for communication 16
2.9.1 Labels . . . 16
2.9.2 Annotations . . . 17
2.9.3 Scales, legends and zooming . . . 17
2.9.4 Themes and saving . . . 18

Lecture 3. Data Transformation 19

3.1 Why transform data 19
3.1.1 The five verbs . . . 19

3.2 Filter rows with filter() 19

3.2.1 Comparisons . . . 20
3.2.2 Logical operators . . . 20
3.2.3 Missing values . . . 20

3.3 Arrange rows with arrange() 21

3.4 Select columns with select() 21

3.5 Add new variables with mutate() 22
3.5.1 Useful creation functions . . . 22

3.6 Grouped summaries with summarise() 23
3.6.1 The pipe . . . 23
3.6.2 Missing values and counts . . . 24
3.6.3 Useful summary functions . . . 25

3.7 Grouped mutates and filters 26

Lecture 4. Exploratory Data Analysis 27

4.1 What EDA is 27

4.2 Variation 27
4.2.1 Typical values . . . 28
4.2.2 Subgroups . . . 29
4.2.3 Unusual values . . . 29

4.3 Missing values 30

4.4 Covariation 30
4.4.1 A categorical and a continuous variable . . . 31
4.4.2 Two categorical variables . . . 32
4.4.3 Two continuous variables . . . 32

4.5 Patterns and models 33

Lecture 5. Import and Tidy Data 35

5.1 Tibbles 35

5.2 Data import with readr 36
5.2.1 Parsing a vector . . . 36
5.2.2 Parsing a file . . . 38
5.2.3 Writing files and other formats . . . 38

5.3 Tidy data 38

5.4 Gathering and spreading 40

5.5 Separating and uniting 41

Lecture 6. Relational Data 42

6.1 Relational data 42
6.1.1 Keys . . . 42

6.2 Mutating joins 42
6.2.1 Duplicate keys . . . 43
6.2.2 Defining the key columns . . . 44

6.3 Filtering joins 45

6.4 Set operations 45

6.5 Selected exercises 46

Lecture 7. Strings 48

7.1 String basics 48
7.1.1 Length, combining, subsetting . . . 48

7.2 Regular expressions 49
7.2.1 Basic matches . . . 49
7.2.2 Anchors and character classes . . . 49
7.2.3 Repetition . . . 49
7.2.4 Grouping and backreferences . . . 50

7.3 Tools 50
7.3.1 Detecting matches . . . 50
7.3.2 Extracting matches . . . 51
7.3.3 Grouped matches . . . 51
7.3.4 Replacing and splitting . . . 51

7.4 Selected exercises 52

Lecture 8. Factors, Dates and Times 54

8.1 Factors 54
8.1.1 Creating factors . . . 54
8.1.2 The General Social Survey . . . 54
8.1.3 Modifying factor order . . . 54
8.1.4 Modifying factor levels . . . 55

8.2 Dates and times 56
8.2.1 Creating dates and date-times . . . 56
8.2.2 Date-time components . . . 57
8.2.3 Time spans . . . 58
8.2.4 Time zones . . . 59

8.3 Selected exercises 59

Lecture 9. Pipes 61

9.1 Why pipes 61

9.2 When not to use the pipe **61**

9.3 Other tools from magrittr **62**

9.4 Selected exercises **62**

Lecture 10. Functions **65**

10.1 When to write a function **65**

10.2 Functions are for humans and computers **65**

10.3 Conditional execution **66**
10.3.1 Conditions . . . 66
10.3.2 Multiple conditions . . . 66
10.3.3 Code style . . . 67

10.4 Function arguments **67**
10.4.1 Checking values . . . 67
10.4.2 Dot-dot-dot . . . 68

10.5 Return values **68**

10.6 Environment **68**

10.7 Selected exercises **69**

Lecture 12. Iteration **71**

12.1 Iteration **71**

12.2 For loops **71**
12.2.1 Variations . . . 71

12.3 For loops versus functionals **73**

12.4 The map functions **73**
12.4.1 Shortcuts . . . 73
12.4.2 Base R . . . 74

12.5 Dealing with failure **74**

12.6 Mapping over multiple arguments **75**

12.7 Walk, reduce and accumulate **75**

12.8 Selected exercises **75**

Lecture 13. Model Basics **78**

13.1 Models and their purpose **78**

13.2 A simple model **78**

13.3 Visualising models **79**

13.4 Formulas and model families **80**
13.4.1 Categorical variables 80
13.4.2 Interactions 81
13.4.3 Transformations 82

13.5 Missing values and other model families **82**

13.6 Selected exercises **83**

Lecture 14. Model Building **85**

14.1 Building models step by step **85**

14.2 Why are low-quality diamonds more expensive? **85**
14.2.1 Price and carat 85
14.2.2 A more complicated model 86

14.3 What affects the number of daily flights? **87**
14.3.1 Day of week 88
14.3.2 Seasonal Saturday effect 88
14.3.3 Time of year: an alternative approach 90

14.4 Selected exercises **90**

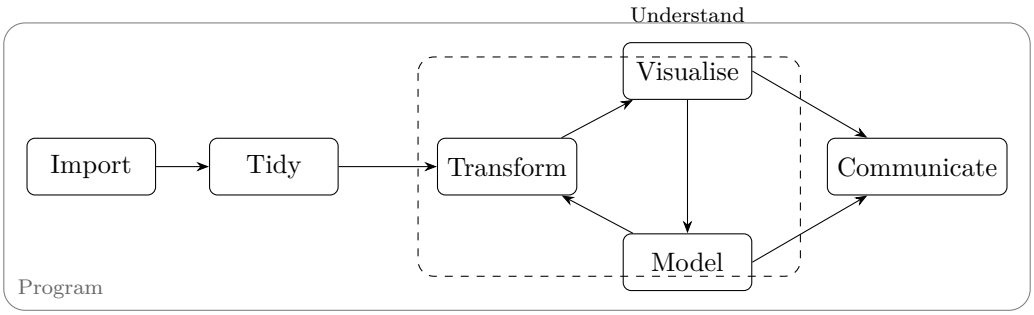
Lecture 1. Introduction

Based on the slides (*R for Data Science*, 1st ed., Wickham and Grolemund). All code was run at compile time; `##` lines are output. [Supplement] marks material not on the slides.

1.1 What the course is about

Motivation. Most of the work happens before the model: importing, cleaning and looking at data, done in reproducible code.

The course teaches how to collect, process, analyse and present data with a programming language (**data science**). A typical project:



- **Import**: file, database or web API → data frame.
- **Tidy**: *each column is a variable, each row an observation.*
- **Transform**: select observations, create variables, compute summaries. Tidy + transform = **wrangling**.
- **Visualise**: shows the unexpected and raises questions; needs a human, so does not scale.
- **Model**: answers a precise question; scales, but cannot check its own assumptions.
- **Communicate**: results matter only if others understand them.
- **Program**: runs through every stage; automates repeated tasks.

Visualise, transform and model form a loop traversed many times.

1.1.1 Organisation and scope

Order follows the book: visualisation and transformation first, then wrangling, programming and modelling.

Not covered: big data, languages other than R, non-rectangular data (images, sound, trees, text), and hypothesis *confirmation*. The course is about **hypothesis generation**.

Generation versus confirmation [Supplement]

An observation can suggest a hypothesis or test it, never both: testing a pattern on the data that suggested it is biased toward success (Lecture 13).

1.2 Software

Motivation. R (the language), RStudio (the editor) and the tidyverse (packages with one shared design).

R comes from CRAN (mirror `https://cloud.r-project.org`). **RStudio** is an IDE: run code in the console pane or write scripts in the editor pane.

A **package** bundles functions, data and documentation. The **tidyverse** is a family of packages with a common design. Install once, load every session:

```
install.packages("tidyverse")
```

```
library(tidyverse)
```

Core packages:

package	role
`ggplot2`	graphics (Lecture 2)
`dplyr`	data transformation (Lectures 3, 6)
`tidyr`	tidying (Lecture 5)
`readr`	reading rectangular files (Lecture 5)
`tibble`	a modern data frame (Lecture 5)
`stringr`, `forcats`	strings and factors (Lectures 7, 8)
`purrr`	functional programming (Lecture 12)

Conflicts: `dplyr::filter()` masks `stats::filter()`; `package::function()` names a function explicitly. Update with `tidyverse_update()`. Data packages used:

```
install.packages(c("nycflights13", "gapminder", "Lahman"))
```

1.3 Running R code

Motivation. How code and output are displayed.

At the prompt `>`, R evaluates a line and prints the result:

```
1 + 2
```

```
## [1] 3
```

Code is shown without the prompt; output lines start with `##` (a comment, so safe to copy). `[1]` is the index of the first element on the line: basic objects are vectors.

```
seq(10, 100, by = 5)
```

```
##  [1]  10  15  20  25  30  35  40  45  50  55  60  65  70  75  80  85  90  95
## [19] 100
```

Functions carry parentheses (`sum()`), other objects do not (`flights`); `dplyr::mutate()` denotes `mutate()` from `dplyr`.

1.4 Getting help

`?mean` opens the help page of `mean()`; `??regression` searches all help pages. Outside R, search the exact error message (Stack Overflow, R-bloggers).

A **reproducible example** (reprex) gives the packages, a small data set (`dput(head(df))` prints it as R code) and the shortest code showing the problem.

```
dput(head(mtcars[, 1:3], 3))
```

```
## structure(list(mpg = c(21, 21, 22.8), cyl = c(6, 6, 4), disp = c(160,
## 160, 108)), row.names = c("Mazda RX4", "Mazda RX4 Wag", "Datsun 710"
## ), class = "data.frame")
```

1.5 Selected exercises

Motivation. Selection bias, floating point, masking and tidiness.

Exercise 1.1 (exploring and confirming on the same data)

An analyst explores a data set, finds k candidate patterns, and tests each at level α. Suppose no pattern is real and the k p-values are independent and uniform on $[0, 1]$.
(a) Show that the probability that at least one test is significant is $1 - (1 - \alpha)^k$; evaluate it for $\alpha = 0.05$, $k = 20$.
(b) The analyst reports only the most striking pattern and "confirms" it with its p-value on the same data. Show that this confirmation rejects with probability $1 - (1 - \alpha)^k$, not α.
(c) Show that testing the selected pattern on new, independent data rejects with probability exactly α.

Solution [Supplement].

(a) $P(\text{none significant}) = P(\text{all } p_j > \alpha) = (1 - \alpha)^k$ by independence, so $P(\text{at least one}) = 1 - (1 - \alpha)^k$; for $\alpha = 0.05$, $k = 20$: $1 - 0.95^{20} = 0.642$.

(b) The reported p-value is $p_{(1)} = \min_j p_j$, and $p_{(1)} \le \alpha$ iff some $p_j \le \alpha$: probability $1 - (1 - \alpha)^k$ (64%, not 5%).

(c) Let J be the selected index and q_j the p-values on new data, independent of J and uniform under the null:

$$P(q_J \le \alpha) = \sum_j P(J = j)\, P(q_j \le \alpha \mid J = j) = \sum_j P(J = j)\, \alpha = \alpha.$$

■

Exercise 1.2 (why $0.1 + 0.2 \neq 0.3$)

Explain the output below. Show that $1/10$ has no finite binary expansion, and characterise the decimals that are stored exactly.

```
0.1 + 0.2 == 0.3
```

```
## [1] FALSE
```

```
sprintf("%.20f", 0.1)
```

```
## [1] "0.10000000000000000555"
```

```
isTRUE(all.equal(0.1 + 0.2, 0.3))
```

```
## [1] TRUE
```

Solution [Supplement].

Step 1. Doubles are $m/2^k$ (integers m, k, m of at most 53 bits); exactly these numbers have finite binary expansions.

Step 2. $1/10 = m/2^k \Rightarrow 2^k = 10m \Rightarrow 5 \mid 2^k$, impossible. So 0.1 is stored as the nearest double, slightly above 0.1 (20 digits above); 0.2, 0.3 are rounded too.

Step 3. The errors do not cancel: computed $0.1 + 0.2$ and stored 0.3 are different doubles, so `==` gives `FALSE`. Exact decimals are the $m/2^k$: 0.5, 0.25, 0.375.

Step 4. Compare with a tolerance: `all.equal()` (relative tolerance $\approx 1.5 \times 10^{-8}$), wrapped in `isTRUE()` since it returns a description, not `FALSE`, on a mismatch. ■

Exercise 1.3 (masking)

After `library(tidyverse)` the message says that `dplyr::filter()` masks `stats::filter()`. Explain the two results below.

```
stats::filter(1:5, rep(1, 3) / 3)
```

```
## Time Series:
## Start = 1
## End = 5
## Frequency = 1
## [1] NA  2  3  4 NA
```

```
filter(1:5, rep(1, 3) / 3)
```

```
## Error in UseMethod("filter"): no applicable method for 'filter' applied to an object of
    class "c('integer', 'numeric')"
```

Solution [Supplement].

Step 1. `stats::filter()` with weights $(1/3, 1/3, 1/3)$ is a centred moving average: $2, 3, 4$, with `NA` at both ends.

Step 2. A bare name is looked up along `search()`, global environment → attached packages → base. `library()` attaches dplyr ahead of `stats`, so `filter` finds dplyr's first.

Step 3. dplyr's `filter()` has no method for numeric vectors, hence the error. Writing `package::function()` bypasses the search path. ■

Exercise 1.4 (what is tidy; adapted from R4DS, Exercise 12.2.1.1)

A table has one row per country and columns `country`, `1999` and `2000`, the last two holding the number of tuberculosis cases in that year. Name the variables and the observations, explain why the table is not tidy, and describe the tidy table for c countries.

Solution [Supplement].

Step 1. Variables: country, year, cases; observation: one country in one year. Values of year are column names and cases is spread over two columns, so a row holds two observations.

Step 2. Tidy: columns `country`, `year`, `cases`, one row per country–year, $2c$ rows; its $2c$ values of `cases` are the $2c$ year-column entries, so nothing is lost (`gather()`/`pivot_longer()`, Lecture 5). ■

Lecture 2. Data Visualisation

2.1 Data visualisation with ggplot2

Motivation. ggplot2 builds every plot from one small set of parts.

Load the tidyverse (attaches `ggplot2`):

```
library(tidyverse)
```

2.1.1 The mpg data frame

Do cars with big engines use more fuel? `ggplot2::mpg` records 234 cars; `displ` is engine size (litres), `hwy` highway efficiency (miles per gallon).

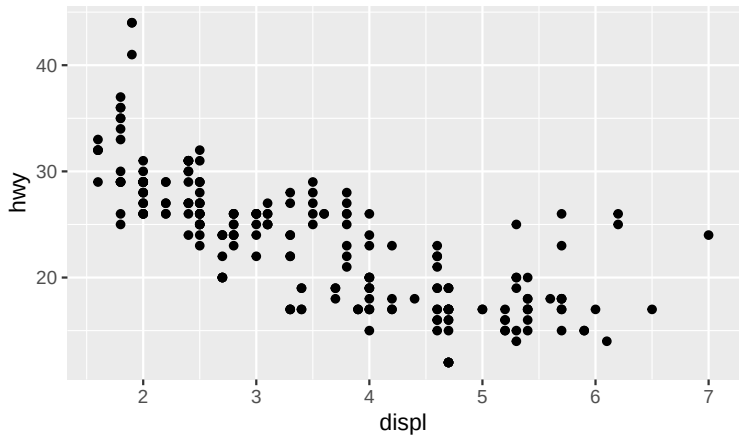
```
mpg
```

```
## # A tibble: 234 x 11
##    manufacturer model      displ  year   cyl trans      drv     cty   hwy fl    class
##    <chr>        <chr>      <dbl> <int> <int> <chr>      <chr> <int> <int> <chr> <chr>
##  1 audi         a4           1.8  1999     4 auto(l5)   f        18    29 p     compact
##  2 audi         a4           1.8  1999     4 manual(m5) f        21    29 p     compact
##  3 audi         a4           2    2008     4 manual(m6) f        20    31 p     compact
##  4 audi         a4           2    2008     4 auto(av)   f        21    30 p     compact
##  5 audi         a4           2.8  1999     6 auto(l5)   f        16    26 p     compact
##  6 audi         a4           2.8  1999     6 manual(m5) f        18    26 p     compact
##  7 audi         a4           3.1  2008     6 auto(av)   f        18    27 p     compact
##  8 audi         a4 quattro   1.8  1999     4 manual(m5) 4        18    26 p     compact
##  9 audi         a4 quattro   1.8  1999     4 auto(l5)   4        16    25 p     compact
## 10 audi         a4 quattro   2    2008     4 manual(m6) 4        20    28 p     compact
## # i 224 more rows
```

A **data frame** has variables in columns and observations in rows; `mpg` is a tibble (Lecture 5).

2.1.2 Creating a ggplot

```
ggplot(data = mpg) +
  geom_point(mapping = aes(x = displ, y = hwy))
```



Bigger engines, lower efficiency.

- `ggplot(data = mpg)`: empty coordinate system (blank canvas) with data `mpg`.

- `geom_point()`: a **layer** of points, added with `+`.
- `mapping = aes(x = displ, y = hwy)`: variables (looked up in the data) → visual properties.

Graphing template:

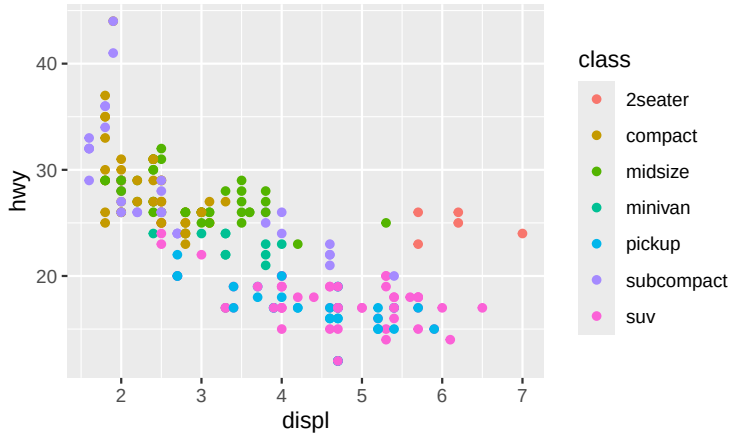
```
ggplot(data = <DATA>) +
  <GEOM_FUNCTION>(mapping = aes(<MAPPINGS>))
```

2.2 Aesthetic mappings

Motivation. Aesthetics bring a third variable into a 2-D plot, e.g. to explain the large-engine, high-mileage cars.

An **aesthetic** is a visual property (position, size, shape, colour, transparency); mapping a variable to it makes it vary. Colour by `class`: the unusual cars are two-seaters.

```
ggplot(data = mpg) +
  geom_point(mapping = aes(x = displ, y = hwy, color = class))
```



Scaling assigns a level to each value and adds a legend. Other aesthetics:

```
ggplot(data = mpg) + geom_point(mapping = aes(x = displ, y = hwy, size = class))
ggplot(data = mpg) + geom_point(mapping = aes(x = displ, y = hwy, alpha = class))
ggplot(data = mpg) + geom_point(mapping = aes(x = displ, y = hwy, shape = class))
```

- `size`, `alpha` imply an order: not for unordered `class` (ggplot2 warns).
- `shape`: at most six at a time, extra groups dropped (SUVs vanish). Shapes 0–24; 21–24 have border (`colour`) and interior (`fill`).

Setting versus mapping. A constant property goes *outside* `aes()`:

```
ggplot(data = mpg) +
  geom_point(mapping = aes(x = displ, y = hwy), color = "blue")
```

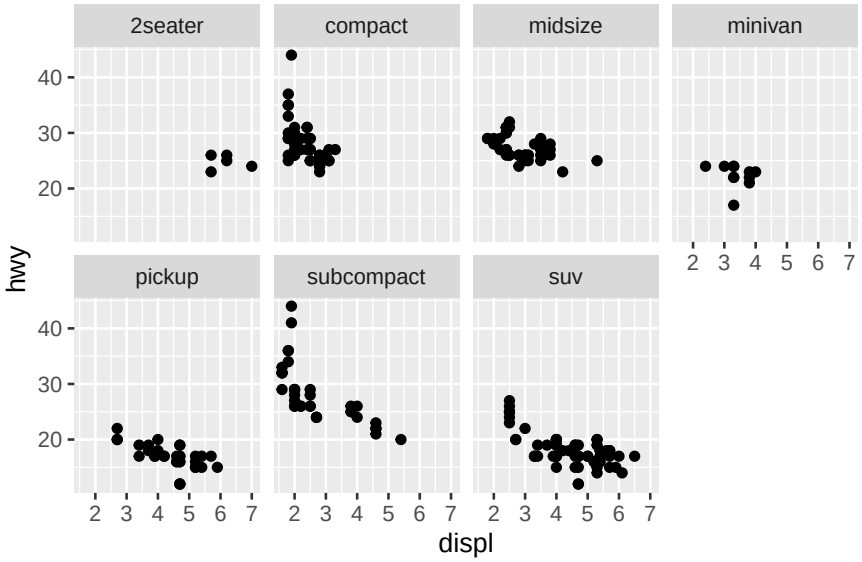
`aes(color = "blue")` would map the string `"blue"`: one group "blue", drawn red, with a legend.

2.3 Facets

Motivation. One small plot per level, side by side.

`facet_wrap()` splits by one discrete variable (a formula):

```
ggplot(data = mpg) +
  geom_point(mapping = aes(x = displ, y = hwy)) +
  facet_wrap(~ class, nrow = 2)
```



`facet_grid(drv ~ cyl)`: rows by `drv`, columns by `cyl`; `facet_grid(. ~ cyl)`: columns only. Groups are clearer, comparisons harder than with colour.

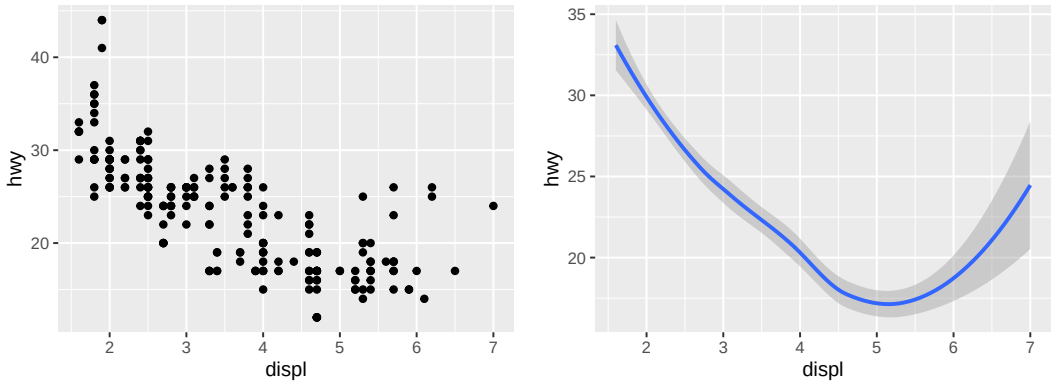
```
ggplot(data = mpg) +
  geom_point(mapping = aes(x = displ, y = hwy)) +
  facet_grid(drv ~ class)
```

2.4 Geometric objects

Motivation. Points, curve, bars or boxes: one function, the geom.

A **geom** is the object representing the data. Same data and mapping, different geoms:

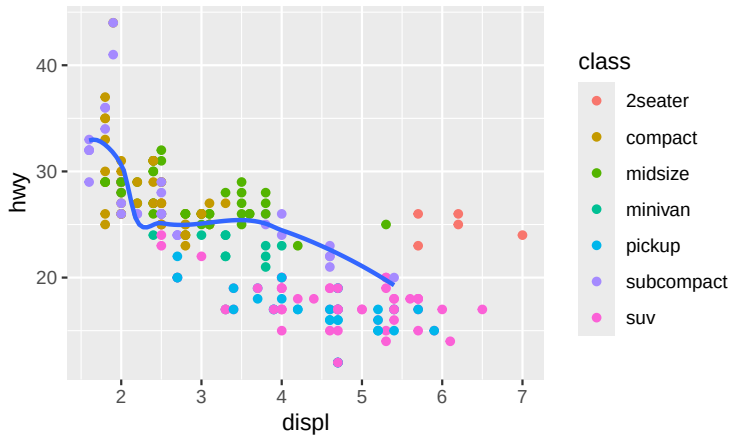
```
ggplot(data = mpg) + geom_point(mapping = aes(x = displ, y = hwy))
ggplot(data = mpg) + geom_smooth(mapping = aes(x = displ, y = hwy))
```



`geom_smooth(aes(linetype = drv))` draws one curve per drive train (`linetype` fits lines, not points); mapping a discrete variable groups automatically, and `aes(group = drv)` groups without a legend.

Mappings in `ggplot()` are **global**; mappings in a geom are **local** and extend or override them. A layer can have its own data:

```
ggplot(data = mpg, mapping = aes(x = displ, y = hwy)) +
  geom_point(mapping = aes(color = class)) +
  geom_smooth(data = filter(mpg, class == "subcompact"), se = FALSE)
```



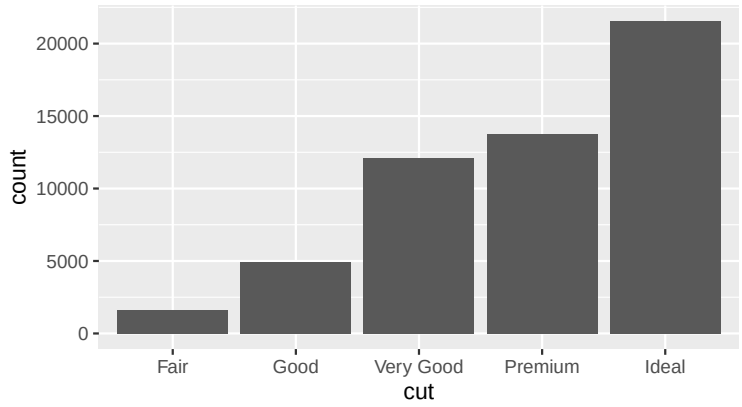
The smooth is fitted to subcompacts only (`filter()`: Lecture 3); `se = FALSE` drops the band.

2.5 Statistical transformations

Motivation. Some plots draw values computed from the data, such as counts.

`diamonds`: about 54 000 diamonds with price, carat, cut, colour, clarity.

```
ggplot(data = diamonds) +
  geom_bar(mapping = aes(x = cut))
```



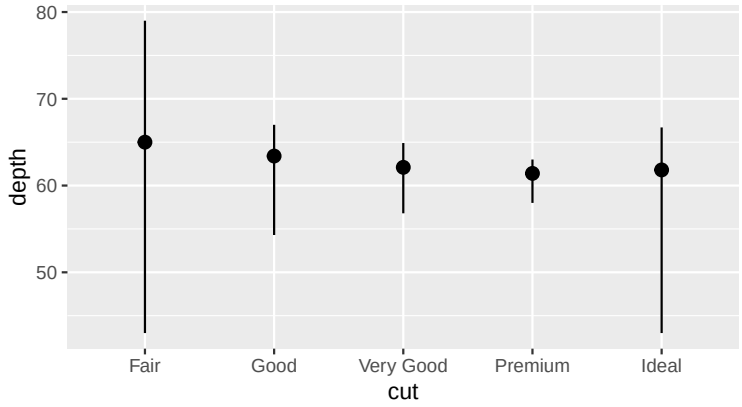
`count` is not a variable of `diamonds`. Bars, histograms, frequency polygons bin and count; smoothers fit a model; boxplots summarise. The algorithm is a **stat**: `geom_bar()` has default stat `stat_count()` and vice versa, so `ggplot(diamonds) + stat_count(aes(x = cut))` is the same plot.

Explicit stats:

- another computed variable: `aes(x = cut, y = after_stat(prop), group = 1)` (without `group = 1` every proportion is 1);

- a summary per x, e.g. median depth with range:

```
ggplot(data = diamonds) +
  stat_summary(
    mapping = aes(x = cut, y = depth),
    fun.min = min, fun.max = max, fun = median
  )
```

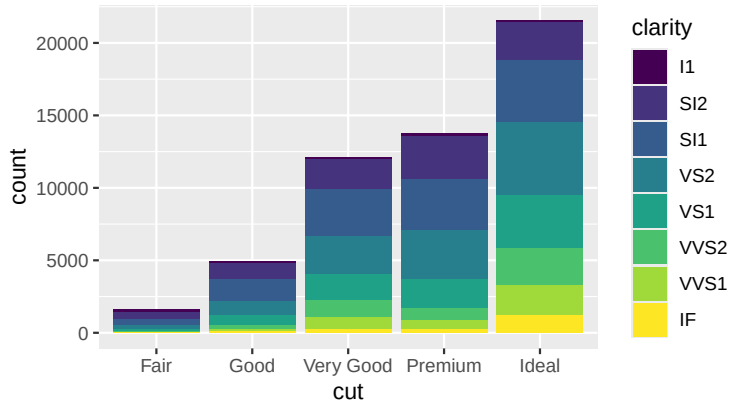


2.6 Position adjustments

Motivation. How overlapping objects are arranged changes what the plot shows.

Mapping a second variable to `fill` stacks the bars:

```
ggplot(data = diamonds) +
  geom_bar(mapping = aes(x = cut, fill = clarity))
```



Segment heights are clarity counts within the cut, e.g. ideal:

```
diamonds %>% filter(cut == "Ideal") %>% count(clarity)
```

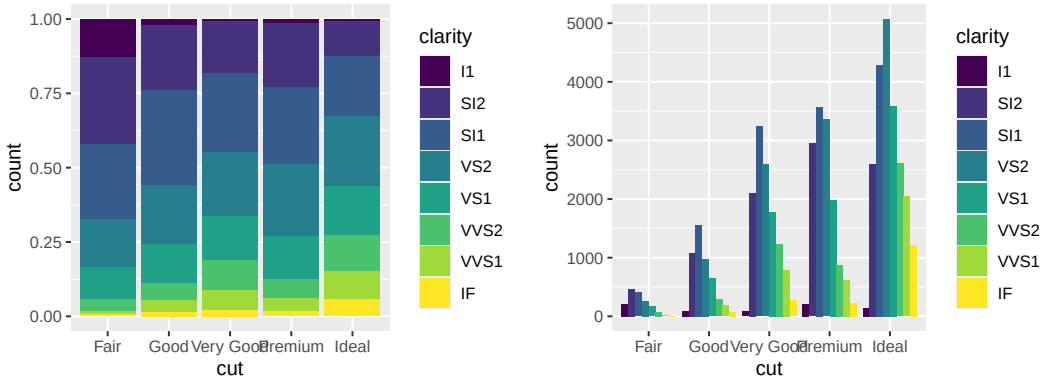
```
## # A tibble: 8 x 2
##   clarity     n
##   <ord>   <int>
## 1 I1        146
## 2 SI2      2598
## 3 SI1      4282
## 4 VS2      5071
## 5 VS1      3589
## 6 VVS2     2606
```

```
## 7 VVS1     2047
## 8 IF       1212
```

`position`:

- `"stack"` (default): segments on top of each other;
- `"identity"`: each bar from zero, overlapping (for `alpha` or lines);
- `"fill"`: stacked, rescaled to height 1: compares *proportions*;
- `"dodge"`: side by side: compares counts.

```
ggplot(diamonds) + geom_bar(aes(x = cut, fill = clarity), position = "fill")
ggplot(diamonds) + geom_bar(aes(x = cut, fill = clarity), position = "dodge")
```



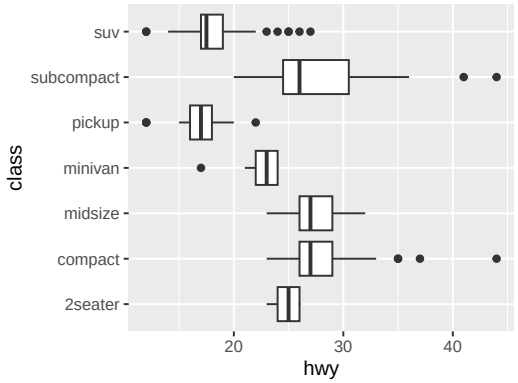
Rounded values make `mpg`'s 234 points coincide (**overplotting**). `position = "jitter"` (`geom_jitter()`) adds small noise: less accurate at small scales, more revealing at large ones.

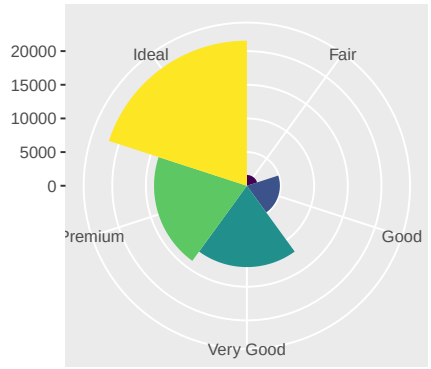
2.7 Coordinate systems

Default `coord_cartesian()` (zooms with `xlim`, `ylim`). Others:

- `coord_flip()`: swaps axes (horizontal boxplots);
- `coord_fixed(ratio = )`: fixed aspect ratio;
- `coord_quickmap()`: aspect ratio for maps (`map_data("nz")` polygons for `geom_polygon()`);
- `coord_polar()`: a bar chart becomes a Coxcomb chart.

```
ggplot(data = mpg, mapping = aes(x = class, y = hwy)) +
  geom_boxplot() + coord_flip()
bar <- ggplot(data = diamonds) +
  geom_bar(aes(x = cut, fill = cut), show.legend = FALSE, width = 1) +
  theme(aspect.ratio = 1) + labs(x = NULL, y = NULL)
bar + coord_polar()
```





2.8 The layered grammar of graphics

Motivation. Every plot fills the slots of one template.

```
ggplot(data = <DATA>) +
  <GEOM_FUNCTION>(
     mapping = aes(<MAPPINGS>),
     stat = <STAT>,
     position = <POSITION>
  ) +
  <COORDINATE_FUNCTION> +
  <FACET_FUNCTION>
```

Seven parameters: data → stat → geom, mapping to aesthetics, position, coordinates, optional facets.

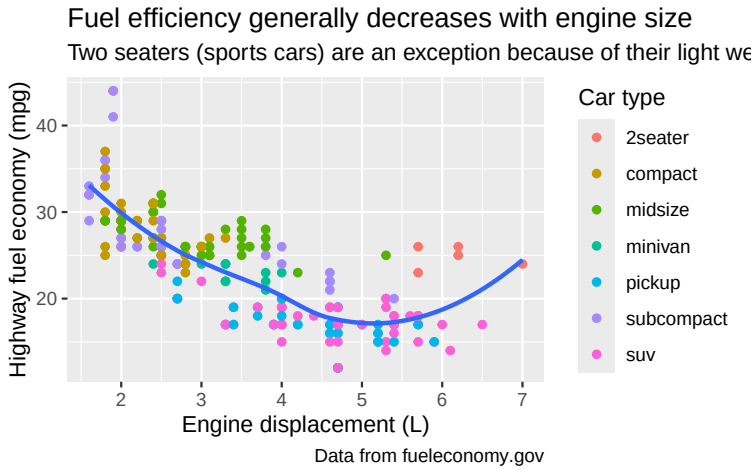
2.9 Graphics for communication

Motivation. Report plots need titles, readable labels and careful scales.

2.9.1 Labels

`labs()`: `title` (the main finding), `subtitle`, `caption` (data source), axis and legend titles:

```
ggplot(mpg, aes(displ, hwy)) +
  geom_point(aes(color = class)) +
  geom_smooth(se = FALSE) +
  labs(
    title = "Fuel efficiency generally decreases with engine size",
    subtitle = "Two seaters (sports cars) are an exception because of their light weight
    ",
    caption = "Data from fueleconomy.gov",
    x = "Engine displacement (L)", y = "Highway fuel economy (mpg)", colour = "Car type"
  )
```



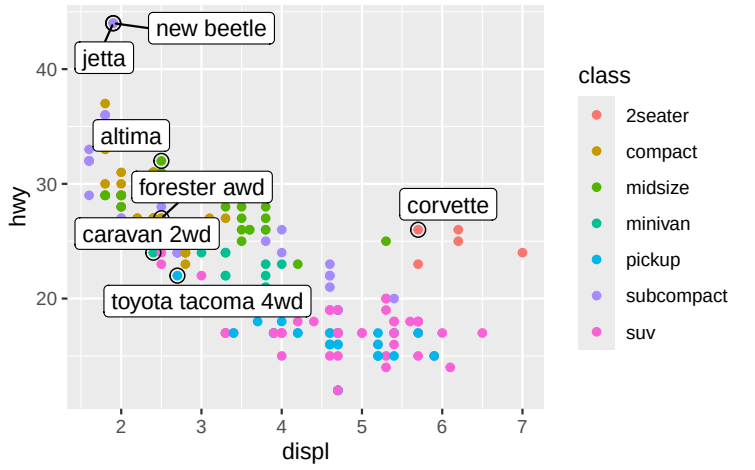
Math in labels: `quote()`, e.g. `labs(x = quote(sum(x[i]^2, i == 1, n)))`; see `?plotmath`.

2.9.2 Annotations

`geom_text()` with its own data labels observations, e.g. the most efficient car per class:

```
best_in_class <- mpg %>%
  group_by(class) %>%
  filter(row_number(desc(hwy)) == 1)
```

```
ggplot(mpg, aes(displ, hwy)) +
  geom_point(aes(colour = class)) +
  geom_point(size = 3, shape = 1, data = best_in_class) +
  ggrepel::geom_label_repel(aes(label = model), data = best_in_class)
```



`ggrepel::geom_label_repel()` keeps labels from overlapping.

2.9.3 Scales, legends and zooming

Default scales (`scale_x_continuous()`, `scale_colour_discrete()`, ...) can be overridden to change

- the **breaks** (tick positions): `scale_y_continuous(breaks = seq(15, 40, by = 5))`;
- the **labels**: `labels = NULL` removes them;

- the **transformation**: `scale_y_log10()`, `scale_x_reverse()`.

Legend: `theme(legend.position = "bottom")` (`"left"`, `"top"`, `"right"`, `"none"`).

Zooming:

- `coord_cartesian(xlim = c(5, 7), ylim = c(10, 30))` zooms *without removing data* (smoother unchanged);
- `xlim(5, 7)`, `scale_x_continuous(limits = c(5, 7))` or filtering drop points (smoother changes).

2.9.4 Themes and saving

Eight built-in themes for non-data elements: `theme_bw()`, `theme_classic()`, `theme_minimal()`, ... `ggsave("my-plot.pdf")` saves the last plot (device size unless `width`, `height` given). Over 30 geoms, more in extensions; see the cheat sheet.

Lecture 3. Data Transformation

3.1 Why transform data

Motivation. dplyr turns each data manipulation into one verb; a chain of verbs answers most descriptive questions.

Data: `flights` from `nycflights13`, all 336 776 flights that left New York City in 2013.

```
library(nycflights13)
library(tidyverse)
```

```
flights
```

```
## # A tibble: 336,776 x 19
##    year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time arr_delay
##   <int> <int> <int>    <int>          <int>     <dbl>    <int>          <int>     <dbl>
## 1  2013     1     1      517            515         2      830            819        11
## 2  2013     1     1      533            529         4      850            830        20
## 3  2013     1     1      542            540         2      923            850        33
## 4  2013     1     1      544            545        -1     1004           1022       -18
## 5  2013     1     1      554            600        -6      812            837       -25
## # i 336,771 more rows
## # i 10 more variables: carrier <chr>, flight <int>, tailnum <chr>, origin <chr>,
## #   dest <chr>, air_time <dbl>, distance <dbl>, hour <dbl>, minute <dbl>,
## #   time_hour <dttm>
```

Column types: `int`, `dbl` (real), `chr`, `dttm` (date-time); later `lgl`, `fctr`, `date`.

3.1.1 The five verbs

verb	action
`filter()`	pick observations (rows) by their values
`arrange()`	reorder rows
`select()`	pick variables (columns) by their names
`mutate()`	create new variables from existing ones
`summarise()`	collapse many values to a single summary

First argument a data frame, then what to do with unquoted variable names; the result is a new data frame. `group_by()` makes every verb act group by group.

3.2 Filter rows with filter()

Motivation. Restrict to the relevant cases, with care for real numbers and missing values.

```
jan1 <- filter(flights, month == 1, day == 1)
jan1
```

```
## # A tibble: 842 x 19
##    year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time arr_delay
##   <int> <int> <int>    <int>          <int>     <dbl>    <int>          <int>     <dbl>
## 1  2013     1     1      517            515         2      830            819        11
## 2  2013     1     1      533            529         4      850            830        20
## 3  2013     1     1      542            540         2      923            850        33
## 4  2013     1     1      544            545        -1     1004           1022       -18
## 5  2013     1     1      554            600        -6      812            837       -25
## # i 837 more rows
## # i 10 more variables: carrier <chr>, flight <int>, tailnum <chr>, origin <chr>,
```

```
## #   dest <chr>, air_time <dbl>, distance <dbl>, hour <dbl>, minute <dbl>,
## #   time_hour <dttm>
```

Commas mean "and". dplyr never modifies its input: save with `<-`; `(jan1 <- ...)` also prints.

3.2.1 Comparisons

`>`, `>=`, `<`, `<=`, `!=`, `==`. `filter(flights, month = 1)` is an error (suggests `month == 1`).

Doubles have finite precision:

```
sqrt(2) ^ 2 == 2
```

```
## [1] FALSE
```

```
1 / 49 * 49 == 1
```

```
## [1] FALSE
```

```
near(sqrt(2) ^ 2, 2)
```

```
## [1] TRUE
```

Compare reals with `near()` (a tolerance), not `==`.

Why $1/49 \cdot 49 \neq 1$ [Supplement]

With a 53-bit mantissa $1/49$ is rounded; multiplying by 49 keeps the error, off by one last bit:

```
print(1 / 49 * 49, digits = 17)
```

```
## [1] 0.99999999999999989
```

`near(x, y)` returns `abs(x - y) < tol` with `tol` about 1.5×10^{-8} (the square root of machine epsilon).

3.2.2 Logical operators

`&` is "and", `|` is "or", `!` is "not"; `xor()` is exclusive or. Flights in November or December:

```
filter(flights, month == 11 | month == 12)
filter(flights, month %in% c(11, 12))      # the same, shorter
```

`filter(flights, month == 11 | 12)` means `(month == 11) | 12`, and `12` is `TRUE`: every flight is kept. By De Morgan, $!(x \,\&\, y) = !x \,|\, !y$, $!(x \,|\, y) = !x \,\&\, !y$, "delayed at most two hours on arrival and departure" is

```
filter(flights, !(arr_delay > 120 | dep_delay > 120))
filter(flights, arr_delay <= 120, dep_delay <= 120)
```

Make complicated conditions explicit variables first. `&&`, `||` take single values (for `if`, Lecture 10).

3.2.3 Missing values

`NA` is an unknown value, and it is contagious:

```
NA > 5
```

```
## [1] NA
```

```
NA + 10
```

```
## [1] NA
```

```
NA == NA
```

```
## [1] NA
```

Two unknown ages may or may not be equal, so `NA == NA` is `NA`; test with `is.na()`.

`filter()` keeps `TRUE` rows, dropping `FALSE` and `NA`; keep `NA` explicitly:

```
df <- tibble(x = c(1, NA, 3))
filter(df, x > 1)
```

```
## # A tibble: 1 x 1
##       x
##   <dbl>
## 1     3
```

```
filter(df, is.na(x) | x > 1)
```

```
## # A tibble: 2 x 1
##       x
##   <dbl>
## 1    NA
## 2     3
```

3.3 Arrange rows with arrange()

`arrange()` sorts by columns, later ones breaking ties; `desc()` reverses. `NA` always goes last.

```
arrange(flights, desc(dep_delay))
```

```
## # A tibble: 336,776 x 19
##    year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time arr_delay
##   <int> <int> <int>    <int>          <int>     <dbl>    <int>          <int>     <dbl>
## 1  2013     1     9      641            900      1301     1242           1530      1272
## 2  2013     6    15     1432           1935      1137     1607           2120      1127
## 3  2013     1    10     1121           1635      1126     1239           1810      1109
## 4  2013     9    20     1139           1845      1014     1457           2210      1007
## 5  2013     7    22      845           1600      1005     1044           1815       989
## # i 336,771 more rows
## # i 10 more variables: carrier <chr>, flight <int>, tailnum <chr>, origin <chr>,
## #   dest <chr>, air_time <dbl>, distance <dbl>, hour <dbl>, minute <dbl>,
## #   time_hour <dttm>
```

```
df <- tibble(x = c(5, 2, NA))
arrange(df, desc(x))
```

```
## # A tibble: 3 x 1
##       x
##   <dbl>
## 1     5
## 2     2
## 3    NA
```

3.4 Select columns with select()

Motivation. Narrow hundreds of variables down to the few that matter.

```
select(flights, year, month, day)          # by name
select(flights, year:day)                  # all columns between year and day
select(flights, -(year:day))               # all except those
select(flights, seq(1, 10, by = 2))        # by position
select(flights, time_hour, air_time, everything())   # move to the front
rename(flights, tail_num = tailnum)        # rename, keep all others
```

Helper functions select by name pattern: `starts_with("abc")`, `ends_with("xyz")`, `contains("ijk")`, `matches("(.)\textbackslash \textbackslash 1")` (a regular expression, Lecture 7) and `num_range("x", 1:3)` for `x1`, `x2`, `x3`.

3.5 Add new variables with mutate()

Motivation. Compute unrecorded quantities (speed, time gained) from existing columns.

`mutate()` adds columns at the end; start narrow to see them:

```
flights_sml <- select(flights, year:day, ends_with("delay"), distance, air_time)
mutate(flights_sml,
  gain = dep_delay - arr_delay,
  hours = air_time / 60,
  gain_per_hour = gain / hours
)
```

```
## # A tibble: 336,776 x 10
##    year month   day dep_delay arr_delay distance air_time  gain hours gain_per_hour
##   <int> <int> <int>     <dbl>     <dbl>    <dbl>    <dbl> <dbl> <dbl>         <dbl>
## 1  2013     1     1         2        11     1400      227    -9  3.78         -2.38
## 2  2013     1     1         4        20     1416      227   -16  3.78         -4.23
## 3  2013     1     1         2        33     1089      160   -31  2.67        -11.6
## 4  2013     1     1        -1       -18     1576      183    17  3.05          5.57
## 5  2013     1     1        -6       -25      762      116    19  1.93          9.83
## # i 336,771 more rows
```

New columns are usable in the same call; `transmute()` keeps only new ones.

3.5.1 Useful creation functions

Must be **vectorised**: vector in, vector of the same length out.

- Arithmetic `+ - * / ^` with recycling: `air_time / 60`, `x / sum(x)`, `y - mean(y)`.
- `%/%` (integer division), `%%` (remainder); times are `HHMM` numbers:

```
transmute(flights, dep_time, hour = dep_time %/% 100, minute = dep_time %% 100)
```

```
## # A tibble: 336,776 x 3
##   dep_time  hour minute
##      <int> <dbl>  <dbl>
## 1      517     5     17
## 2      533     5     33
## 3      542     5     42
## 4      544     5     44
## 5      554     5     54
## # i 336,771 more rows
```

- `log()`, `log2()`, `log10()` for data over orders of magnitude; `log2`: one unit = doubling.
- Offsets `lag()`, `lead()`; cumulative `cumsum()`, `cumprod()`, `cummin()`, `cummax()`, `cummean()`:

```
x <- 1:10
lag(x)
```

```
##  [1] NA  1  2  3  4  5  6  7  8  9
```

```
cumsum(x)
```

```
##  [1]  1  3  6 10 15 21 28 36 45 55
```

```
cummean(x)
```

```
##  [1] 1.0 1.5 2.0 2.5 3.0 3.5 4.0 4.5 5.0 5.5
```

- `min_rank()`: ties get the smallest rank; also `row_number()`, `dense_rank()`, `percent_rank()`, `cume_dist()`, `ntile()`.

```
y <- c(1, 2, 2, NA, 3, 4)
min_rank(y)
```

```
## [1]  1  2  2 NA  4  5
```

```
min_rank(desc(y))
```

```
## [1]  5  3  3 NA  2  1
```

3.6 Grouped summaries with summarise()

Motivation. Summaries compared across groups: group_by() plus summarise().

`summarise()` collapses a data frame to one row:

```
summarise(flights, delay = mean(dep_delay, na.rm = TRUE))
```

```
## # A tibble: 1 x 1
##   delay
##   <dbl>
## 1  12.6
```

`group_by()` changes the unit of analysis to groups; it changes only how verbs act, not the data:

```
by_day <- group_by(flights, year, month, day)
summarise(by_day, delay = mean(dep_delay, na.rm = TRUE))
```

```
## # A tibble: 365 x 4
## # Groups:   year, month [12]
##    year month   day delay
##   <int> <int> <int> <dbl>
## 1  2013     1     1 11.5
## 2  2013     1     2 13.9
## 3  2013     1     3 11.0
## 4  2013     1     4  8.95
## 5  2013     1     5  5.73
## # i 360 more rows
```

Each `summarise()` peels off one grouping level (the message).

3.6.1 The pipe

Distance versus average delay per destination, with intermediate objects:

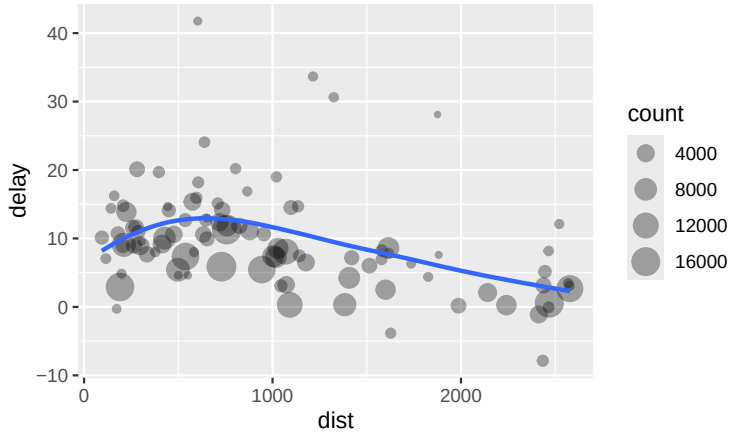
```
by_dest <- group_by(flights, dest)
delay <- summarise(by_dest, count = n(),
  dist = mean(distance, na.rm = TRUE), delay = mean(arr_delay, na.rm = TRUE))
delay <- filter(delay, count > 20, dest != "HNL")
```

The **pipe**: `x %>% f(y)` is `f(x, y)`; read "then":

```
delays <- flights %>%
  group_by(dest) %>%
```

```
  summarise(count = n(),
            dist = mean(distance, na.rm = TRUE),
            delay = mean(arr_delay, na.rm = TRUE)) %>%
  filter(count > 20, dest != "HNL")

ggplot(data = delays, mapping = aes(x = dist, y = delay)) +
  geom_point(aes(size = count), alpha = 1/3) +
  geom_smooth(se = FALSE)
```



Delay rises with distance up to about 750 miles, then falls (time made up in the air?). ggplot2 layers use `+`, not the pipe.

3.6.2 Missing values and counts

Without `na.rm = TRUE` one `NA` makes the summary `NA`. Missing delays are cancelled flights; remove them once:

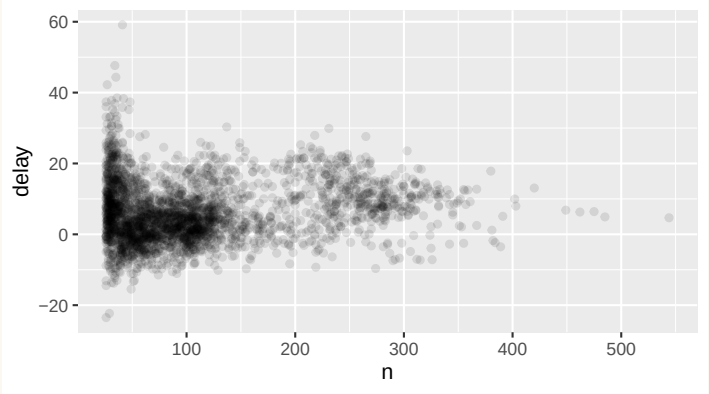
```
not_cancelled <- flights %>% filter(!is.na(dep_delay), !is.na(arr_delay))
```

Always include a count, `n()` or `sum(!is.na(x))`: averages of few observations vary much more.

Small groups have extreme averages [Supplement]

Average delay per plane against its number of flights is a funnel: $\mathrm{Var}(\bar{X}) = \sigma^2/n$, not a property of the planes. Dropping small groups removes the noise:

```
not_cancelled %>%
  group_by(tailnum) %>%
  summarise(delay = mean(arr_delay), n = n()) %>%
  filter(n > 25) %>%
  ggplot(mapping = aes(x = n, y = delay)) +
  geom_point(alpha = 1/10)
```



3.6.3 Useful summary functions

- Location: `mean(x)`, `median(x)`; `mean(arr_delay[arr_delay > 0])` is the average positive delay.
- Spread: `sd(x)`; robust `IQR(x)`, `mad(x)`.
- Rank: `min(x)`, `quantile(x, 0.25)`, `max(x)`.
- Position: `first(x)`, `nth(x, 2)`, `last(x)`.
- Counts: `n()`, `sum(!is.na(x))`, `n_distinct(x)`; `count(dest)` groups and counts; `count(tailnum, wt = distance)` sums a weight (miles per plane).
- Logicals: `TRUE`= 1, `FALSE`= 0, so `sum(x > 10)` counts and `mean(y == 0)` is a proportion.

```
not_cancelled %>%
  group_by(dest) %>%
  summarise(carriers = n_distinct(carrier)) %>%
  arrange(desc(carriers))
```

```
## # A tibble: 104 x 2
##   dest  carriers
##   <chr>    <int>
## 1 ATL          7
## 2 BOS          7
## 3 CLT          7
## 4 ORD          7
## 5 TPA          7
## # i 99 more rows
```

```
not_cancelled %>%
  group_by(year, month, day) %>%
  summarise(n_early = sum(dep_time < 500),     # flights before 5am
            hour_prop = mean(arr_delay > 60))  # share delayed over an hour
```

```
## # A tibble: 365 x 5
## # Groups:   year, month [12]
##    year month   day n_early hour_prop
##   <int> <int> <int>   <int>     <dbl>
## 1  2013     1     1       0    0.0722
## 2  2013     1     2       3    0.0851
## 3  2013     1     3       4    0.0567
## 4  2013     1     4       3    0.0396
## 5  2013     1     5       3    0.0349
## # i 360 more rows
```

`ungroup()` removes grouping.

3.7 Grouped mutates and filters

Under grouping, `rank()`, `n()`, `mean()` in `filter()` and `mutate()` act within groups.

```
flights_sml %>%                          # the worst members of each group
  group_by(year, month, day) %>%
  filter(rank(desc(arr_delay)) < 10)
```

```
## # A tibble: 3,306 x 7
## # Groups:   year, month, day [365]
##    year month   day dep_delay arr_delay distance air_time
##   <int> <int> <int>     <dbl>     <dbl>    <dbl>    <dbl>
## 1  2013     1     1       853       851      184       41
## 2  2013     1     1       290       338     1134      213
## 3  2013     1     1       260       263      266       46
## 4  2013     1     1       157       174      213       60
## 5  2013     1     1       216       222      708      121
## # i 3,301 more rows
```

```
popular_dests <- flights %>%             # all groups bigger than a threshold
  group_by(dest) %>%
  filter(n() > 10000)
nrow(popular_dests)
```

```
## [1] 131440
```

```
popular_dests %>%                        # standardise to per-group metrics
  filter(arr_delay > 0) %>%
  mutate(prop_delay = arr_delay / sum(arr_delay)) %>%
  select(year:day, dest, arr_delay, prop_delay)
```

```
## # A tibble: 50,245 x 6
## # Groups:   dest [9]
##    year month   day dest  arr_delay prop_delay
##   <int> <int> <int> <chr>     <dbl>      <dbl>
## 1  2013     1     1 MIA          33  0.000235
## 2  2013     1     1 ORD          12  0.0000424
## 3  2013     1     1 FLL          19  0.0000938
## 4  2013     1     1 ORD           8  0.0000283
## 5  2013     1     1 LAX           7  0.0000344
## # i 50,240 more rows
```

Lecture 4. Exploratory Data Analysis

4.1 What EDA is

Motivation. EDA has no fixed procedure: each answer suggests the next question.

Exploratory data analysis (EDA) is an iterative cycle:

1. generate questions about the data;
2. search for answers by visualising, transforming and modelling;
3. use what you learn to refine the questions or generate new ones.

Early on, investigate every idea. EDA includes **data cleaning** (same tools).

"There are no routine statistical questions, only questionable statistical routines" (D. R. Cox); "Far better an approximate answer to the right question, which is often vague, than an exact answer to the wrong question, which can always be made precise" (J. W. Tukey). Two questions are always useful:

- What type of **variation** occurs *within* my variables?
- What type of **covariation** occurs *between* my variables?

Definitions. *Variable*: a measurable quantity, quality or property; *value*: its measured state; *observation*: measurements made under similar conditions (same object, same time); *tabular data*: values, each tied to a variable and an observation.

4.2 Variation

Motivation. The pattern of variation is information; plot the distribution.

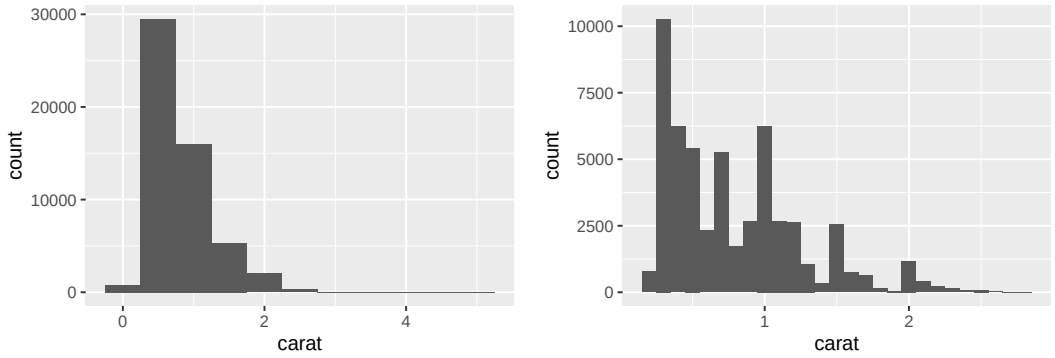
Variation: the tendency of values to change between measurements. The plot depends on the type.

Categorical (few values): bar chart of counts:

```
ggplot(data = diamonds) + geom_bar(mapping = aes(x = cut))
diamonds %>% count(cut)                    # the same numbers
```

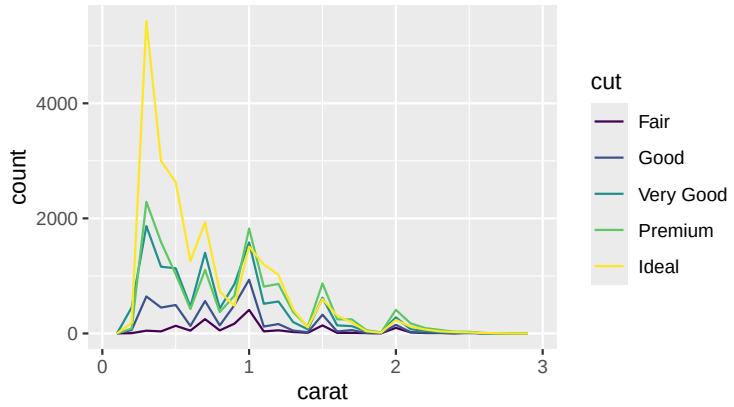
Continuous (numbers, date-times): histogram, counts in equal-width bins:

```
ggplot(data = diamonds) +
  geom_histogram(mapping = aes(x = carat), binwidth = 0.5)
smaller <- diamonds %>% filter(carat < 3)
ggplot(data = smaller, mapping = aes(x = carat)) +
  geom_histogram(binwidth = 0.1)
```



`diamonds %>% count(cut_width(carat, 0.5))` computes the bars. Try several binwidths. `geom_freqpoly()` draws counts as lines, which overlay better:

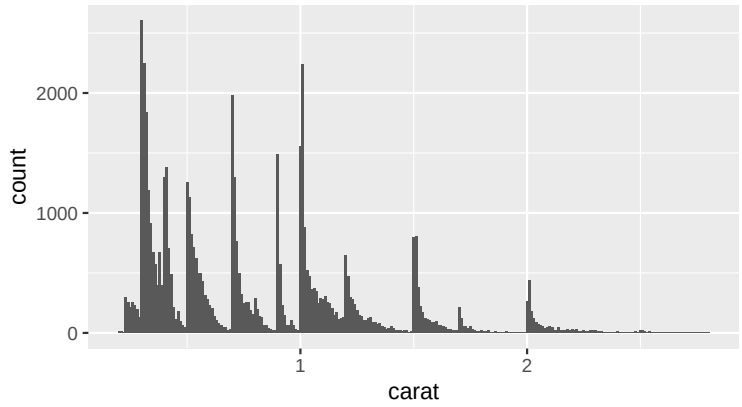
```
ggplot(data = smaller, mapping = aes(x = carat, colour = cut)) +
  geom_freqpoly(binwidth = 0.1)
```



4.2.1 Typical values

Which values are common, which rare, and why? Unusual patterns? Fine binwidth:

```
ggplot(data = smaller, mapping = aes(x = carat)) +
  geom_histogram(binwidth = 0.01)
```

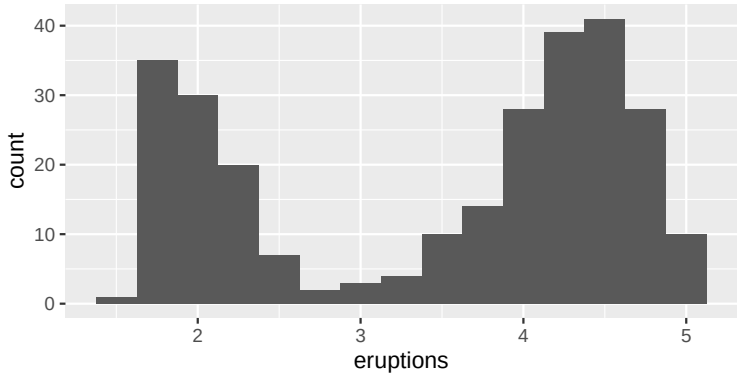


Why peaks at whole carats and common fractions, more mass just right of each peak, nothing above 3 carats? (Cutters aim for round weights, which earn a premium [Supplement].)

4.2.2 Subgroups

Clusters suggest subgroups: how are they alike within, different between, and could they mislead? Old Faithful eruptions are short (about 2 min) or long (4–5 min):

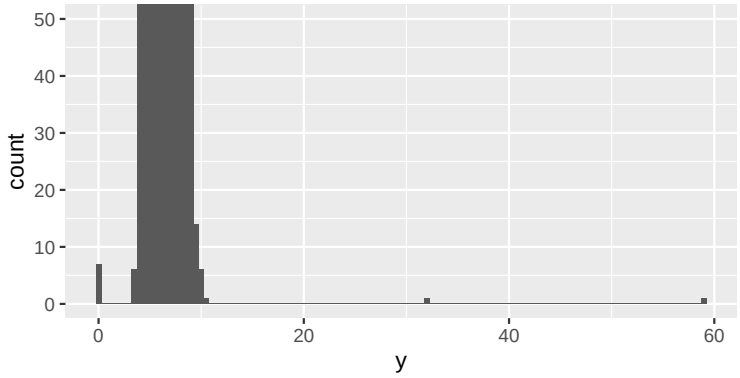
```
ggplot(data = faithful, mapping = aes(x = eruptions)) +
  geom_histogram(binwidth = 0.25)
```



4.2.3 Unusual values

The histogram of width `y` has a suspiciously wide x-axis; `coord_cartesian(ylim = c(0, 50))` (zoom, no data dropped, unlike `ylim()`) reveals values near 0, 30 and 60 mm.

```
ggplot(diamonds) +
  geom_histogram(mapping = aes(x = y), binwidth = 0.5) +
  coord_cartesian(ylim = c(0, 50))
```




```
unusual <- diamonds %>%
  filter(y < 3 | y > 20) %>%
  select(price, x, y, z) %>%
  arrange(y)
unusual
```

```
## # A tibble: 9 x 4
##   price     x     y     z
##   <int> <dbl> <dbl> <dbl>
## 1  5139     0     0     0
## 2  6381     0     0     0
## 3 12800     0     0     0
## 4 15686     0     0     0
## 5 18034     0     0     0
## # i 4 more rows
```

Width 0 is impossible (errors; some have `x`, `y`, `z` all 0); 32 and 59 mm are implausible for stones priced as ordinary. Outliers are data errors or new science:

- repeat the analysis with and without them;
- small effect, no explanation: replace by `NA` and move on;
- substantial effect: find the cause; disclose any removal.

4.3 Missing values

Motivation. Mark bad values missing instead of deleting the rows, which keeps the good measurements.

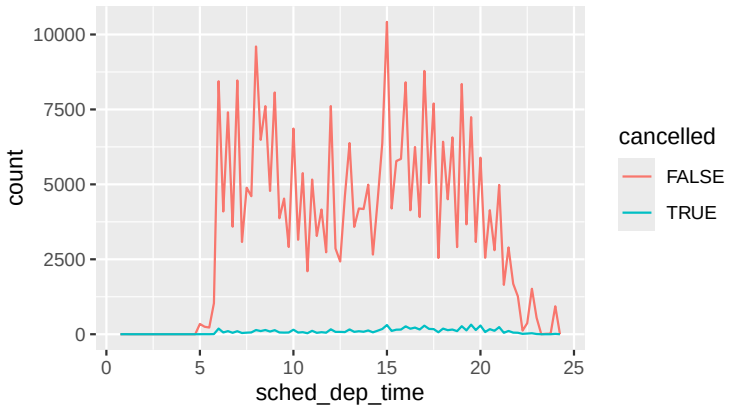
Instead of `filter(between(y, 3, 20))`:

```
diamonds2 <- diamonds %>%
  mutate(y = ifelse(y < 3 | y > 20, NA, y))
```

`ifelse(test, yes, no)` is vectorised: `yes` where `test` is `TRUE`, else `no` (`case_when()` for more cases). ggplot2 drops `NA` with a warning; `na.rm = TRUE` silences it.

Missingness can be informative: missing `dep_time` = cancelled. Scheduled times, cancelled versus not:

```
flights %>%
  mutate(
    cancelled = is.na(dep_time),
    sched_hour = sched_dep_time %/% 100,
    sched_min = sched_dep_time %% 100,
    sched_dep_time = sched_hour + sched_min / 60
  ) %>%
  ggplot(mapping = aes(sched_dep_time)) +
  geom_freqpoly(mapping = aes(colour = cancelled), binwidth = 1/4)
```



Cancelled flights are few, so the comparison is poor (fixed in the next section).

4.4 Covariation

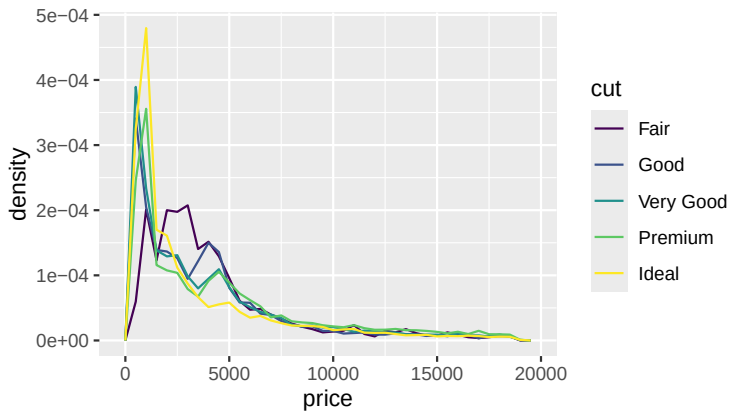
Motivation. How two variables vary together; the plot depends on their types.

Covariation: the tendency of variables to vary together; visualise it.

4.4.1 A categorical and a continuous variable

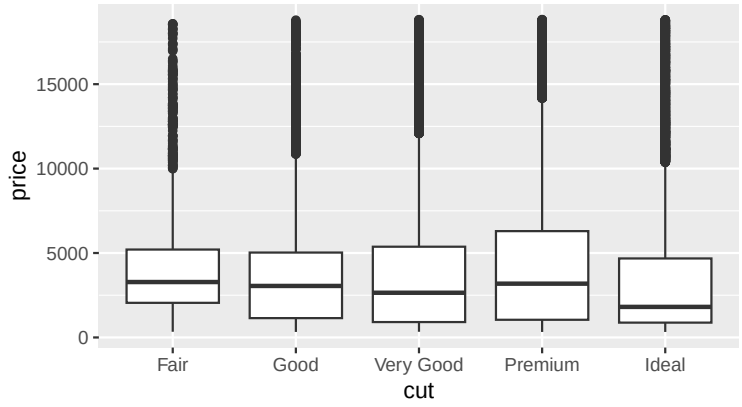
Counts differ greatly between cuts; **density** (area one per polygon) makes shapes comparable:

```
ggplot(data = diamonds, mapping = aes(x = price, y = after_stat(density))) +
  geom_freqpoly(mapping = aes(colour = cut), binwidth = 500)
```



Fair (lowest quality) diamonds seem the most expensive. **Boxplot**: box = IQR with median line, whiskers to the farthest non-outliers, points beyond 1.5×IQR drawn individually.

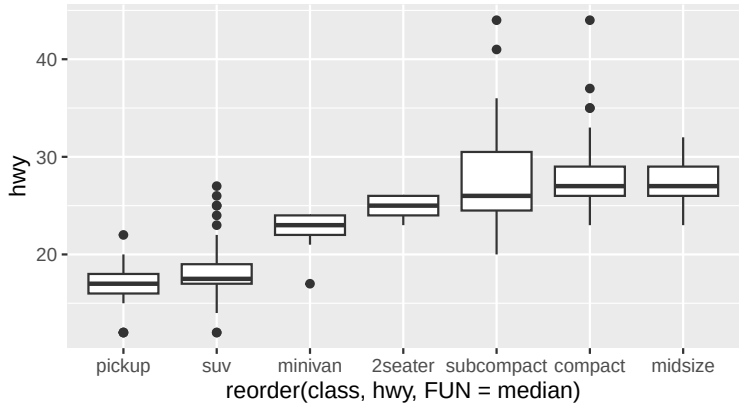
```
ggplot(data = diamonds, mapping = aes(x = cut, y = price)) +
  geom_boxplot()
```



Better diamonds are cheaper on average (explained in Section 4.5).

Reorder unordered categories by a summary:

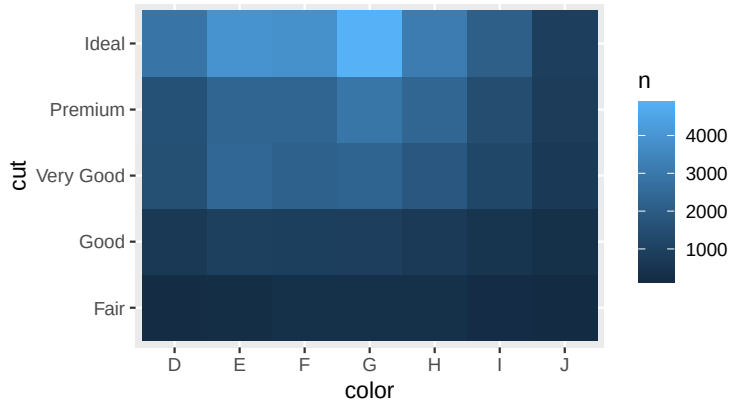
```
ggplot(data = mpg) +
  geom_boxplot(mapping = aes(x = reorder(class, hwy, FUN = median), y = hwy))
```



4.4.2 Two categorical variables

Count each combination: `geom_count()` (circle size $\propto$ count) or a heat map of dplyr counts:

```
diamonds %>%
  count(color, cut) %>%
  ggplot(mapping = aes(x = color, y = cut)) +
  geom_tile(mapping = aes(fill = n))
```

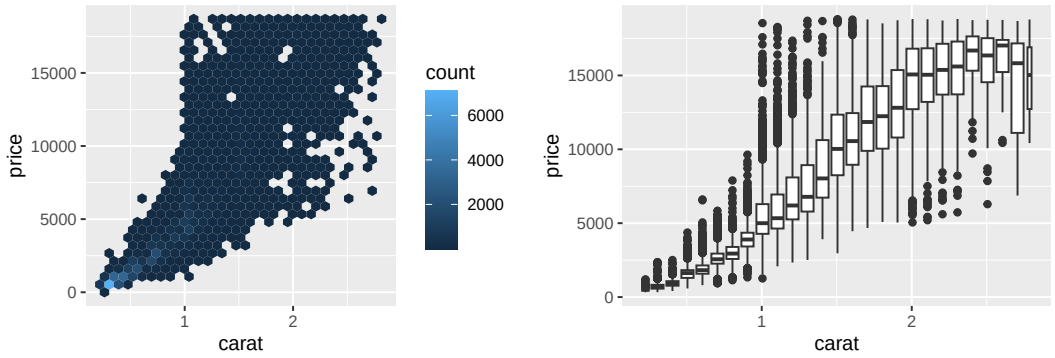


4.4.3 Two continuous variables

54 000 points overplot. Remedies: `alpha = 1/100`; 2-D bins coloured by count, `geom_bin2d()` or `geom_hex()` (needs `hexbin`); or bin one variable to make it categorical:

```
ggplot(data = smaller) +
  geom_hex(mapping = aes(x = carat, y = price))
ggplot(data = smaller, mapping = aes(x = carat, y = price)) +
  geom_boxplot(mapping = aes(group = cut_width(carat, 0.1)))
```

```
## Warning: Orientation is not uniquely specified when both the x and y aesthetics are continuous.
## Picking default orientation 'x'.
```



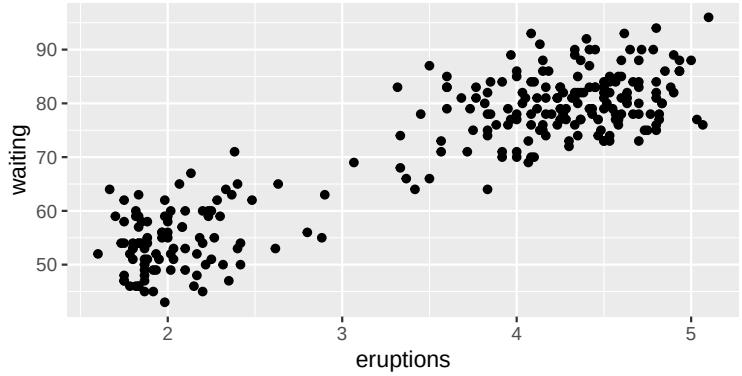
`cut_width(x, w)`: bins of width `w`; `cut_number(x, n)`: `n` bins of about equal size.

4.5 Patterns and models

Motivation. A model removes a strong relationship to reveal weaker ones: the cheap good diamonds.

For a pattern ask: coincidence? How described, how strong? Other variables? Different in subgroups? Old Faithful: longer waits, longer eruptions, two clusters:

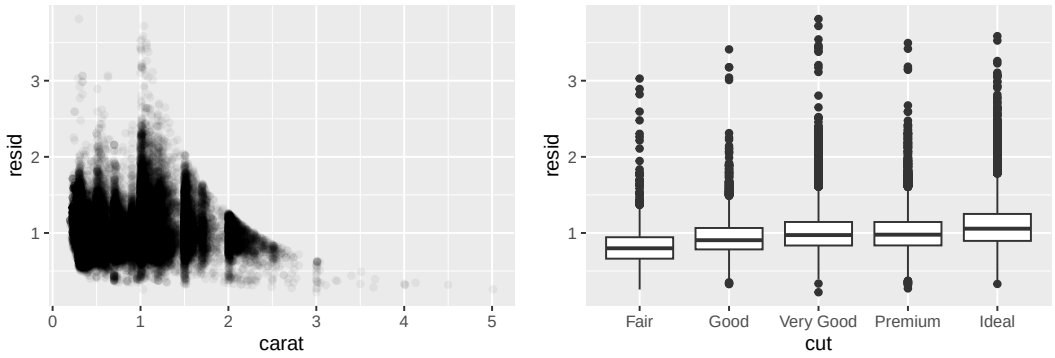
```
ggplot(data = faithful) +
  geom_point(mapping = aes(x = eruptions, y = waiting))
```



Variation creates uncertainty; covariation reduces it (one variable predicts the other). Only causal covariation allows *control*.

Models extract patterns. Price depends strongly on carat, and better cuts are smaller. Fit price on carat and look at the residuals:

```
mod <- lm(log(price) ~ log(carat), data = diamonds)
diamonds2 <- diamonds %>%
  add_residuals(mod) %>%
  mutate(resid = exp(resid))
ggplot(data = diamonds2) +
  geom_point(mapping = aes(x = carat, y = resid), alpha = 1/20)
ggplot(data = diamonds2) +
  geom_boxplot(mapping = aes(x = cut, y = resid))
```



Relative to size, better diamonds are more expensive (models: Lectures 13–14).

Why log–log [Supplement]

$\log(\text{price}) = a_0 + a_1 \log(\text{carat}) \iff \text{price} = e^{a_0}\,\text{carat}^{a_1}$, a power law, linear with roughly constant spread on the log scale. The residual is $\log(\text{price}/\widehat{\text{price}})$, so `exp(resid)` > 1 means dearer than a typical diamond of that size.

```
coef(mod)
```

```
## (Intercept)  log(carat)
##    8.448661    1.675817
```

Lecture 5. Import and Tidy Data

5.1 Tibbles

Motivation. Tibbles are data frames with a few old behaviours removed.

A **tibble** never changes input types (no strings to factors) or names, never creates row names, and complains about missing variables.

```
as_tibble(iris)                          # data frame -> tibble
```

```
## # A tibble: 150 x 5
##   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
##          <dbl>       <dbl>        <dbl>       <dbl> <fct>
## 1          5.1         3.5          1.4         0.2 setosa
## 2          4.9         3            1.4         0.2 setosa
## 3          4.7         3.2          1.3         0.2 setosa
## 4          4.6         3.1          1.5         0.2 setosa
## 5          5           3.6          1.4         0.2 setosa
## # i 145 more rows
```

```
tibble(x = 1:5, y = 1, z = x ^ 2 + y)  # from vectors; y is recycled
```

```
## # A tibble: 5 x 3
##       x     y     z
##   <int> <dbl> <dbl>
## 1     1     1     2
## 2     2     1     5
## 3     3     1    10
## 4     4     1    17
## 5     5     1    26
```

```
tribble(                                 # "transposed tibble": written row by row
  ~x, ~y, ~z,
  "a", 2, 3.6,
  "b", 1, 8.5
)
```

```
## # A tibble: 2 x 3
##   x         y     z
##   <chr> <dbl> <dbl>
## 1 a         2   3.6
## 2 b         1   8.5
```

`as.data.frame()` converts back. Non-syntactic names (`':)'`, `'2000'`) need backticks.

Printing. First 10 rows, columns that fit, with types. More: `print(n = 10, width = Inf)`, `options(tibble.print_max = n, tibble.print_min = m)` (over `n` rows, print `m`), `options(tibble.width = Inf)`.

Subsetting. `$` by name, `[[ ]]` by name or position; no partial matching, a warning for missing columns. In a pipe `.` is the incoming object:

```
df <- tibble(x = runif(3), y = rnorm(3))
df$x
```

```
## [1] 0.08075014 0.83433304 0.60076089
```

```
df[[2]]
```

```
## [1] -1.00599730 -0.08433889 -0.55406480
```

```
df %>% .$x
```

```
## [1] 0.08075014 0.83433304 0.60076089
```

5.2 Data import with readr

Motivation. Turning characters into numbers, dates and factors is guesswork: how readr guesses, and how to correct it.

`readr` turns flat files into tibbles:

- `read_csv()` (comma), `read_csv2()` (semicolon; comma decimal mark), `read_tsv()` (tab), `read_delim()` (any);
- `read_fwf()` (fixed width), `read_table()` (white space), `read_log()` (Apache logs).

Shared arguments. First argument a path (`read_csv("data/heights.csv")`) or an inline string:

```
read_csv("a,b,c
1,2,3
4,5,6")
```

```
## # A tibble: 2 x 3
##       a     b     c
##   <dbl> <dbl> <dbl>
## 1     1     2     3
## 2     4     5     6
```

Useful arguments:

```
read_csv("The first line of metadata
  The second line of metadata
  x,y,z
  1,2,3", skip = 2)                                  # skip n lines
read_csv("# A comment I want to skip
  x,y,z
  1,2,3", comment = "#")                             # drop comment lines
read_csv("1,2,3\n4,5,6", col_names = FALSE)          # no header: X1, X2, X3
read_csv("1,2,3\n4,5,6", col_names = c("x", "y", "z"))
read_csv("a,b,c\n1,2,.", na = ".")                   # symbol for missing values
```

Versus `read.csv()`: faster, tibbles, no factors, same on every OS.

5.2.1 Parsing a vector

`parse_*()` turn character vectors into specialised ones (building blocks of `read_csv()`).

```
str(parse_logical(c("TRUE", "FALSE", "NA")))
```

```
##  logi [1:3] TRUE FALSE NA
```

```
str(parse_date(c("2010-01-01", "1979-10-14")))
```

```
##  Date[1:2], format: "2010-01-01" "1979-10-14"
```

```
parse_integer(c("1", "231", ".", "456"), na = ".")
```

```
## [1]   1 231  NA 456
```

```
x <- parse_integer(c("123", "345", "abc", "123.45"))
```

```
## Warning: 2 parsing failures.
## row col               expected actual
##   3  -- no trailing characters abc
##   4  -- no trailing characters 123.45
```

```
x
```

```
## [1] 123 345  NA  NA
## attr(,"problems")
## # A tibble: 2 x 4
##     row   col expected               actual
##   <int> <int> <chr>                  <chr>
## 1     3    NA no trailing characters abc
## 2     4    NA no trailing characters 123.45
```

```
problems(x)
```

```
## # A tibble: 2 x 4
##     row   col expected               actual
##   <int> <int> <chr>                  <chr>
## 1     3    NA no trailing characters abc
## 2     4    NA no trailing characters 123.45
```

Failures become `NA` with a warning; `problems()` lists them.

Numbers. Decimal marks differ (`.`, `,`); context (`"$1000"`, `"10%"`); grouping marks (`1,000,000`, `1.000.000`, `1'000'000`).

```
parse_double("1,23", locale = locale(decimal_mark = ","))
```

```
## [1] 1.23
```

```
parse_number("It cost $123.45")
```

```
## [1] 123.45
```

```
parse_number("123.456.789", locale = locale(grouping_mark = "."))
```

```
## [1] 123456789
```

`parse_number()` ignores surrounding non-numeric characters.

Strings. Bytes (`charToRaw("Hadley")`) map to characters by an **encoding**: usually UTF-8; Latin-1 or EUC-KR (Korean) must be declared:

```
x2 <- "\xbe\xc8\xb3\xe7\xc7\xcf\xbc\xbc\xbf\xe4"     # bytes of a Korean greeting in EUC
    -KR
parse_character(x2, locale = locale(encoding = "EUC-KR"))
guess_encoding(charToRaw(x2))                        # when the encoding is unknown
```

Factors: `parse_factor(x, levels = fruit)` warns about values outside `levels`.

Dates and times. *Date*: days since 1970-01-01; *date-time*: seconds since 1970-01-01 00:00 UTC; *time*: seconds since midnight.

- `parse_datetime()`: ISO 8601, biggest to smallest component; missing time = midnight.
- `parse_date()`: year, `-` or `/`, month, `-` or `/`, day.
- `parse_time()`: hour `:` minutes, optional `:` seconds and am/pm.

```
parse_datetime("2010-10-01T2010")
```

```
## [1] "2010-10-01 20:10:00 UTC"
```

```
parse_date("2010-10-01")
```

```
## [1] "2010-10-01"
```

```
parse_time("01:10 am")
```

```
## 01:10:00
```

```
parse_date("01/02/15", "%d/%m/%y")
```

```
## [1] "2015-02-01"
```

Format codes: %Y (4-digit year), %y (2-digit), %m, %b (month abbreviation), %d, %H, %M, %S.

5.2.2 Parsing a file

readr guesses from the first 1000 rows, in order: logical (T, F, TRUE, FALSE), integer, double, number, time, date, date-time, else character. It fails if those rows are special (all missing) or NAs abound:

```
challenge <- read_csv(readr_example("challenge.csv"))
problems(challenge)
```

```
## # A tibble: 0 x 5
## # i 5 variables: row <int>, col <int>, expected <chr>, actual <chr>, file <chr>
```

y is all NA in the first 1000 rows, guessed logical. Specify types:

```
challenge <- read_csv(readr_example("challenge.csv"),
  col_types = cols(x = col_double(), y = col_date()))
tail(challenge)
```

```
## # A tibble: 6 x 2
##       x y
##   <dbl> <date>
## 1 0.805 2019-11-21
## 2 0.164 2018-03-29
## 3 0.472 2014-08-04
## 4 0.718 2015-08-16
## 5 0.270 2020-02-04
## # i 1 more row
```

Always give col_types: it documents the data and fails loudly if they change. Alternatives: guess_max = 1001; or read all as character (cols(.default = col_character())) then type_convert().

5.2.3 Writing files and other formats

write_csv(), write_tsv(): UTF-8, ISO 8601 dates, types lost. write_rds()/read_rds(): binary, types kept. Packages: haven (SPSS, Stata, SAS), readxl (.xls, .xlsx), DBI + RSQLite (SQL), jsonlite, xml2.

5.3 Tidy data

Motivation. Only one layout lets the tidyverse work without fighting the data.

table1–table5: tuberculosis cases and population, three countries, 1999 and 2000, in different layouts:

```
table1
```

```
## # A tibble: 6 x 4
##   country      year  cases population
##   <chr>       <dbl>  <dbl>      <dbl>
## 1 Afghanistan  1999    745   19987071
## 2 Afghanistan  2000   2666   20595360
## 3 Brazil       1999  37737  172006362
## 4 Brazil       2000  80488  174504898
## 5 China        1999 212258 1272915272
## # i 1 more row
```

```
table2
```

```
## # A tibble: 12 x 4
##    country      year type           count
##    <chr>       <dbl> <chr>          <dbl>
## 1 Afghanistan  1999 cases            745
## 2 Afghanistan  1999 population 19987071
## 3 Afghanistan  2000 cases           2666
## 4 Afghanistan  2000 population 20595360
## 5 Brazil       1999 cases          37737
## # i 7 more rows
```

```
table3
```

```
## # A tibble: 6 x 3
##    country      year rate
##    <chr>       <dbl> <chr>
## 1 Afghanistan  1999 745/19987071
## 2 Afghanistan  2000 2666/20595360
## 3 Brazil       1999 37737/172006362
## 4 Brazil       2000 80488/174504898
## 5 China        1999 212258/1272915272
## # i 1 more row
```

Definition (Tidy data)

A data set is **tidy** if

1. each variable has its own column;
2. each observation has its own row;
3. each value has its own cell.

Only `table1` is tidy: `table2` spreads an observation over two rows, `table3` has two variables in a cell, `table4a`/`table4b` use `year` values as column names.

Why tidy: one set of tools, and columns suit vectorised functions.

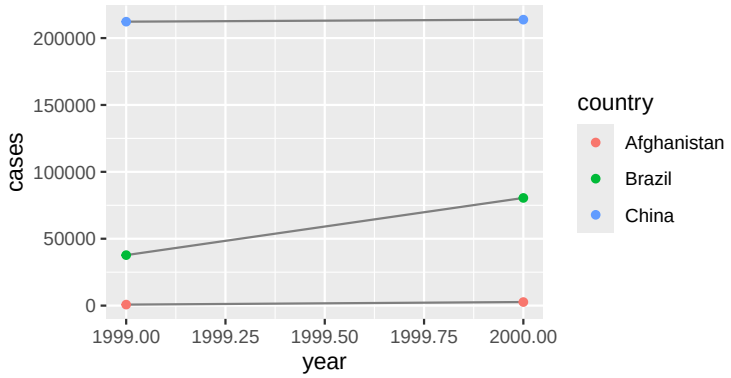
```
table1 %>% mutate(rate = cases / population * 10000)   # rate per 10,000
```

```
## # A tibble: 6 x 5
##    country      year  cases population  rate
##    <chr>       <dbl>  <dbl>      <dbl> <dbl>
## 1 Afghanistan  1999    745   19987071 0.373
## 2 Afghanistan  2000   2666   20595360 1.29
## 3 Brazil       1999  37737  172006362 2.19
## 4 Brazil       2000  80488  174504898 4.61
## 5 China        1999 212258 1272915272 1.67
## # i 1 more row
```

```
table1 %>% count(year, wt = cases)                      # cases per year
```

```
## # A tibble: 2 x 2
##    year      n
##   <dbl>  <dbl>
## 1  1999 250740
## 2  2000 296920
```

```
ggplot(table1, aes(year, cases)) +
  geom_line(aes(group = country), colour = "grey50") +
  geom_point(aes(colour = country))
```



5.4 Gathering and spreading

Motivation. A variable spread over columns, or an observation over rows: two inverse fixes.

Gathering. In `table4a` the names `1999`, `2000` are values of `year`. `gather()` makes a **key** (former column names) and a **value** (former cells).

```
table4a
```

```
## # A tibble: 3 x 3
##   country     `1999` `2000`
##   <chr>        <dbl>  <dbl>
## 1 Afghanistan    745   2666
## 2 Brazil       37737  80488
## 3 China       212258 213766
```

```
tidy4a <- table4a %>% gather(`1999`, `2000`, key = "year", value = "cases")
tidy4b <- table4b %>% gather(`1999`, `2000`, key = "year", value = "population")
left_join(tidy4a, tidy4b)
```

```
## # A tibble: 6 x 4
##   country     year   cases population
##   <chr>       <chr>  <dbl>      <dbl>
## 1 Afghanistan 1999     745   19987071
## 2 Brazil      1999   37737  172006362
## 3 China       1999  212258 1272915272
## 4 Afghanistan 2000    2666   20595360
## 5 Brazil      2000   80488  174504898
## # i 1 more row
```

`left_join()`: Lecture 6. Gathering makes tables longer.

Spreading is the inverse (wider): `spread()` turns values of `type` into columns filled from `count`.

```
spread(table2, key = type, value = count)
```

```
## # A tibble: 6 x 4
##   country      year  cases population
##   <chr>       <dbl>  <dbl>      <dbl>
## 1 Afghanistan  1999    745   19987071
## 2 Afghanistan  2000   2666   20595360
## 3 Brazil       1999  37737  172006362
## 4 Brazil       2000  80488  174504898
## 5 China        1999 212258 1272915272
## # i 1 more row
```

pivot_longer() and pivot_wider() [Supplement]

Current tidyr supersedes them by `pivot_longer()`, `pivot_wider()` (clearer arguments):

```
table4a %>% pivot_longer(c(`1999`, `2000`), names_to = "year", values_to = "cases")
table2 %>% pivot_wider(names_from = type, values_from = count)
```

`names_transform`, `values_drop_na` handle the type of `year` and missing cells.

5.5 Separating and uniting

`table3`'s `rate` holds two variables; `separate()` splits at any non-alphanumeric character:

```
table3 %>% separate(rate, into = c("cases", "population"), convert = TRUE)
```

```
## # A tibble: 6 x 4
##   country      year  cases population
##   <chr>       <dbl>  <int>      <int>
## 1 Afghanistan  1999    745   19987071
## 2 Afghanistan  2000   2666   20595360
## 3 Brazil       1999  37737  172006362
## 4 Brazil       2000  80488  174504898
## 5 China        1999 212258 1272915272
## # i 1 more row
```

Without `convert = TRUE` they stay character. Numeric `sep` is a position: `sep = 2` splits after character 2.

`unite()` is the inverse; `table5` has century and year split:

```
table5 %>% unite(new, century, year, sep = "")
```

```
## # A tibble: 6 x 3
##   country     new   rate
##   <chr>       <chr> <chr>
## 1 Afghanistan 1999  745/19987071
## 2 Afghanistan 2000  2666/20595360
## 3 Brazil      1999  37737/172006362
## 4 Brazil      2000  80488/174504898
## 5 China       1999  212258/1272915272
## # i 1 more row
```

(default separator `_`).

Lecture 6. Relational Data

6.1 Relational data

Motivation. Joins combine tables through shared variables, e.g. delays with weather.

Relational data: several tables, related in *pairs*. Besides `flights`, `nycflights13` has

- `airlines`: carrier names by code;
- `airports`: airports by `faa` code;
- `planes`: planes by `tailnum`;
- `weather`: hourly weather at each NYC airport.

`flights` links to `planes` by `tailnum`, `airlines` by `carrier`, `airports` by `origin`, `dest`, and `weather` by `origin`, `year`, `month`, `day`, `hour`.

6.1.1 Keys

A **key** is a variable (set) that uniquely identifies an observation.

- **Primary key**: in its own table, e.g. `planes$tailnum`.
- **Foreign key**: points to another table, e.g. `flights$tailnum`.

Verify uniqueness:

```
planes %>% count(tailnum) %>% filter(n > 1)
```

```
## # A tibble: 0 x 2
## # i 2 variables: tailnum <chr>, n <int>
```

```
weather %>% count(year, month, day, hour, origin) %>% filter(n > 1)
```

```
## # A tibble: 3 x 6
##    year month   day  hour origin     n
##   <int> <int> <int> <int> <chr>  <int>
## 1  2013    11     3     1 EWR        2
## 2  2013    11     3     1 JFK        2
## 3  2013    11     3     1 LGA        2
```

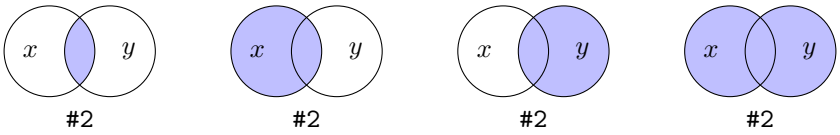
A few duplicated hours (daylight saving): `weather` is not perfectly keyed [Supplement]. Primary + foreign key = a **relation**, typically one-to-many.

6.2 Mutating joins

Motivation. Add columns from another table, matching rows by key.

Two small tables:

```
x <- tribble(~key, ~val_x,  1, "x1",  2, "x2",  3, "x3")
y <- tribble(~key, ~val_y,  1, "y1",  2, "y2",  4, "y3")
```



Inner join: pairs with equal keys; unmatched rows silently dropped.

```
x %>% inner_join(y, by = "key")
```

```
## # A tibble: 2 x 3
##     key val_x val_y
##   <dbl> <chr> <chr>
## 1     1 x1    y1
## 2     2 x2    y2
```

Outer joins keep rows in at least one table, filling `NA`:

- `left_join(x, y)` keeps all rows of `x`;
- `right_join(x, y)` keeps all rows of `y`;
- `full_join(x, y)` keeps all rows of both.

```
left_join(x, y, by = "key")
```

```
## # A tibble: 3 x 3
##     key val_x val_y
##   <dbl> <chr> <chr>
## 1     1 x1    y1
## 2     2 x2    y2
## 3     3 x3    <NA>
```

```
full_join(x, y, by = "key")
```

```
## # A tibble: 4 x 3
##     key val_x val_y
##   <dbl> <chr> <chr>
## 1     1 x1    y1
## 2     2 x2    y2
## 3     3 x3    <NA>
## 4     4 <NA>  y3
```

The left join is the usual lookup: no original observation is lost.

6.2.1 Duplicate keys

Duplicates in one table (one-to-many): the other row is copied to every match:

```
x <- tribble(~key, ~val_x,  1, "x1",  2, "x2",  2, "x3",  1, "x4")
y <- tribble(~key, ~val_y,  1, "y1",  2, "y2")
left_join(x, y, by = "key")
```

```
## # A tibble: 4 x 3
##     key val_x val_y
##   <dbl> <chr> <chr>
## 1     1 x1    y1
## 2     2 x2    y2
## 3     2 x3    y2
## 4     1 x4    y1
```

Duplicates in both: all combinations within each key, usually an error (dplyr warns of many-to-many).

```
x <- tribble(~key, ~val_x,  1, "x1",  2, "x2",  2, "x3",  3, "x4")
```

```
y <- tribble(~key, ~val_y,  1, "y1",  2, "y2",  2, "y3",  3, "y4")
left_join(x, y, by = "key")
```

```
## Warning in left_join(x, y, by = "key"): Detected an unexpected many-to-many relationship
    between `x` and `y`.
## i Row 2 of `x` matches multiple rows in `y`.
## i Row 2 of `y` matches multiple rows in `x`.
## i If a many-to-many relationship is expected, set `relationship = "many-to-many"` to
##   silence this warning.
```

```
## # A tibble: 6 x 3
##     key val_x val_y
##   <dbl> <chr> <chr>
## 1     1 x1    y1
## 2     2 x2    y2
## 3     2 x2    y3
## 4     2 x3    y2
## 5     2 x3    y3
## # i 1 more row
```

6.2.2 Defining the key columns

A narrow table shows the added columns:

```
flights2 <- flights %>% select(year:day, hour, origin, dest, tailnum, carrier)
```

- `by = NULL` (**natural join**): all common variables, reported:

```
flights2 %>% left_join(weather)
```

```
## # A tibble: 336,776 x 18
##    year month   day  hour origin dest  tailnum carrier  temp  dewp humid wind_dir
##   <int> <int> <int> <dbl> <chr>  <chr> <chr>   <chr>   <dbl> <dbl> <dbl>    <dbl>
## 1  2013     1     1     5 EWR    IAH   N14228  UA       39.0  28.0  64.4      260
## 2  2013     1     1     5 LGA    IAH   N24211  UA       39.9  25.0  54.8      250
## 3  2013     1     1     5 JFK    MIA   N619AA  AA       39.0  27.0  61.6      260
## 4  2013     1     1     5 JFK    BQN   N804JB  B6       39.0  27.0  61.6      260
## 5  2013     1     1     6 LGA    ATL   N668DN  DL       39.9  25.0  54.8      260
## # i 336,771 more rows
## # i 6 more variables: wind_speed <dbl>, wind_gust <dbl>, precip <dbl>, pressure <dbl>,
## #   visib <dbl>, time_hour <dttm>
```

- `by = "tailnum"`: that variable only. Both tables have `year` with different meanings (flight, manufacture), so a natural join is wrong; here they become `year.x`, `year.y`.
- `by = c("dest" = "faa")`: different names in the two tables; `c("origin" = "faa")` would add the origin airport.

```
flights2 %>% left_join(airports, c("dest" = "faa")) %>% select(dest, name, lat, lon)
```

```
## # A tibble: 336,776 x 4
##   dest  name                           lat   lon
##   <chr> <chr>                        <dbl> <dbl>
## 1 IAH   George Bush Intercontinental   30.0 -95.3
## 2 IAH   George Bush Intercontinental   30.0 -95.3
## 3 MIA   Miami Intl                     25.8 -80.3
## 4 BQN   <NA>                           NA    NA
## 5 ATL   Hartsfield Jackson Atlanta Intl 33.6 -84.4
## # i 336,771 more rows
```

6.3 Filtering joins

Motivation. Keep or drop rows by whether they match, adding no columns.

Top ten destinations:

```
top_dest <- flights %>% count(dest, sort = TRUE) %>% head(10)
```

- `semi_join(x, y)`: rows of `x` *with* a match; any number of keys, unlike `filter(dest %in% top_dest$dest)`.
- `anti_join(x, y)`: rows *without* a match; diagnoses mismatches.

```
semi_join(flights, top_dest, by = "dest") %>% nrow()
```

```
## [1] 141145
```

```
flights %>%
  anti_join(planes, by = "tailnum") %>%
  count(tailnum, sort = TRUE)
```

```
## # A tibble: 722 x 2
##   tailnum     n
##   <chr>   <int>
## 1 <NA>     2512
## 2 N725MQ    575
## 3 N722MQ    513
## 4 N723MQ    507
## 5 N713MQ    483
## # i 717 more rows
```

Unmatched tail numbers are mostly AA and MQ, which report fleet numbers [Supplement].

Checking a join [Supplement]

Identify the primary key from the data's meaning; check no key is missing; check foreign keys with `anti_join()`; if a left join adds rows, the key was not unique.

6.4 Set operations

Whole rows, all variables compared:

```
df1 <- tribble(~x, ~y,  1, 1,  2, 1)
df2 <- tribble(~x, ~y,  1, 1,  1, 2)
intersect(df1, df2)   # rows in both
```

```
## # A tibble: 1 x 2
##       x     y
##   <dbl> <dbl>
## 1     1     1
```

```
union(df1, df2)       # unique rows in either
```

```
## # A tibble: 3 x 2
##       x     y
##   <dbl> <dbl>
## 1     1     1
## 2     2     1
## 3     1     2
```

```
setdiff(df1, df2)     # rows in df1 but not in df2
```

```
## # A tibble: 1 x 2
##       x     y
##   <dbl> <dbl>
## 1     2     1
```

```
setdiff(df2, df1)
```

```
## # A tibble: 1 x 2
##       x     y
##   <dbl> <dbl>
## 1     1     2
```

6.5 Selected exercises

Motivation. Keys, mismatches, and when a join silently duplicates rows.

Exercise 6.1 (R4DS Exercises 13.2.1.2 and 13.2.1.3)

(a) What is the relation between `weather` and `airports`?
(b) `weather` only contains information for the three NYC origin airports. If it contained weather records for all airports in the USA, what additional relation would it define with `flights`?

Solution.

(a) Foreign key `weather$origin` → primary key `airports$faa`; one-to-many (many hourly records per airport).

(b) Destination weather at arrival: `flights$dest` to `weather$origin` with `year`, `month`, `day` and the hour of `arr_time`, a second relation besides `origin` and departure hour. ■

Exercise 6.2 (R4DS Exercise 13.5.1.5)

What do the following two anti joins tell you?

```
anti_join(flights, airports, by = c("dest" = "faa")) %>% count(dest)
```

```
## # A tibble: 4 x 2
##   dest      n
##   <chr> <int>
## 1 BQN     896
## 2 PSE     365
## 3 SJU    5819
## 4 STT     522
```

```
anti_join(airports, flights, by = c("faa" = "dest")) %>% nrow()
```

```
## [1] 1357
```

Solution. First: flights whose destination is not in `airports`: BQN, PSE, SJU (Puerto Rico) and STT (US Virgin Islands). Second: 1357 airports received no NYC flight in 2013. So `flights$dest` is not fully matched, while unused rows of a lookup table are normal. ■

Exercise 6.3 (how many rows a join returns)

For a key value k let a_k and b_k be the numbers of rows of `x` and `y` with that key.
(a) Show that `left_join(x, y)` has $\sum_k a_k \max(b_k, 1)$ rows, and hence exactly as many rows as `x` if and only if no key value of `x` occurs more than once in `y`.

(b) Use (a) to explain the check below.

```
planes %>% count(tailnum) %>% filter(n > 1) %>% nrow()
## [1] 0

nrow(left_join(flights, planes, by = "tailnum")) == nrow(flights)
## [1] TRUE
```

Solution [Supplement].

(a) One row per matching pair, one `NA` row per unmatched row of `x`: key k gives $a_k b_k$ rows if $b_k \geq 1$, a_k if $b_k = 0$, i.e. $a_k \max(b_k, 1)$; sum over k. The excess over $\sum_k a_k$ is $\sum_k a_k\{\max(b_k, 1) - 1\} \geq 0$, zero iff $b_k \leq 1$ for every key of `x`.

(b) No tail number repeats in `planes` (a primary key), so by (a) the row count is kept, as line two confirms; a duplicate would silently copy that plane's flights. ■

Lecture 7. Strings

7.1 String basics

Motivation. Information packed into text is pulled out with stringr and regular expressions.

`stringr` (core tidyverse): consistent `str_*` functions.

Double or single quotes (single when the string contains double quotes):

```
string1 <- "This is a string"
string2 <- 'If I want to include a "quote" inside a string, I use single quotes'
```

A missing closing quote gives the `+` prompt; press Esc.

Escapes. `"\""` is a double quote, `"\\"` a backslash. Printing shows escapes; `writeLines()` the raw contents:

```
x <- c("\"", "\\")
x
```

```
## [1] "\"" "\\"
```

```
writeLines(x)
```

```
## "
## \
```

Also `"\n"` (newline), `"\t"` (tab); see `?"'"`. Unicode: `"\u"` + code, e.g. `"\uc804\uc0b0\ud1b5\uacc4"` (course name in Hangul). Character vector: `c("one", "two", "three")`.

7.1.1 Length, combining, subsetting

```
str_length(c("a", "R for data science", NA))
```

```
## [1]  1 18 NA
```

```
str_c("x", "y", sep = ", ")
```

```
## [1] "x, y"
```

```
str_c("prefix-", c("a", "b", "c"), "-suffix")     # vectorised, recycles
```

```
## [1] "prefix-a-suffix" "prefix-b-suffix" "prefix-c-suffix"
```

```
str_c(c("x", "y", "z"), collapse = ", ")          # collapse a vector into one string
```

```
## [1] "x, y, z"
```

`str_c()` drops length-0 objects: in `str_c("Good ", time_of_day, " ", name, if (birthday) " and HAPPY BIRTHDAY", ".")` the clause vanishes when `birthday` is `FALSE`. `NA` is contagious; `str_replace_na()` prints `"NA"`.

`str_sub(x, start, end)`: inclusive positions, negative from the end, out of range allowed:

```
x <- c("Apple", "Banana", "Pear")
str_sub(x, 1, 3)
```

```
## [1] "App" "Ban" "Pea"
```

```
str_sub(x, -3, -1)
```

```
## [1] "ple" "ana" "ear"
```

```
str_sub("a", 1, 5)
```

```
## [1] "a"
```

```r
str_sub(x, 1, 1) <- str_to_lower(str_sub(x, 1, 1))   # assignment form
x
```

```
## [1] "apple"  "banana" "pear"
```

Positions count characters, not bytes (fine for Hangul).

7.2 Regular expressions

Motivation. A terse language for patterns such as "any digit" or "a repeated letter".

`str_view()` marks matches with `< >` (`str_view_all()` is deprecated).

7.2.1 Basic matches

Exact strings; `.` is any character but newline:

```r
x <- c("apple", "banana", "pear")
str_view(x, "an")
```

```
## [2] | b<an><an>a
```

```r
str_view(x, ".a.")
```

```
## [2] | <ban>ana
## [3] | p<ear>
```

A literal `.` is the regex `\.`, written as the string `"\\."`; a literal backslash is the regex `\\`, the string `"\\\\"`:

```r
str_view(c("abc", "a.c", "bef"), "a\\.c")
```

```
## [2] | <a.c>
```

```r
writeLines("a\\b")
```

```
## a\b
```

```r
str_view("a\\b", "\\\\")
```

```
## [1] | a<\>b
```

7.2.2 Anchors and character classes

`^` start, `$` end ("begin with power, end up with money"); both force a complete match:

```r
x <- c("apple pie", "apple", "apple cake", "pine apple")
str_view(x, "^apple$")
```

```
## [2] | <apple>
```

`\d` digit, `\s` whitespace, `[abc]` one of a, b, c, `[^abc]` none of them; in strings `"\\d"` etc. Alternation `|`, delimited by parentheses:

```r
str_view(c("grey", "gray"), "gr(e|a)y")
```

```
## [1] | <grey>
## [2] | <gray>
```

7.2.3 Repetition

`?` means 0 or 1, `+` 1 or more, `*` 0 or more; `{n}` exactly n, `{n,}` n or more, `{,m}` at most m, `{n,m}` between n and m. **Greedy** (longest match) by default; a trailing `?` makes it **lazy**.

```
x <- "1888 is the longest year in Roman numerals: MDCCCLXXXVIII"
str_view(x, "CC+")
```
```
## [1] | 1888 is the longest year in Roman numerals: MD<CCC>LXXXVIII
```

```
str_view(x, "C[LX]+")
```
```
## [1] | 1888 is the longest year in Roman numerals: MDCC<CLXXX>VIII
```

```
str_view(x, "C{2,3}")
```
```
## [1] | 1888 is the longest year in Roman numerals: MD<CCC>LXXXVIII
```

```
str_view(x, "C{2,3}?")
```
```
## [1] | 1888 is the longest year in Roman numerals: MD<CC>CLXXXVIII
```

7.2.4 Grouping and backreferences

Parentheses define **groups**, referred to as `\1`, `\2`, ... Fruits with a repeated pair:

```
str_subset(fruit, "(..)\\1")
```
```
## [1] "banana"      "coconut"     "cucumber"    "jujube"      "papaya"      "salal berry"
```

7.3 Tools

Motivation. Detect, extract, replace, split: one function each.

7.3.1 Detecting matches

`str_detect()` is logical, so `sum()`, `mean()` count; `words`: 980 common words.

```
str_detect(c("apple", "banana", "pear"), "e")
```
```
## [1]  TRUE FALSE  TRUE
```

```
sum(str_detect(words, "^t"))          # how many words start with t?
```
```
## [1] 65
```

```
mean(str_detect(words, "[aeiou]$"))   # proportion ending with a vowel
```
```
## [1] 0.2765306
```

```
str_subset(words, "x$")               # same as words[str_detect(words, "x$")]
```
```
## [1] "box" "sex" "six" "tax"
```

In a data frame: `filter(str_detect(word, "x$"))`. `str_count()` counts matches:

```
tibble(word = words) %>%
  mutate(vowels = str_count(word, "[aeiou]"),
         consonants = str_count(word, "[^aeiou]"))
```
```
## # A tibble: 980 x 3
##   word     vowels consonants
##   <chr>     <int>      <int>
## 1 a             1          0
## 2 able          2          2
## 3 about         3          2
## 4 absolute      4          4
## 5 accept        2          4
## # i 975 more rows
```

Several `str_detect()` calls with logical operators beat one complex regex: no vowels is `!str_detect(words, "[aeiou]")`.

7.3.2 Extracting matches

`str_extract()`: first match (`NA` if none); `str_extract_all()`: all, as a list (matrix with `simplify = TRUE`). Colours in the 720 `sentences`:

```
colours <- c("red", "orange", "yellow", "green", "blue", "purple")
colour_match <- str_c(colours, collapse = "|")
has_colour <- str_subset(sentences, colour_match)
head(str_extract(has_colour, colour_match))
```

```
## [1] "blue" "blue" "red"  "red"  "red"  "blue"
```

(It matches "red" in "flickered"; word boundaries `\b` fix that [Supplement].)

7.3.3 Grouped matches

`str_match()`: full match and groups as matrix columns. Noun phrase, "a"/"the" + word:

```
noun <- "(a|the) ([^ ]+)"
sentences %>% str_subset(noun) %>% head(4) %>% str_match(noun)
```

```
##      [,1]         [,2]  [,3]
## [1,] "the smooth" "the" "smooth"
## [2,] "the sheet"  "the" "sheet"
## [3,] "the depth"  "the" "depth"
## [4,] "a chicken"  "a"   "chicken"
```

For tibbles, `tidyr::extract()` names the groups as columns:

```
tibble(sentence = sentences) %>%
  extract(sentence, c("article", "noun"), "(a|the) ([^ ]+)", remove = FALSE)
```

```
## # A tibble: 720 x 3
##   sentence                                  article noun
##   <chr>                                     <chr>   <chr>
## 1 The birch canoe slid on the smooth planks. the     smooth
## 2 Glue the sheet to the dark blue background. the     sheet
## 3 It's easy to tell the depth of a well.      the     depth
## 4 These days a chicken leg is a rare dish.    a       chicken
## 5 Rice is often served in round bowls.        <NA>    <NA>
## # i 715 more rows
```

7.3.4 Replacing and splitting

`str_replace()`: first match, `str_replace_all()`: all; a named vector does several, backreferences reorder:

```
x <- c("1 house", "2 cars", "3 people")
str_replace_all(x, c("1" = "one", "2" = "two", "3" = "three"))
```

```
## [1] "one house"   "two cars"    "three people"
```

```
sentences %>%
  str_replace("([^ ]+) ([^ ]+) ([^ ]+)", "\\1 \\3 \\2") %>%   # swap 2nd and 3rd words
  head(3)
```

```
## [1] "The canoe birch slid on the smooth planks."
## [2] "Glue sheet the to the dark blue background."
## [3] "It's to easy tell the depth of a well."
```

`str_split()` returns a list (piece counts differ); `boundary("word")` splits on words:

```
"a|b|c|d" %>% str_split("\\|")
```

```
## [[1]]
## [1] "a" "b" "c" "d"
```

```
x <- "This is a sentence.  This is another sentence."
str_split(x, boundary("word"))[[1]]
```

```
## [1] "This"     "is"       "a"        "sentence" "This"     "is"       "another"
## [8] "sentence"
```

`str_locate()`: start and end positions, for `str_sub()`.

7.4 Selected exercises

Motivation. A regex is read twice, by R's string parser and by the regex engine.

Exercise 7.1 (R4DS Exercise 14.3.1.1.1)

Explain why each of the strings `"\"`, `"\\"` and `"\\\"` fails to match a literal backslash.

```
x <- "a\\b"
writeLines(x)
```

```
## a\b
```

```
str_detect(x, "\\")
```

```
## Error in stri_detect_regex(string, pattern, negate = negate, opts_regex = opts(pattern)):
     Unrecognized backslash escape sequence in pattern. (U_REGEX_BAD_ESCAPE_SEQUENCE,
    context=`\`)
```

```
str_detect(x, "\\\\")
```

```
## [1] TRUE
```

Solution.

Step 1. `"\"`: the backslash escapes the closing quote; the string never ends (`+` prompt).

Step 2. `"\\"`: a valid one-backslash string, but as a regex an escape with nothing to escape: error.

Step 3. `"\\\"`: a backslash, then an escaped closing quote: unterminated.

Step 4. The regex is `\\`; escaping both characters in a string gives `"\\\\"`, which matches. ■

Exercise 7.2 (R4DS Exercise 14.3.5.1.1)

Describe in words what these expressions match: `(.)\1\1`, `"(.)(.)\\2\\1"`, `(..)\1`, `"(.).\\1.\\1"`, `"(.)(.)(.).*\\3\\2\\1"`. Check the answers on suitable strings.

Solution. Expressions 1 and 3 are regexes; the others are R strings (doubled backslashes become single). As regexes:

Step 1. `(.)\1\1`: one character three times, "aaa".

Step 2. `(.)(.)\2\1`: two characters then the same reversed, "abba".

Step 3. `(..)\1`: a repeated pair, "abab".

Step 4. `(.).\1.\1`: a, any, a, any, a: "abaca".

Step 5. `(.)(.)(.).*\3\2\1`: three characters, anything (possibly nothing), the three reversed: "abcxcba", "abccba".

```
x <- c("aaa", "abba", "abab", "abaca", "abcxcba")
pats <- c(p1 = "(.)\\1\\1", p2 = "(.)(.)\\2\\1", p3 = "(..)\\1",
          p4 = "(.).\\1.\\1", p5 = "(.)(.)(.).*\\3\\2\\1")
sapply(pats, function(p) str_detect(x, p)) %>% `rownames<-`(x)
```

```
##            p1    p2    p3    p4    p5
## aaa      TRUE FALSE FALSE FALSE FALSE
## abba    FALSE  TRUE FALSE FALSE FALSE
## abab    FALSE FALSE  TRUE FALSE FALSE
## abaca   FALSE FALSE FALSE  TRUE FALSE
## abcxcba FALSE FALSE FALSE FALSE  TRUE
```

Diagonal: each string matches only its own pattern ("abba" fails `p5`, needing six characters; "aaa" fails `p3`, needing four). ■

Exercise 7.3 (one pattern or several; R4DS Exercise 14.4.1.1.1)

(a) Prove that for every string, `str_detect(s, "^x|x$")` equals `str_detect(s, "^x") | str_detect(s, "x$")`.
(b) Prove that `str_detect(s, "^[aeiou].*[^aeiou]$")` equals `str_detect(s, "^[aeiou]") & str_detect(s, "[^aeiou]$")` for every string s without newlines.
(c) Check both on `words`.

Solution [Supplement].

(a) s matches `A|B` iff it contains a match of `A` or of `B`; here "x at the start" or "x at the end": the same condition.

(b)

Step 1. If s matches the pattern, its first character is a vowel and its last is not: both tests `TRUE`.

Step 2. Conversely, a vowel first and a non-vowel last are different characters, so $n \geq 2$; `^[aeiou]` takes character 1, `.*` characters $2, \ldots, n-1$ (possibly none; no newlines), `[^aeiou]$` character n.

Step 3. If either test fails, the pattern fails by Step 1; the sides agree for every s.

(c)

```
a1 <- str_detect(words, "^x|x$")
a2 <- str_detect(words, "^x") | str_detect(words, "x$")
b1 <- str_detect(words, "^[aeiou].*[^aeiou]$")
b2 <- str_detect(words, "^[aeiou]") & str_detect(words, "[^aeiou]$")
c(identical(a1, a2), identical(b1, b2))
```

```
## [1] TRUE TRUE
```

```
words[a1]
```

```
## [1] "box" "sex" "six" "tax"
```

```
sum(b1)
```

```
## [1] 122
```

The combined calls are easier to read and modify. ■

Lecture 8. Factors, Dates and Times

8.1 Factors

Motivation. Factors give a fixed set of values (typos caught) and a non-alphabetical order; forcats manipulates them.

8.1.1 Creating factors

Months as strings: typos go undetected, and they sort alphabetically.

```
x1 <- c("Dec", "Apr", "Jan", "Mar")
x2 <- c("Dec", "Apr", "Jam", "Mar")
sort(x1)
```

```
## [1] "Apr" "Dec" "Jan" "Mar"
```

```
month_levels <- c("Jan", "Feb", "Mar", "Apr", "May", "Jun",
                  "Jul", "Aug", "Sep", "Oct", "Nov", "Dec")
y1 <- factor(x1, levels = month_levels)
sort(y1)
```

```
## [1] Jan Mar Apr Dec
## Levels: Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec
```

```
y2 <- factor(x2, levels = month_levels)
y2
```

```
## [1] Dec  Apr  <NA> Mar
## Levels: Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec
```

Values outside the **levels** silently become `NA` (`readr::parse_factor()` warns). Default levels: sorted unique values; `factor(x1, levels = unique(x1))` keeps first appearance. `levels(y1)` lists them.

8.1.2 The General Social Survey

`forcats::gss_cat`: a sample of the US General Social Survey (marital status, race, income, party, religion). See levels with `count()` or a bar chart; `scale_x_discrete(drop = FALSE)` shows empty levels.

```
gss_cat %>% count(race)
```

```
## # A tibble: 3 x 2
##   race      n
##   <fct> <int>
## 1 Other  1959
## 2 Black  3129
## 3 White 16395
```

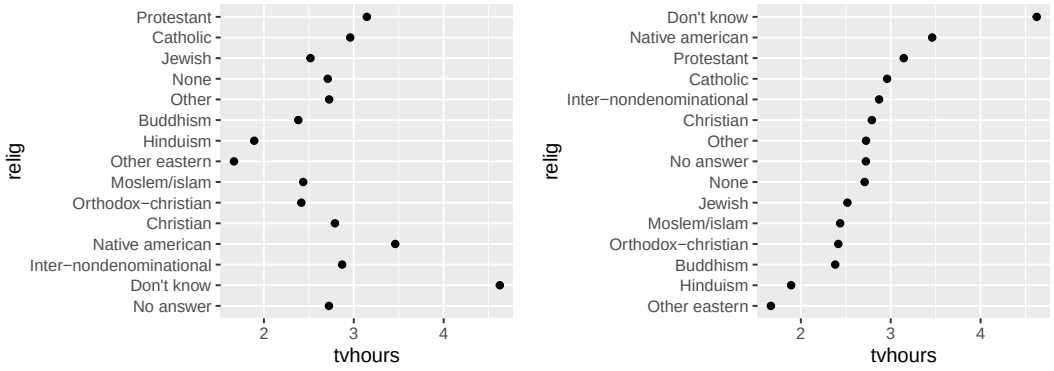
Two operations: change the *order* or the *values* of the levels.

8.1.3 Modifying factor order

`fct_reorder(f, x)` orders levels by the median of `x` (or `.fun`); TV hours by religion:

```
relig_summary <- gss_cat %>%
  group_by(relig) %>%
  summarise(age = mean(age, na.rm = TRUE),
```

```
            tvhours = mean(tvhours, na.rm = TRUE), n = n())
ggplot(relig_summary, aes(tvhours, relig)) + geom_point()
relig_summary %>%
  mutate(relig = fct_reorder(relig, tvhours)) %>%
  ggplot(aes(tvhours, relig)) + geom_point()
```



Reorder in `mutate()`, not `aes()`. Keep natural orders: `fct_reorder(rincome, age)` would scramble income brackets; `fct_relevel(rincome, "Not applicable")` only moves that level to the front.

`fct_reorder2(f, x, y)`: by `y` at the largest `x` (lines match the legend); `fct_infreq(f)`: by frequency (bar charts; `fct_rev()` reverses).

8.1.4 Modifying factor levels

`fct_recode()` renames (new on the left), leaves others; several old to one new combines:

```
gss_cat %>%
  mutate(partyid = fct_recode(partyid,
    "Republican, strong"    = "Strong republican",
    "Republican, weak"      = "Not str republican",
    "Independent, near rep" = "Ind,near rep",
    "Independent, near dem" = "Ind,near dem",
    "Democrat, weak"        = "Not str democrat",
    "Democrat, strong"      = "Strong democrat",
    "Other"                 = "No answer",
    "Other"                 = "Don't know",
    "Other"                 = "Other party"
  )) %>%
  count(partyid)
```

```
## # A tibble: 8 x 2
##   partyid                 n
##   <fct>               <int>
## 1 Other                 548
## 2 Republican, strong   2314
## 3 Republican, weak     3032
## 4 Independent, near rep 1791
## 5 Independent          4119
## # i 3 more rows
```

Combine with care (different groups merge). `fct_collapse()`: a vector of old levels per new one:

```
gss_cat %>%
  mutate(partyid = fct_collapse(partyid,
    other = c("No answer", "Don't know", "Other party"),
    rep = c("Strong republican", "Not str republican"),
```

```
    ind = c("Ind,near rep", "Independent", "Ind,near dem"),
    dem = c("Not str democrat", "Strong democrat"))) %>%
  count(partyid)
```

```
## # A tibble: 4 x 2
##   partyid     n
##   <fct>   <int>
## 1 other     548
## 2 rep      5346
## 3 ind      8409
## 4 dem      7180
```

`fct_lump(f, n = 10)`: the `n` most frequent levels, the rest "Other".

8.2 Dates and times

Motivation. Unequal years, 23- or 25-hour days and time zones: lubridate keeps them apart.

`lubridate` (older tidyverse: load separately) has **date** (`<date>`), **time** (`<time>`, `hms`) and **date-time** (`<dttm>`, POSIXct), an instant. Use the simplest type (date-times bring time zones). `today()`, `now()`.

8.2.1 Creating dates and date-times

From strings. Name the order of `y`, `m`, `d`; add `_h`, `_hm`, `_hms` for times. Unquoted numbers work too.

```
ymd("2017-01-31")
```

```
## [1] "2017-01-31"
```

```
mdy("January 31st, 2017")
```

```
## [1] "2017-01-31"
```

```
dmy("31-Jan-2017")
```

```
## [1] "2017-01-31"
```

```
ymd_hms("2017-01-31 20:11:59")
```

```
## [1] "2017-01-31 20:11:59 UTC"
```

```
mdy_hm("01/31/2017 08:01")
```

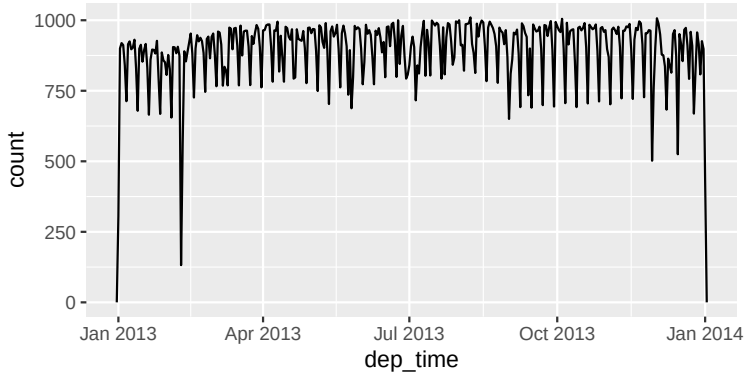
```
## [1] "2017-01-31 08:01:00 UTC"
```

From components. `make_date()`, `make_datetime()`; `HHMM` times need a helper:

```
make_datetime_100 <- function(year, month, day, time) {
  make_datetime(year, month, day, time %/% 100, time %% 100)
}
flights_dt <- flights %>%
  filter(!is.na(dep_time), !is.na(arr_time)) %>%
  mutate(
    dep_time = make_datetime_100(year, month, day, dep_time),
    arr_time = make_datetime_100(year, month, day, arr_time),
    sched_dep_time = make_datetime_100(year, month, day, sched_dep_time),
    sched_arr_time = make_datetime_100(year, month, day, sched_arr_time)
  ) %>%
  select(origin, dest, ends_with("delay"), ends_with("time"))
```

Numerically a date-time is seconds (a date days): binwidth 86400 is one day:

```
flights_dt %>%
  ggplot(aes(dep_time)) +
  geom_freqpoly(binwidth = 86400)
```

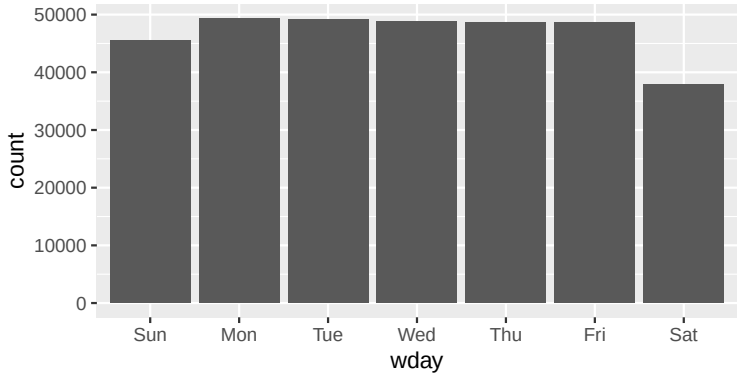


From other types. `as_datetime()`, `as_date()`; numbers are offsets from the Unix epoch 1970-01-01.

8.2.2 Date-time components

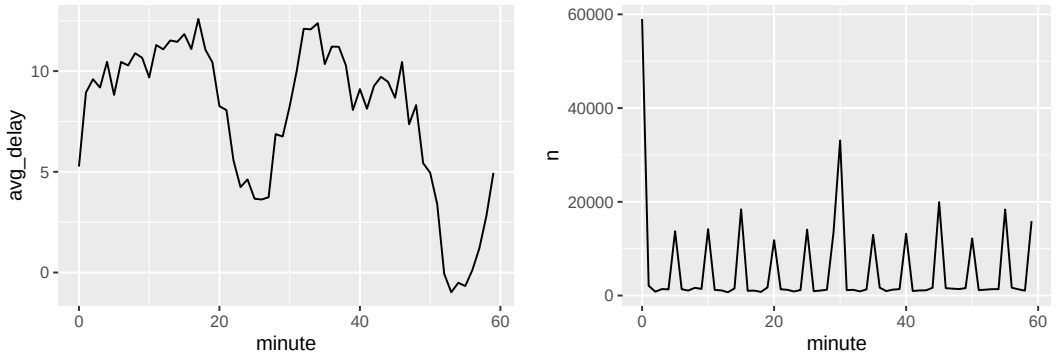
Accessors `year()`, `month()`, `mday()`, `yday()`, `wday()`, `hour()`, `minute()`, `second()`; `label = TRUE` gives names. Fewer flights at weekends:

```
flights_dt %>%
  mutate(wday = wday(dep_time, label = TRUE)) %>%
  ggplot(aes(x = wday)) + geom_bar()
```



By minute of *actual* departure, minutes 20–30 and 50–60 have much lower delays; by *scheduled* minute there is no such pattern: early departures cluster before "nice" scheduled times [Supplement].

```
flights_dt %>%
  mutate(minute = minute(dep_time)) %>%
  group_by(minute) %>%
  summarise(avg_delay = mean(arr_delay, na.rm = TRUE), n = n()) %>%
  ggplot(aes(minute, avg_delay)) + geom_line()
sched_dep <- flights_dt %>%
  mutate(minute = minute(sched_dep_time)) %>%
  group_by(minute) %>%
  summarise(avg_delay = mean(arr_delay, na.rm = TRUE), n = n())
ggplot(sched_dep, aes(minute, n)) + geom_line()
```



Right: scheduled departures prefer round minutes; expect human rounding in human-made data.

Rounding: `floor_date()`, `round_date()`, `ceiling_date()`; flights per week: `count(week = floor_date(dep_time, "week"))`. **Setting**: `year(datetime) <- 2020`, or `update()` for several; overflow rolls over (`mday = 30` in February is in March). `update(dep_time, yday = 1)` shows departures over the hours of one day.

```
datetime <- ymd_hms("2016-07-08 12:34:56")
update(datetime, year = 2020, month = 2, mday = 2, hour = 2)
```

```
## [1] "2020-02-02 02:34:56 UTC"
```

8.2.3 Time spans

Motivation. "Plus one day": 86 400 seconds or the same clock time tomorrow? They differ across daylight saving.

Date subtraction gives a `difftime` (varying unit). Three classes:

- **Durations**: exact seconds, `as.duration(today() - ymd(19791014))`; `dseconds()`, `dminutes()`, `dhours()`, `ddays()`, `dweeks()`, `dyears()`. `ddays(1)` across daylight saving gives an unexpected clock time.
- **Periods**: human units `seconds()`, `days()`, `months()`, `years()`; `days(1)` keeps the clock time, `years(1)` respects leap years. Overnight flights: add `days(1)` when `arr_time < dep_time`.
- **Intervals**: `start %--% end`; divided by a period, the exact number of months etc. (impossible for durations).

```
one_pm <- ymd_hms("2016-03-12 13:00:00", tz = "America/New_York")
one_pm + ddays(1)     # daylight saving started overnight: 14:00
```

```
## [1] "2016-03-13 14:00:00 EDT"
```

```
one_pm + days(1)      # same clock time: 13:00
```

```
## [1] "2016-03-13 13:00:00 EDT"
```

```
next_year <- today() + years(1)
(today() %--% next_year) / days(1)
```

```
## [1] 365
```

Physical time: duration; human time: period; a span in human units: interval.

8.2.4 Time zones

IANA "continent/city" names (`"Asia/Seoul"`); `Sys.timezone()`, `OlsonNames()`. The zone affects only printing: one instant in three zones has differences 0. Default UTC (no daylight saving).

- `with_tz(x, "Asia/Seoul")`: same instant, new display;
- `force_tz(x, "Asia/Seoul")`: same clock reading, new instant (fixes wrong labels).

```
x <- ymd_hms("2024-06-01 12:00:00", tz = "UTC")
with_tz(x, "Asia/Seoul")
```

```
## [1] "2024-06-01 21:00:00 KST"
```

```
force_tz(x, "Asia/Seoul") - x
```

```
## Time difference of -9 hours
```

8.3 Selected exercises

Motivation. Storage versus human reading: level positions, months, ages.

Exercise 8.1 (R4DS Exercise 15.4.1.3)

In the plot of average age by reported income, `fct_relevel(rincome, "Not applicable")` was used. Why did moving "Not applicable" to the front of the levels move it to the bottom of the plot?

Solution. A discrete axis puts the levels at $1, 2, \ldots$ in level order; on the upward y-axis level 1 is at the bottom. `fct_relevel()` makes "Not applicable" level 1 and keeps the others' order, so it drops to the bottom with the brackets in order above (on an x-axis: left). ■

Exercise 8.2 (R4DS Exercise 16.4.5.1)

Why is there `months()` but (in the book's version of lubridate) no `dmonths()`? Explain the outputs below.

```
dmonths(1)
```

```
## [1] "2629800s (~4.35 weeks)"
```

```
ymd("2026-01-31") + months(1)
```

```
## [1] NA
```

```
ymd("2026-01-31") %m+% months(1)
```

```
## [1] "2026-02-28"
```

```
ymd("2026-01-31") + dmonths(1)
```

```
## [1] "2026-03-02 10:30:00 UTC"
```

Solution.

Step 1. Months have 28–31 days, so no fixed number of seconds is "one month"; a period ("same day next month") needs no fixed length. Hence only `months()`.

Step 2. Newer `dmonths(1)` is the average month, $365.25/12 = 30.4375$ days $= 2\,629\,800$ s; 31 January + that is 10:30 on 2 March.

Step 3. `months(1)` asks for 31 February: `NA`, not a silent guess. `%m+%` rolls back to 28 February. The right answer depends on the application. ■

Exercise 8.3 (an age that is right on the birthday; after R4DS Exercise 16.4.5.4)

(a) Write a function that returns the age in completed years of a person born on `bday` at date `on`.
(b) Someone born on 31 October 2000 turns 26 on 31 October 2026. Show that dividing the duration of the span by `dyears(1)` gives a number below 26 on that day, while dividing the interval by `years(1)` gives exactly 26.

```
age <- function(bday, on = today()) floor((bday %--% on) / years(1))
bday <- ymd("2000-10-31")
age(bday, on = ymd("2026-10-31"))
```

```
## [1] 26
```

```
age(bday, on = ymd("2026-10-30"))
```

```
## [1] 25
```

```
as.duration(bday %--% ymd("2026-10-31")) / dyears(1)
```

```
## [1] 25.99863
```

Solution [Supplement].

(a) Interval / `years(1)` counts calendar years (fraction for the rest); `floor()` keeps completed years, so `age()` steps up exactly on each birthday.

(b)

Step 1. The span has 26 years and six leap days (29 Feb 2004, . . . , 2024): $26 \cdot 365 + 6 = 9496$ days.

Step 2. `dyears(1)` $= 365.25$ days and $26 \cdot 365.25 = 9496.5$, so $9496/365.25 = 25.9986 < 26$ (6 leap days, not 6.5); `floor()` would give 25.

Step 3. The period counts calendar years, regardless of leap days: exactly 26. Ages need periods. ■

Lecture 9. Pipes

9.1 Why pipes

Motivation. Code communicates; the pipe reads in the order of the analysis.

`%>%` is from `magrittr` (loaded by the tidyverse). Four ways to write "Little bunny Foo Foo went hopping through the forest, scooping up the field mice and bopping them on the head":

Save each intermediate step.

```
foo_foo <- little_bunny()
foo_foo_1 <- hop(foo_foo, through = forest)
foo_foo_2 <- scoop(foo_foo_1, up = field_mice)
foo_foo_3 <- bop(foo_foo_2, on = head)
```

Cluttered names, error-prone suffixes. Memory is not the issue: copies share unchanged columns.

Overwrite the original.

```
foo_foo <- hop(foo_foo, through = forest)
foo_foo <- scoop(foo_foo, up = field_mice)
foo_foo <- bop(foo_foo, on = head)
```

Debugging means rerunning from the start; the repeated name hides what changes.

Function composition.

```
bop(scoop(hop(foo_foo, through = forest), up = field_mice), on = head)
```

Read inside out; arguments far from their function ("Dagwood sandwich").

The pipe.

```
foo_foo %>%
  hop(through = forest) %>%
  scoop(up = field_mice) %>%
  bop(on = head)
```

A sequence of verbs: hops, then scoops, then bops. magrittr rewrites it roughly as

```
my_pipe <- function(.) {
  . <- hop(., through = forest)
  . <- scoop(., up = field_mice)
  bop(., on = head)
}
```

So it fails with functions using the current environment (`assign()`, `get()`, `load()`) or special lazy evaluation (`tryCatch()`, `try()`, `suppressMessages()`).

The native pipe [Supplement]

R 4.1's `|>`: `x |> f(y)` is `f(x, y)`; faster, no package, but no `.` placeholder (R 4.2: `_` for named arguments) and no special pipes. These notes use `%>%`.

9.2 When not to use the pipe

Pipes suit a linear chain on one object. Avoid them when

- there are more than about ten steps (name intermediate objects);

- there are several inputs or outputs;
- the steps form a directed graph, not a chain.

9.3 Other tools from magrittr

Tee pipe `%T>%` returns its left-hand side, so a side-effect step (plot, print, save) does not end the chain; otherwise `plot()` passes `NULL`:

```
rnorm(100) %>%
  matrix(ncol = 2) %>%
  plot() %>%
  str()
```

```
##  NULL
```

```
rnorm(100) %>%
  matrix(ncol = 2) %T>%
  plot() %>%
  str()
```

```
##  num [1:50, 1:2] -0.387 -0.785 -1.057 -0.796 -1.756 ...
```

Exposition pipe `%$%` exposes columns by name to functions without a `data` argument:

```
mtcars %$% cor(disp, mpg)
```

```
## [1] -0.8475514
```

Assignment pipe `%<>%`: `mtcars %<>% transform(cyl = cyl * 2)` for `mtcars <- mtcars %>% transform(cyl = cyl * 2)`; often avoided, as assignment should be visible at the line start.

9.4 Selected exercises

Motivation. Consequences of the pipe's rewriting rule.

Exercise 9.1 (the rewriting rule)

Take as definition that `x %>% f(y)` means `f(x, y)` when the dot `.` does not appear as an argument of `f`, and means `f(y)` with `.` replaced by `x` when it does.
(a) Prove that a chain `x %>% f1(y1) %>% ⋯ %>% fk(yk)` without dots evaluates to $f_k(\cdots f_2(f_1(x, y_1), y_2) \cdots, y_k)$.
(b) Predict, then check, the values of `1:3 %>% c(4, .)` and `1:3 %>% c(length(.))`.

Solution [Supplement].

(a)

Step 1. Left-associative: `((x %>% f1(y1)) %>% f2(y2)) %>% ⋯`. Induct on k; $k = 1$ is the definition.

Step 2. If length $k-1$ gives $v = f_{k-1}(\cdots f_1(x, y_1) \cdots, y_{k-1})$, length k pipes v into `fk(yk)`: $f_k(v, y_k)$.

(b) In `c(4, .)` the dot is an argument: `c(4, 1:3)`. In `c(length(.))` it is only inside `length(.)`, so the left side is still inserted first: `c(1:3, length(1:3))`:

```
1:3 %>% c(4, .)
```

```
## [1] 4 1 2 3
```

```
1:3 %>% c(length(.))
```

```
## [1] 1 2 3 3
```

■

Exercise 9.2 (assignment inside a pipe)

Explain the following output. Why does `assign()` inside a pipe not create a variable unless the environment is given explicitly?

```
"zz" %>% assign(100)
exists("zz")
```

```
## [1] FALSE
```

```
env <- environment()
"zz" %>% assign(100, envir = env)
exists("zz")
```

```
## [1] TRUE
```

Solution [Supplement].

Step 1. `assign(x, value)` writes into the environment where it is evaluated (default `pos = -1`).

Step 2. The pipe evaluates the right side in a temporary function-like environment binding `.` (cf. `my_pipe`); `zz` is created there and vanishes, so `exists("zz")` is `FALSE`.

Step 3. Passing the captured calling environment as `envir` makes `zz` visible. Likewise `get()`, `load()` see the pipe's environment, not ours. ■

Exercise 9.3 (when the exposition pipe is needed)

Why does the first call below fail while the second works?

```
mtcars %>% sum(cyl)
```

```
## Error: object 'cyl' not found
```

```
mtcars %$% sum(cyl)
```

```
## [1] 198
```

Solution [Supplement]. By Exercise 9.1 the first is `sum(mtcars, cyl)`, and `cyl` is looked up in the calling environment, where it does not exist (`sum()` has no `data` argument, unlike `lm()`). `%$%` evaluates `sum(cyl)` with the columns as variables, like `with(mtcars, sum(cyl))`: 198 cylinders. ■

Exercise 9.4 (the native pipe is syntax)

Explain the following outputs, and why `1:3 |> sum` is an error while `1:3 %>% sum` works.

```
quote(x |> f(y))
```

```
## f(x, y)
```

```
quote(x %>% f(y))
```

```
## x %>% f(y)
```

```
tryCatch(str2lang("1:3 |> sum"), error = conditionMessage)
```

```
## [1] "The pipe operator requires a function call as RHS (<input>:1:8)"
```

Solution [Supplement].

Step 1. `quote()` does not evaluate. The parser rewrites `x |> f(y)`: it *is* `f(x, y)`, no pipe function, no overhead.

Step 2. `` `%>%` `` is an ordinary two-argument function: the quoted call stays a call to it, rewritten only when evaluated.

Step 3. The parser inserts `x` into a call, so `|>` needs `sum()`; magrittr gets the unevaluated right side and accepts a bare name: `1:3 %>% sum` is `sum(1:3)`. ■

Lecture 10. Functions

10.1 When to write a function

Motivation. Functions remove copy-and-paste bugs and name an idea.

Rescale each column to $[0, 1]$:

```
df <- tibble::tibble(a = rnorm(10), b = rnorm(10), c = rnorm(10), d = rnorm(10))
df$a <- (df$a - min(df$a, na.rm = TRUE)) /
  (max(df$a, na.rm = TRUE) - min(df$a, na.rm = TRUE))
df$b <- (df$b - min(df$b, na.rm = TRUE)) /
  (max(df$b, na.rm = TRUE) - min(df$a, na.rm = TRUE))
# ... and the same for c and d
```

The `b` line has a hidden error (`min(df$a)`). Rule: copied more than twice, write a function (DRY).

How. Count inputs (one: call it `x`); remove duplication (`range()` once instead of three times):

```
rescale01 <- function(x) {
  rng <- range(x, na.rm = TRUE)
  (x - rng[1]) / (rng[2] - rng[1])
}
rescale01(c(-10, 0, 10))
```

```
## [1] 0.0 0.5 1.0
```

```
rescale01(c(1, 2, 3, NA, 5))
```

```
## [1] 0.00 0.25 0.50   NA 1.00
```

A **name**, **arguments** in `function()`, a **body** in braces; start from working code. Each column is now `df$a <- rescale01(df$a)` (Lecture 12 removes this too).

Changes happen in one place: infinite values make everything `NaN`, and one edit fixes every use:

```
x <- c(1:10, Inf)
rescale01 <- function(x) {
  rng <- range(x, na.rm = TRUE, finite = TRUE)
  (x - rng[1]) / (rng[2] - rng[1])
}
rescale01(x)
```

```
##  [1] 0.0000000 0.1111111 0.2222222 0.3333333 0.4444444 0.5555556 0.6666667 0.7777778
##  [9] 0.8888889 1.0000000       Inf
```

10.2 Functions are for humans and computers

R does not care about names or comments, but readers do.

- **Names**: short and descriptive; verbs for functions (`impute_missing()`), nouns for arguments; not `f()` or `my_awesome_function()`.
- One style, `snake_case` or `camelCase`, not `col_mins()` with `rowMaxes()`; a common prefix for a family (`input_select()`, `input_text()`).
- Never overwrite `T <- FALSE`, `c <- 10`, `mean <- function(x) sum(x)`.
- **Comments** explain *why*; the code shows what and how. `# Load data ---` lines make sections.

10.3 Conditional execution

Motivation. Branches for special cases and choices of method.

```
if (condition) {
  # code executed when condition is TRUE
} else {
  # code executed when condition is FALSE
}
```

```
has_name <- function(x) {
  nms <- names(x)
  if (is.null(nms)) {
    rep(FALSE, length(x))
  } else {
    !is.na(nms) & nms != ""
  }
}
has_name(c(a = 1, 2, b = 3))
```

```
## [1]  TRUE FALSE  TRUE
```

A function returns its last value.

10.3.1 Conditions

One `TRUE` or `FALSE`; a longer vector (formerly a warning) or `NA` is an error:

```
if (c(TRUE, FALSE)) {}
```

```
## Error in if (c(TRUE, FALSE)) {: the condition has length > 1
```

```
if (NA) {}
```

```
## Error in if (NA) {: missing value where TRUE/FALSE needed
```

Combine with short-circuit `||`, `&&`, not vectorised `|`, `&`; reduce vectors with `any()`, `all()`. `==` is vectorised and gives `NA`; `identical()` returns one value but distinguishes types (`identical(0L, 0)` is `FALSE`); for reals `dplyr::near()`, as `sqrt(2)^2 == 2` is `FALSE`.

10.3.2 Multiple conditions

Chain `if (this) {...} else if (that) {...} else {...}`; `switch()` selects by name or position:

```
calc <- function(x, y, op) {
  switch(op,
    plus = x + y,
    minus = x - y,
    times = x * y,
    divide = x / y,
    stop("Unknown op!")
  )
}
calc(6, 3, "divide")
```

```
## [1] 2
```

`cut()` replaces long `if` chains for continuous variables.

10.3.3 Code style

An opening `{` ends a line, never alone; a closing `}` gets its own line unless `else` follows; indent two spaces. Short: `x <- if (y < 20) "Too low" else "Too high"`. `else` on a new line after `}` is a top-level syntax error (the `if` is complete).

10.4 Function arguments

Motivation. The interface: data versus details, sensible defaults, loud failures.

Data arguments first; **detail** arguments last, usually with **defaults**:

```
mean_ci <- function(x, conf = 0.95) {
  se <- sd(x) / sqrt(length(x))
  alpha <- 1 - conf
  mean(x) + se * qnorm(c(alpha / 2, 1 - alpha / 2))
}
x <- runif(100)
mean_ci(x)
```

```
## [1] 0.4194814 0.5276171
```

```
mean_ci(x, conf = 0.99)
```

```
## [1] 0.4024921 0.5446064
```

Default = the most common value (except `na.rm = FALSE`: ignoring missing values must be deliberate). Call with unnamed data and fully named details: `mean(1:10, na.rm = TRUE)`, not `mean(, TRUE, x = c(1:10, NA))`; spaces around `=` and after commas.

Common names: `x`, `y`, `z` (vectors), `w` (weights), `df`, `i`, `j` (indices), `n` (length, rows), `p` (columns); follow conventions like `na.rm`.

10.4.1 Checking values

Recycling hides mistakes:

```
wt_mean <- function(x, w) sum(x * w) / sum(w)
wt_mean(1:6, 1:3)
```

```
## [1] 7.666667
```

Weights recycled to length 6, no warning (6 is a multiple of 3). Check with `stop()` or `stopifnot()` (each argument must be `TRUE`):

```
wt_mean <- function(x, w, na.rm = FALSE) {
  stopifnot(is.logical(na.rm), length(na.rm) == 1)
  stopifnot(length(x) == length(w))
  if (na.rm) {
    miss <- is.na(x) | is.na(w)
    x <- x[!miss]
    w <- w[!miss]
  }
  sum(w * x) / sum(w)
}
wt_mean(1:6, 6:1, na.rm = "foo")
```

```
## Error in wt_mean(1:6, 6:1, na.rm = "foo"): is.logical(na.rm) is not TRUE
```

Checks trade time for robustness.

10.4.2 Dot-dot-dot

`...` captures all unmatched arguments (as in `sum()`) and passes them on:

```
commas <- function(...) stringr::str_c(..., collapse = ", ")
commas(letters[1:10])
```

```
## [1] "a, b, c, d, e, f, g, h, i, j"
```

Misspellings are swallowed: `sum(x, na.mr = TRUE)` adds `TRUE`, i.e. 1.

10.5 Return values

`return()` exits early, for simple special cases:

```
complicated_function <- function(x, y, z) {
  if (length(x) == 0 || length(y) == 0) {
    return(0)
  }
  # complicated code here
}
```

Pipeable functions. *Transformations* return a modified first argument; *side-effect* functions return it *invisibly*:

```
show_missings <- function(df) {
  n <- sum(is.na(df))
  cat("Missing values: ", n, "\n", sep = "")
  invisible(df)
}
mtcars %>%
  show_missings() %>%
  mutate(mpg = ifelse(mpg < 20, NA, mpg)) %>%
  show_missings()
```

```
## Missing values: 0
## Missing values: 18
```

10.6 Environment

Motivation. Where R looks up variables that are not arguments.

```
f <- function(x) {
  x + y
}
y <- 100
f(10)
```

```
## [1] 110
```

```
y <- 1000
f(10)
```

```
## [1] 1010
```

Lexical scoping: a name not found inside is looked up where the function was *defined* (here global), so the result depends on a global variable: avoid. The same rules let R override even `+` or `(`.

Defined, not called [Supplement]

```
make_adder <- function(y) function(x) x + y
add2 <- make_adder(2)
y <- 1000
add2(10)
```

```
## [1] 12
```

`add2` keeps `y = 2` from its creating environment (a *closure*); the global `y` is irrelevant.

10.7 Selected exercises

Motivation. Interfaces, scalar versus vector conditions, defaults, and lookup time.

Exercise 10.1 (R4DS Exercise 19.2.1.1)

Why is `TRUE` not an argument of `rescale01()`? What would happen if `x` contained a single missing value and `na.rm` were `FALSE`?

```
rescale01_strict <- function(x) {
  rng <- range(x, na.rm = FALSE)
  (x - rng[1]) / (rng[2] - rng[1])
}
rescale01_strict(c(1, 2, NA, 4))
```

```
## [1] NA NA NA NA
```

Solution.

Step 1. Arguments are what callers may want to change. Mapping observed values to $[0, 1]$ needs the range without `NA`s; `na.rm = FALSE` has no sensible use, so it is fixed in the body.

Step 2. With `na.rm = FALSE` one `NA` gives `range()` = `c(NA, NA)`, so every entry, observed or not, is `NA`: one missing value destroys the column. ■

Exercise 10.2 (R4DS Exercises 19.4.4.1 and 19.4.4.5)

(a) What is the difference between `if` and `ifelse()`?
(b) What happens if `switch()` is used with numeric values?

```
ifelse(c(TRUE, FALSE, NA), "yes", "no")
```

```
## [1] "yes" "no"  NA
```

```
if (c(TRUE, FALSE)) "yes" else "no"
```

```
## Error in if (c(TRUE, FALSE)) "yes" else "no": the condition has length > 1
```

```
switch(2, "a", "b", "c")
```

```
## [1] "b"
```

```
is.null(switch(4, "a", "b", "c"))
```

```
## [1] TRUE
```

Solution.

(a) `if` is control flow: one `TRUE`/`FALSE` condition selects one of two blocks. `ifelse(test, yes, no)` is vectorised: length of `test`, elements from `yes` or `no`, `NA` where `test` is `NA`. `if` chooses what

to do; `ifelse()` computes a column.

(b) With a number n, `switch()` ignores names and returns the nth remaining argument; beyond their number, invisible `NULL` (hence `is.null()`). ■

Exercise 10.3 (R4DS Exercise 19.5.5.3)

The default value of the `method` argument of `cor()` is `c("pearson", "kendall", "spearman")`. What does that mean, and which value is used by default?

```
centre <- function(x, type = c("mean", "median")) {
  type <- match.arg(type)
  switch(type, mean = mean(x), median = median(x))
}
centre(c(1, 2, 10))
```

```
## [1] 4.333333
```

```
centre(c(1, 2, 10), "median")
```

```
## [1] 2
```

```
centre(c(1, 2, 10), "mode")
```

```
## Error in match.arg(type): 'arg' should be one of "mean", "median"
```

Solution. The vector lists the allowed values for `match.arg()`. Omitted, it gets the whole vector and returns the first element: `"pearson"`. Supplied, it must be listed (partial matching allowed), else an error. So `centre()` defaults to the mean, allows the median, rejects `"mode"`; the signature documents the choices. ■

Exercise 10.4 (when is a free variable looked up)

Predict the value of `f()`, and explain it with lexical scoping.

```
f <- function() {
  x <- 1
  g <- function() x
  x <- 2
  g()
}
f()
```

```
## [1] 2
```

Solution [Supplement].

Step 1. `g` has no local `x`; it is looked up in `g`'s defining environment, `f`'s execution environment.

Step 2. Scoping fixes *where*, not *when*: at the call `g()`, that `x` is already 2, so `g()` returns 2.

Step 3. Consistent with `add2`: nothing changes `y` in its defining environment; the global `y` is elsewhere. ■

Lecture 12. Iteration

12.1 Iteration

Motivation. Iteration repeats a computation over many inputs: loops or functionals.

Less duplication: clearer intent, changes in one place, fewer bugs. **Iteration**: *imperative* for/while loops, or *functional* tools (`purrr`) that hide the boilerplate.

12.2 For loops

Median of each column of

```
df <- tibble(a = rnorm(10), b = rnorm(10), c = rnorm(10), d = rnorm(10))
```

without copying `median(df$a)` four times:

```
output <- vector("double", ncol(df))   # 1. output
for (i in seq_along(df)) {             # 2. sequence
  output[[i]] <- median(df[[i]])       # 3. body
}
output
```

```
## [1] -0.24576245 -0.28730721 -0.05669771  0.14426335
```

Three components:

1. **Output**: preallocate with `vector(type, length)`; growing with `c()` copies everything each step.
2. **Sequence**: `seq_along(df)`, a safe `1:length()`: empty for length 0, whereas `1:length(y)` is `c(1, 0)`.
3. **Body**: run once per `i`; use `[[ ]]` to show one element is handled.

```
y <- vector("double", 0)
seq_along(y)
```

```
## integer(0)
```

```
1:length(y)
```

```
## [1] 1 0
```

12.2.1 Variations

Modifying an existing object. Rescaling every column of `df` (Lecture 10):

```
rescale01 <- function(x) {
  rng <- range(x, na.rm = TRUE)
  (x - rng[1]) / (rng[2] - rng[1])
}
for (i in seq_along(df)) {
  df[[i]] <- rescale01(df[[i]])
}
```

The output is `df` itself.

Looping patterns. Over elements, `for (x in xs)` (awkward for output); over names, `for (nm in names(xs))` with `x[[nm]]`; over indices, the most general (name and value):

```
results <- vector("list", length(x))
names(results) <- names(x)
for (i in seq_along(x)) {
  name <- names(x)[[i]]
  value <- x[[i]]
}
```

Unknown output length. Not `c()` (quadratic): store pieces in a list, combine once with `unlist()` (`purrr::flatten_dbl()`), `paste(collapse = "")` or `dplyr::bind_rows()`:

```
means <- c(0, 1, 2)
out <- vector("list", length(means))
for (i in seq_along(means)) {
  n <- sample(100, 1)
  out[[i]] <- rnorm(n, means[[i]])
}
str(out)
```

```
## List of 3
##  $ : num [1:38] 0.255 -0.553 1.405 -0.795 -1.567 ...
##  $ : num [1:17] 0.91 2.92 2.3 1.75 1.56 ...
##  $ : num [1:14] 3.215 0.722 2.748 5.392 3.619 ...
```

```
str(unlist(out))
```

```
##  num [1:69] 0.255 -0.553 1.405 -0.795 -1.567 ...
```

Unknown sequence length: while. Runs while a condition is `TRUE` (every for loop is a while loop, not conversely). Flips until three heads in a row:

```
flip <- function() sample(c("T", "H"), 1)
flips <- 0
nheads <- 0
while (nheads < 3) {
  if (flip() == "H") nheads <- nheads + 1 else nheads <- 0
  flips <- flips + 1
}
flips
```

```
## [1] 18
```

How many flips on average? [Supplement]

e_k: expected further flips at run length k. $e_3 = 0$, $e_k = 1 + \frac{1}{2}e_{k+1} + \frac{1}{2}e_0$ $(k = 0, 1, 2)$, so $e_0 = 14$; simulation:

```
flips_until_3 <- function() {
  flips <- 0; nheads <- 0
  while (nheads < 3) {
    if (flip() == "H") nheads <- nheads + 1 else nheads <- 0
    flips <- flips + 1
  }
  flips
}
mean(replicate(5000, flips_until_3()))
```

```
## [1] 13.9762
```

12.3 For loops versus functionals

Motivation. Pass the function as an argument: the idea behind purrr.

Loops for means, medians, sds differ only in the function, so make it an argument:

```
col_summary <- function(df, fun) {
  out <- vector("double", length(df))
  for (i in seq_along(df)) {
    out[i] <- fun(df[[i]])
  }
  out
}
col_summary(df, median)
```

```
## [1] 0.6111823 0.4011430 0.4280338 0.5813702
```

```
col_summary(df, sd)
```

```
## [1] 0.3117016 0.2993812 0.3078818 0.2728141
```

A **functional** takes a function as input: `apply()`, `lapply()`, `tapply()`; `purrr` is a consistent family.

12.4 The map functions

One per output type:

function	output
`map()`	list
`map_lgl()`	logical vector
`map_int()`	integer vector
`map_dbl()`	double vector
`map_chr()`	character vector

```
df %>% map_dbl(mean)
```

```
##         a         b         c         d
## 0.5566313 0.4287933 0.4083740 0.5836067
```

```
df %>% map_dbl(sd)
```

```
##         a         b         c         d
## 0.3117016 0.2993812 0.3078818 0.2728141
```

Versus `col_summary()`: written in C, extra arguments via `...` (`map_dbl(df, mean, trim = 0.5)`), names kept. The gain is clarity, not speed (preallocated loops are not slow).

12.4.1 Shortcuts

A model per cylinder group: `split()` into a list and map; in a one-sided formula `.` (`.x`) is the element:

```
models <- mtcars %>%
  split(.$cyl) %>%
  map(~lm(mpg ~ wt, data = .))
models %>%
  map(summary) %>%
  map_dbl(~.x$r.squared)
```

```
##         4         6         8
## 0.5086326 0.4645102 0.4229655
```

A string extracts a named component, an integer a position:

```
models %>% map(summary) %>% map_dbl("r.squared")
```

```
##         4         6         8
## 0.5086326 0.4645102 0.4229655
```

```
x <- list(list(1, 2, 3), list(4, 5, 6), list(7, 8, 9))
x %>% map_dbl(2)
```

```
## [1] 2 5 8
```

12.4.2 Base R

`lapply()` ≈ `map()`. `sapply()` simplifies automatically, so its output type depends on the data:

```
threshold <- function(x, cutoff = 0.8) x[x > cutoff]
x1 <- list(c(0.27, 0.37, 0.57, 0.91, 0.20), c(0.90, 0.94, 0.66, 0.63, 0.06),
           c(0.21, 0.18, 0.69, 0.38, 0.77))
x2 <- list(c(0.50, 0.72, 0.99, 0.38, 0.78), c(0.93, 0.21, 0.65, 0.13, 0.27),
           c(0.39, 0.01, 0.38, 0.87, 0.34))
x1 %>% sapply(threshold) %>% str()
```

```
## List of 3
##  $ : num 0.91
##  $ : num [1:2] 0.9 0.94
##  $ : num(0)
```

```
x2 %>% sapply(threshold) %>% str()
```

```
##  num [1:3] 0.99 0.93 0.87
```

A list, then a numeric vector. Safe: `vapply(df, is.numeric, logical(1))` fixes the type.

12.5 Dealing with failure

Motivation. Adverbs turn errors into data, so one failure does not lose all results.

`safely()` never errors; it returns `result` and `error`, one `NULL`:

```
safe_log <- safely(log)
str(safe_log(10))
```

```
## List of 2
##  $ result: num 2.3
##  $ error : NULL
```

```
x <- list(1, 10, "a")
y <- x %>% map(safely(log)) %>% transpose()
is_ok <- y$error %>% map_lgl(is_null)
x[!is_ok]
```

```
## [[1]]
## [1] "a"
```

```
y$result[is_ok] %>% flatten_dbl()
```

```
## [1] 0.000000 2.302585
```

`transpose()` turns a list of pairs into a pair of lists. `quietly()` captures output, messages, warnings; `possibly()` gives a default: `x %>% map_dbl(possibly(log, NA_real_))`.

12.6 Mapping over multiple arguments

`map(mu, rnorm, n = 5)` varies one argument; `map2()` two in parallel; `pmap()` a (named) list of any number. A tibble of parameters drives `pmap()` directly:

```
mu <- list(5, 10, -3)
sigma <- list(1, 5, 10)
map2(mu, sigma, rnorm, n = 5) %>% str()
```

```
## List of 3
##  $ : num [1:5] 5.45 4.52 4.16 5.48 2.03
##  $ : num [1:5] 17.59 1.02 10.05 8.09 6.07
##  $ : num [1:5] -6.78 7.76 14.72 -6.58 -11.58
```

```
params <- tribble(
  ~mean, ~sd, ~n,
      5,   1,  1,
     10,   5,  3,
     -3,  10,  5
)
params %>% pmap(rnorm) %>% str()
```

```
## List of 3
##  $ : num 3.99
##  $ : num [1:3] 13.2 14.4 12.6
##  $ : num [1:5] -8.43 4.12 -12.89 5.54 -15.36
```

`map2()` is a loop calling `f(x[[i]], y[[i]], ...)`. Varying the function too: `map2(c("runif", "rnorm", "rpois"), param, function(fn, args) exec(fn, !!!args))` (`invoke_map()` is deprecated).

12.7 Walk, reduce and accumulate

walk(), `walk2()`, `pwalk()`: for side effects, returning the input invisibly; `pwalk(list(paths, plot = plots), ggsave, path = tempdir())` saves plots.

reduce() applies a two-argument function repeatedly: `dfs %>% reduce(full_join)`, `vs %>% reduce(intersect)`. **accumulate()** keeps intermediate results:

```
x <- sample(10)
x
```

```
##  [1]  1  7  8  2  6 10  3  9  4  5
```

```
x %>% accumulate(`+`)
```

```
##  [1]  1  8 16 18 24 34 37 46 50 55
```

12.8 Selected exercises

Motivation. Cost of growing, loops over names, the coin-flip mean, map over a vector.

Exercise 12.1 (the cost of growing a vector; after R4DS Exercise 21.2.1.4)

A loop builds a vector of length n either by `out <- c(out, new)` at every step or by filling a preallocated `vector("double", n)`. Assume that `c()` copies all elements of its arguments into a new vector. Count the element copies of each method, and compare them for $n = 10^5$.

Solution [Supplement].

Step 1. At step i, `c(out, new)` writes $i-1$ old elements and one new: i copies.

Step 2. Total $\sum_{i=1}^{n} i = n(n+1)/2$, quadratic; preallocation: n.

Step 3. Ratio $(n+1)/2$: for $n = 10^5$, about 5×10^9 copies against 10^5. ■

Exercise 12.2 (R4DS Exercise 21.3.5.2)

What happens if you use `for (nm in names(x))` and `x` has no names? What if only some of the elements are named? What if the names are not unique?

```
x <- 1:3
for (nm in names(x)) print(nm)
x <- c(a = 1, 2, c = 3)
names(x)
```

```
## [1] "a" ""  "c"
```

```
for (nm in names(x)) print(x[[nm]])
```

```
## [1] 1
```

```
## Error in x[[nm]]: subscript out of bounds
```

```
x <- c(a = 1, a = 2)
for (nm in names(x)) print(x[[nm]])
```

```
## [1] 1
## [1] 1
```

Solution.

Step 1. No names: `names(x)` is `NULL` (length 0); the loop silently does nothing.

Step 2. Partly named: the others have name `""`; `x[[""]]` matches nothing, so after the first element the loop stops with an out-of-bounds error.

Step 3. Duplicated names: `x[[nm]]` returns the first match, so it is processed twice and the second never.

Step 4. Looping over `seq_along(x)` with `names(x)[[i]]` avoids all three. ■

Exercise 12.3 (flips until k heads in a row)

Generalise the note on the coin-flip loop: let N be the number of fair-coin flips until k heads in a row first occur. Show that $\mathbb{E}N < \infty$ and $\mathbb{E}N = 2^{k+1} - 2$ (which is 14 for $k = 3$).

Solution [Supplement].

Step 1. ($\mathbb{E}N < \infty$.) Blocks of k flips are all heads independently w.p. 2^{-k}, and $N \le mk$ if one of the first m is. So $P(N > mk) \le (1-2^{-k})^m$ and $\mathbb{E}N = \sum_{t\ge 0} P(N > t) \le k \sum_{m\ge 0}(1-2^{-k})^m = k2^k < \infty$.

Step 2. (Equations.) e_j: expected further flips at run length j, $e_k = 0$; one flip leads to $j+1$ or 0 w.p. $\frac{1}{2}$ each:

$$e_j = 1 + \tfrac{1}{2}e_{j+1} + \tfrac{1}{2}e_0, \qquad j = 0, \dots, k-1.$$

Step 3. (Solution.) Try $e_j = 2^{k+1} - 2^{j+1}$. Then $e_k = 0$, and

$$1 + \tfrac{1}{2}(2^{k+1} - 2^{j+2}) + \tfrac{1}{2}(2^{k+1} - 2) = 2^{k+1} - 2^{j+1} = e_j.$$

Step 4. (Uniqueness.) Backwards from $j = k-1$, $e_j = a_j + b_j e_0$ with $b_{k-1} = \frac{1}{2}$, $b_j = \frac{1}{2}b_{j+1} + \frac{1}{2}$, so $b_j = 1 - 2^{-(k-j)}$; $e_0 = a_0 + (1-2^{-k})e_0$ fixes the finite e_0. Hence $\mathbb{E}N = e_0 = 2^{k+1} - 2$. ■

Exercise 12.4 (R4DS Exercise 21.5.3.4)

What does `map(-2:2, rnorm, n = 5)` do, and why? What does `map_dbl(-2:2, rnorm, n = 5)` do, and why?

```
set.seed(1)
map(-2:2, rnorm, n = 5) %>% str()
```

```
## List of 5
##  $ : num [1:5] -2.626 -1.816 -2.836 -0.405 -1.67
##  $ : num [1:5] -1.82 -0.513 -0.262 -0.424 -1.305
##  $ : num [1:5] 1.512 0.39 -0.621 -2.215 1.125
##  $ : num [1:5] 0.955 0.984 1.944 1.821 1.594
##  $ : num [1:5] 2.919 2.7821 2.0746 0.0106 2.6198
```

```
map_dbl(-2:2, rnorm, n = 5)
```

```
## Error in `map_dbl()`:
## i In index: 1.
## Caused by error:
## ! Result must be length 1, not 5.
```

Solution.

Step 1. `map()` calls `rnorm(.x, n = 5)`; with `n` named, `.x` goes to `mean`: five vectors of five draws, means $-2, \ldots, 2$.

Step 2. `map_dbl()` needs one number per element but gets five: error. Use `map()` and combine, or `map_dbl()` with scalar functions. ■

Lecture 13. Model Basics

13.1 Models and their purpose

Motivation. A model makes a pattern precise enough to measure, predict and subtract; built from scratch so lm() is no black box.

A model is a low-dimensional summary splitting data into *patterns* and *residuals*; here for exploration, not inference.

Generation versus confirmation. Exploration may reuse an observation freely; confirmation uses it once (twice means exploring). For confirmation split in advance: *training* (about 60%, explore and fit), *query* (about 20%, compare models by hand), *test* (about 20%, used once on the final model).

A model has two parts:

1. a **family of models** expressing a precise but generic pattern, e.g. the lines $y = a_1 + a_2 x$;
2. a **fitted model**, the member of the family closest to the data, e.g. $y = 3x + 7$.

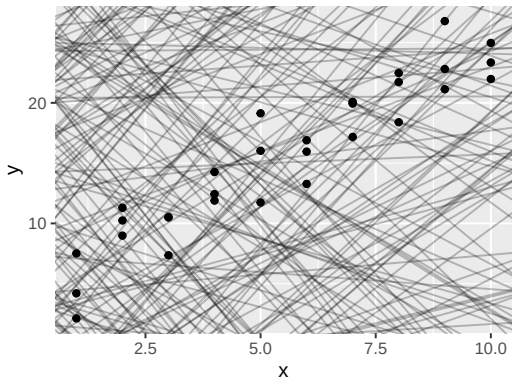
Closest is not "true": "All models are wrong, but some are useful" (G. E. P. Box).

13.2 A simple model

Motivation. Every fit needs a distance between model and data and a search minimising it.

`modelr`; `options(na.action = na.warn)` warns about dropped `NA`s. `sim1` is linear; overlay random lines:

```
models <- tibble(a1 = runif(250, -20, 40), a2 = runif(250, -5, 5))
ggplot(sim1, aes(x, y)) +
  geom_abline(aes(intercept = a1, slope = a2), data = models, alpha = 1/4) +
  geom_point()
```



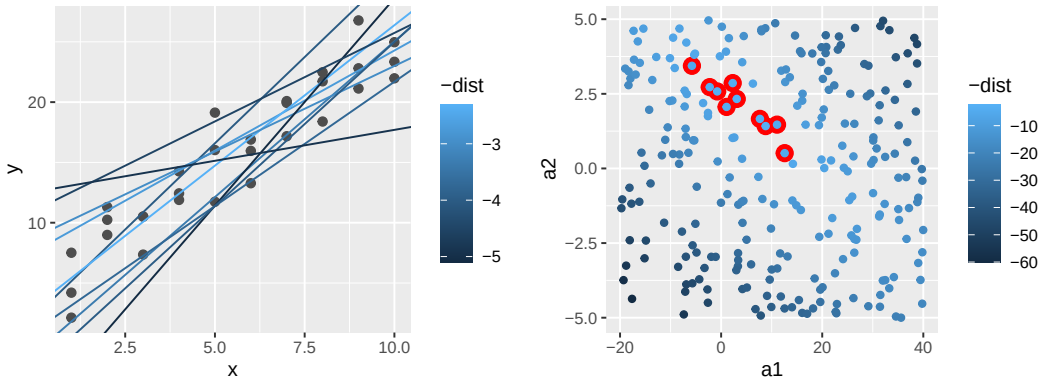
Distance: root-mean-squared vertical deviation:

```
model1 <- function(a, data) a[1] + data$x * a[2]
measure_distance <- function(mod, data) {
  diff <- data$y - model1(mod, data)
  sqrt(mean(diff ^ 2))
}
measure_distance(c(7, 1.5), sim1)
```

```
## [1] 2.665212
```

`map2_dbl()` passes scalar `a1`, `a2`, hence a helper:

```
sim1_dist <- function(a1, a2) measure_distance(c(a1, a2), sim1)
models <- models %>% mutate(dist = map2_dbl(a1, a2, sim1_dist))
ggplot(sim1, aes(x, y)) +
  geom_point(size = 2, colour = "grey30") +
  geom_abline(aes(intercept = a1, slope = a2, colour = -dist),
              data = filter(models, rank(dist) <= 10))
ggplot(models, aes(a1, a2)) +
  geom_point(data = filter(models, rank(dist) <= 10), size = 4, colour = "red") +
  geom_point(aes(colour = -dist))
```



Right: models in parameter space, the ten best in red. **Grid search**: an even grid (`expand.grid()`) around the best region, repeated. Better: `optim()`, Newton–Raphson-type steps downhill:

```
best <- optim(c(0, 0), measure_distance, data = sim1)
best$par
```

```
## [1] 4.222248 2.051204
```

Linear models: `lm()` computes the closed form exactly (QR decomposition):

```
sim1_mod <- lm(y ~ x, data = sim1)
coef(sim1_mod)
```

```
## (Intercept)           x
##    4.220822    2.051533
```

Why the two agree [Supplement]

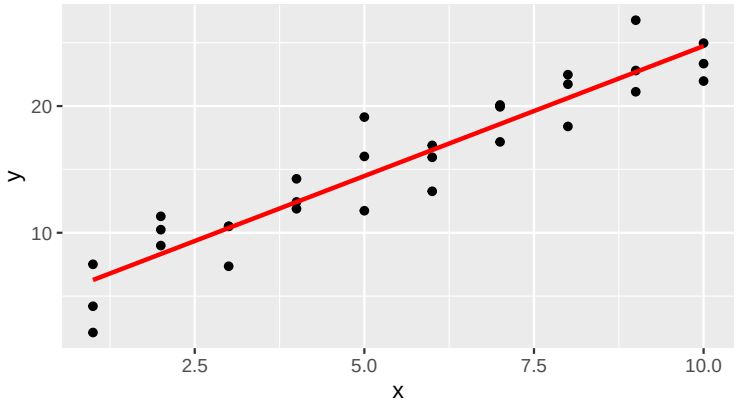
$\sqrt{\cdot}$ and $1/n$ are increasing, so this minimises $\sum_i (y_i - a_1 - a_2 x_i)^2$: least squares, $\hat{a}_2 = \sum (x_i - \bar{x})(y_i - \bar{y}) / \sum (x_i - \bar{x})^2$, $\hat{a}_1 = \bar{y} - \hat{a}_2 \bar{x}$; `optim()` approximates it.

13.3 Visualising models

Motivation. Look at predictions and residuals, for any model.

Predictions. An even grid over the data (`data_grid()`) plus `add_predictions()`:

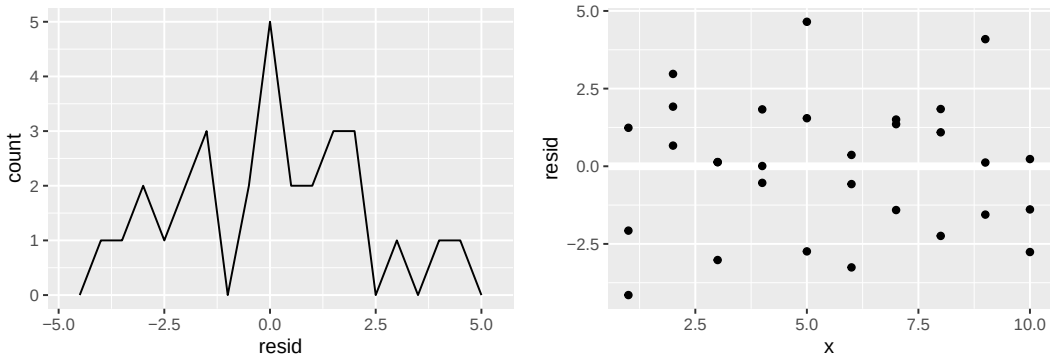
```
grid <- sim1 %>% data_grid(x) %>% add_predictions(sim1_mod)
ggplot(sim1, aes(x)) +
  geom_point(aes(y = y)) +
  geom_line(aes(y = pred), data = grid, colour = "red", linewidth = 1)
```



This shows what any model captured.

Residuals (observed − predicted) show what it missed; `add_residuals()` needs the observed data, not a grid. If the pattern is captured, they look like noise:

```
sim1 <- sim1 %>% add_residuals(sim1_mod)
ggplot(sim1, aes(resid)) + geom_freqpoly(binwidth = 0.5)
ggplot(sim1, aes(x, resid)) + geom_ref_line(h = 0) + geom_point()
```



13.4 Formulas and model families

Motivation. A formula becomes a design matrix, which fixes what a model can fit.

`y ~ x` means $y = a_1 + a_2 x$; `model_matrix()` shows the design matrix, one column per coefficient, intercept of ones unless `- 1`.

```
df <- tribble(~y, ~x1, ~x2,  4, 2, 5,  5, 1, 6)
model_matrix(df, y ~ x1 + x2)
```

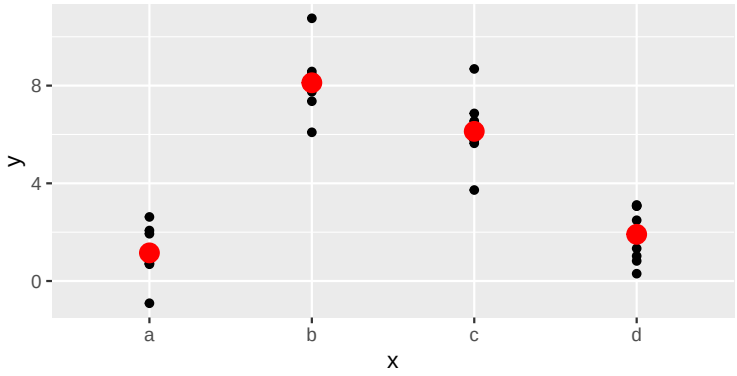
```
## # A tibble: 2 x 3
##   `(Intercept)`    x1    x2
##           <dbl> <dbl> <dbl>
## 1             1     2     5
## 2             1     1     6
```

13.4.1 Categorical variables

`y ~ sex` gives an indicator `sexmale` (`sexfemale` would be redundant, one minus it). A categorical `x` predicts each category's mean:

```
mod2 <- lm(y ~ x, data = sim2)
```

```
grid <- sim2 %>% data_grid(x) %>% add_predictions(mod2)
ggplot(sim2, aes(x)) +
  geom_point(aes(y = y)) +
  geom_point(data = grid, aes(y = pred), colour = "red", size = 4)
```

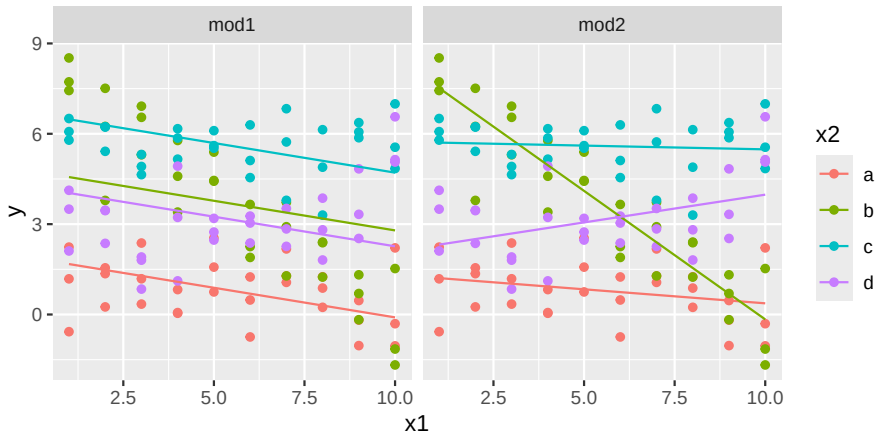


An unseen level (`tibble(x = "e")`) is an error.

13.4.2 Interactions

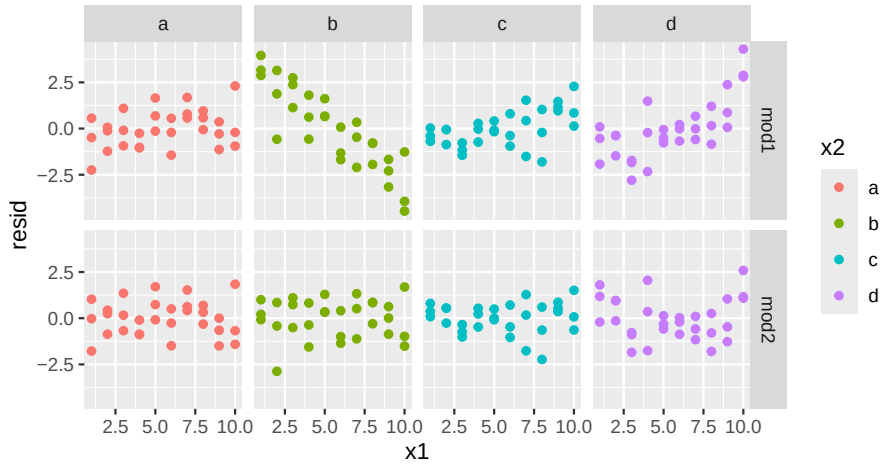
`sim3`: continuous `x1`, categorical `x2`. `y ~ x1 + x2`: parallel lines; `y ~ x1 * x2` adds `x1:x2`, a slope per group. `gather_predictions()` stacks models:

```
mod1 <- lm(y ~ x1 + x2, data = sim3)
mod2 <- lm(y ~ x1 * x2, data = sim3)
grid <- sim3 %>% data_grid(x1, x2) %>% gather_predictions(mod1, mod2)
ggplot(sim3, aes(x1, y, colour = x2)) +
  geom_point() +
  geom_line(data = grid, aes(y = pred)) +
  facet_wrap(~ model)
```



`mod1`'s residuals show patterns in `b`, `c`, `d`; `mod2`'s look random (formal comparison: inference).

```
sim3 <- sim3 %>% gather_residuals(mod1, mod2)
ggplot(sim3, aes(x1, resid, colour = x2)) +
  geom_point() +
  facet_grid(model ~ x2)
```



Two continuous (`sim4`): `y ~ x1 * x2` is $y = a_0 + a_1 x_1 + a_2 x_2 + a_{12} x_1 x_2$; on a grid from `seq_range(x1, 5)` the interaction shows as slopes changing with the other variable.

13.4.3 Transformations

In the formula: `log(y) ~ sqrt(x1) + x2`. Formula operators (`+`, `*`, `^`, `-`) need `I()`: `y ~ x + I(x^2)` is $a_1 + a_2 x + a_3 x^2$, but `y ~ x^2 + x` is `y ~ x`. Linear in the coefficients suffices for curves. `poly(x, 2)`: orthogonal polynomials; `splines::ns(x, df)`: natural splines, tamer than polynomials outside the data.

```
df <- tribble(~y, ~x,  1, 1,  2, 2,  3, 3)
model_matrix(df, y ~ x + I(x^2))
```

```
## # A tibble: 3 x 3
##   `(Intercept)`     x `I(x^2)`
##           <dbl> <dbl>    <dbl>
## 1             1     1        1
## 2             1     2        4
## 3             1     3        9
```

Extrapolation [Supplement]

Outside the data range only the family decides (polynomial to $\pm\infty$, spline linear); neither is evidence, so do not trust far extrapolation.

13.5 Missing values and other model families

`lm()` drops rows with `NA` (silently, or warning with `na.warn`); `nobs(mod)` counts those used.

Linear models assume continuous residuals of constant variance. Extensions:

- `glm()`: binary or count responses;
- `mgcv::gam()`: smooth `y ~ s(x)`;
- penalised (`glmnet`): shrink coefficients;
- robust `MASS::rlm()`: less outlier influence;
- trees and forests (`rpart`, `randomForest`, `xgboost`).

All are explored by predictions and residuals.

13.6 Selected exercises

Motivation. Parameterisations of the same fitted values, and residuals orthogonal to the design.

Exercise 13.1 (R4DS Exercise 23.2.1.3)

What is the problem with optimising the three-parameter model $f(x; a) = a_1 + a_2 x + a_3$ by minimising the root-mean-squared distance?

Solution.

Step 1. Predictions depend on a only via $(a_1 + a_3, a_2)$, so $d(a_1 + t, a_2, a_3 - t) = d(a)$ for all t.

Step 2. With $c = a_1 + a_3$, the minimum over (c, a_2) is the least-squares line ("Why the two agree"), unique for non-constant x; the minimisers of d form the line $\{(c - t, \hat{a}_2, t) : t \in \mathbb{R}\}$: not *identifiable*.

Step 3. `optim()` stops at a start-dependent point of it, reporting different a_1, a_3 for the same line. The parameterisation is redundant; formulas never create two intercepts. ■

Exercise 13.2 (R4DS Exercise 23.4.5.1)

For `sim2`, whose predictor `x` has levels a, b, c, d, compare `y ~ x` with `y ~ x - 1`. What happens to the model equation and to the predictions?

Solution.

Step 1. (Equations.) With intercept: columns $1, x_b, x_c, x_d$, $\hat{y} = a_1 + a_2 x_b + a_3 x_c + a_4 x_d$. Without: all four indicators, $\hat{y} = a_a x_a + a_b x_b + a_c x_c + a_d x_d$.

Step 2. (Same column space.) $x_a + x_b + x_c + x_d = 1$ and $x_a = 1 - x_b - x_c - x_d$, so both designs span the same space; least squares projects y onto it, so fitted values, residuals and predictions coincide.

Step 3. (Coefficients.) Without intercept $\hat{a}_\ell$ is group ℓ's mean; with it $\hat{a}_1$ is group a's mean and $\hat{a}_2, \hat{a}_3, \hat{a}_4$ differences from it. ■

Exercise 13.3 (R4DS Exercise 23.4.5.2)

Which columns does `model_matrix()` create for `y ~ x1 * x2` in `sim3` (continuous `x1`, `x2` with levels a, b, c, d) and in `sim4` (both continuous)? Why is `*` a good shorthand?

Solution.

Step 1. `x1 * x2` = `x1 + x2 + x1:x2`. In `sim3`: intercept, x_1, x_{2b}, x_{2c}, x_{2d}, $x_1 x_{2b}$, $x_1 x_{2c}$, $x_1 x_{2d}$, eight columns: an intercept and a slope per group.

Step 2. In `sim4`: intercept, x_1, x_2, $x_1 x_2$, so $y = a_0 + a_1 x_1 + a_2 x_2 + a_{12} x_1 x_2$ with slope $a_1 + a_{12} x_2$ in x_1.

Step 3. Without main effects the slope in x_1 would vanish at $x_2 = 0$ (or the reference group), depending on the arbitrary origin or coding of x_2; `*` always includes the lower-order terms. ■

Exercise 13.4 (what a residual plot can show)

Let $\hat{a}$ be the least-squares fit of a model whose design matrix X contains an intercept column and a column x, and let $r = y - X\hat{a}$. Show that the residuals have mean 0 and zero sample correlation with x (and with every other column of X). What does this imply for a plot of r against x, and against a variable not in the model?

Solution [Supplement].

Step 1. $\min_a \|y - Xa\|^2$ has gradient $-2X^\mathsf{T}(y - Xa) = 0$ at the minimiser: $X^\mathsf{T} r = 0$.

Step 2. The intercept column gives $\sum_i r_i = 0$, $\bar{r} = 0$; then $\sum_i (x_i - \bar{x})(r_i - \bar{r}) = \sum_i x_i r_i - \bar{x} \sum_i r_i = 0$: zero covariance and correlation.

Step 3. So against an included x there is no linear trend; any pattern (curvature, spread, clusters) is beyond the linear term. Against an omitted variable any correlation is possible, and a trend says: add it. Thus `mod1`'s residuals by group revealed the missing $x_1 x_{2b}, x_1 x_{2c}, x_1 x_{2d}$. ■

Lecture 14. Model Building

14.1 Building models step by step

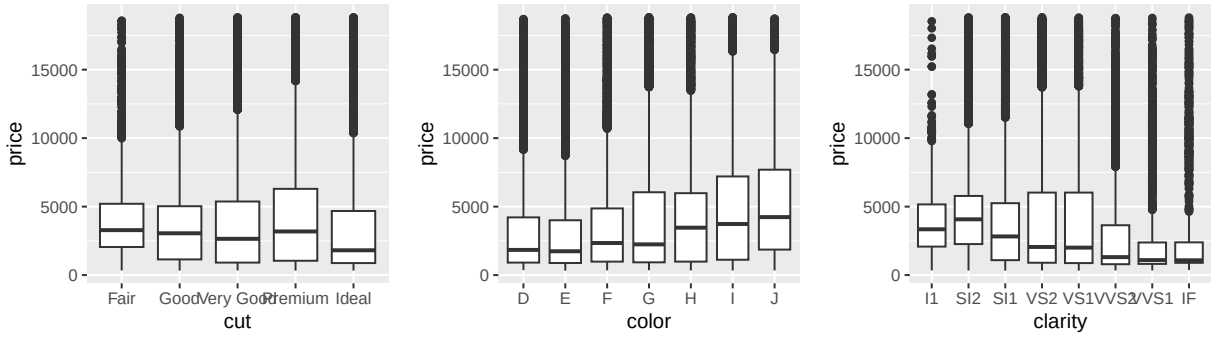
Motivation. Real data overlay patterns; peel them off one at a time through the residuals.

Plot a pattern, model it, study the residuals for the next; implicit knowledge ("prices depend on size") becomes an explicit model. Packages: `modelr`, `nycflights13`, `lubridate`.

14.2 Why are low-quality diamonds more expensive?

Poor cut, colour (J worst) and clarity (I1) have higher median prices (Lecture 4):

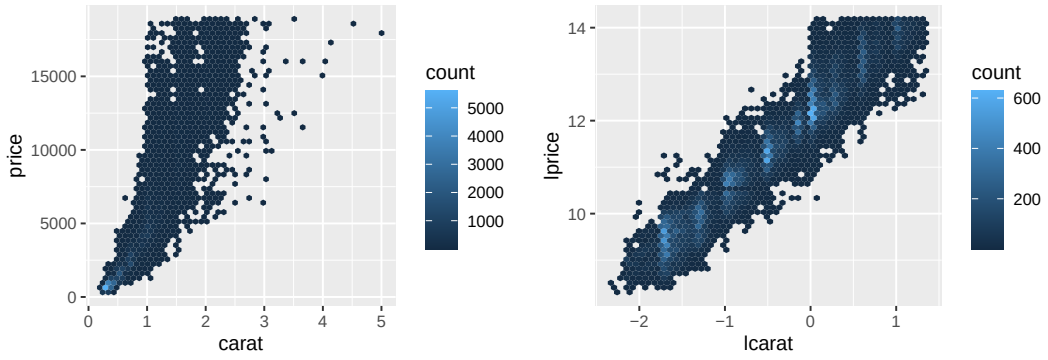
```
ggplot(diamonds, aes(cut, price)) + geom_boxplot()
ggplot(diamonds, aes(color, price)) + geom_boxplot()
ggplot(diamonds, aes(clarity, price)) + geom_boxplot()
```



14.2.1 Price and carat

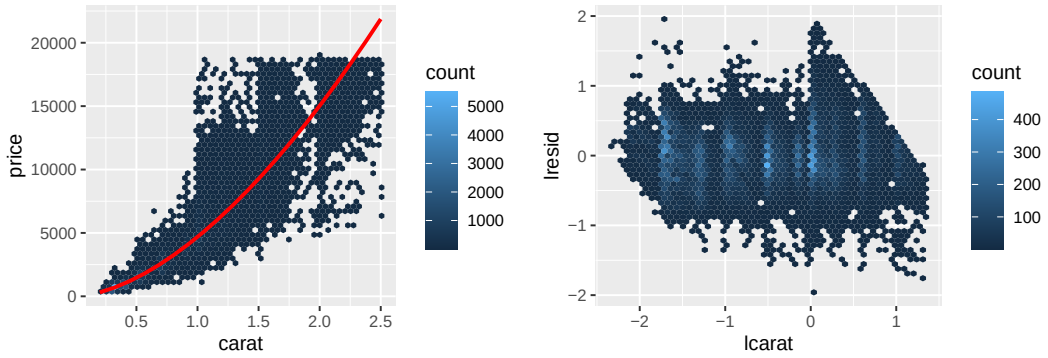
Confounding: `carat` drives price, and low-quality diamonds are larger. Remove carat first: keep ≤ 2.5 carats (99.7%) and log both, which linearises:

```
diamonds2 <- diamonds %>%
  filter(carat <= 2.5) %>%
  mutate(lprice = log2(price), lcarat = log2(carat))
ggplot(diamonds, aes(carat, price)) + geom_hex(bins = 50)
ggplot(diamonds2, aes(lcarat, lprice)) + geom_hex(bins = 50)
```



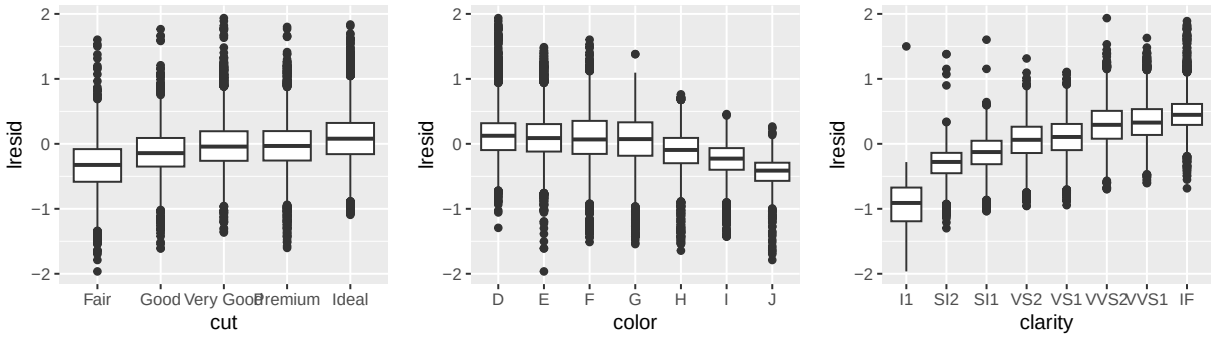
Fit on the log scale, back-transform:

```
mod_diamond <- lm(lprice ~ lcarat, data = diamonds2)
grid <- diamonds2 %>%
  data_grid(carat = seq_range(carat, 20)) %>%
  mutate(lcarat = log2(carat)) %>%
  add_predictions(mod_diamond, "lprice") %>%
  mutate(price = 2 ^ lprice)
ggplot(diamonds2, aes(carat, price)) +
  geom_hex(bins = 50) +
  geom_line(data = grid, colour = "red", linewidth = 1)
diamonds2 <- diamonds2 %>% add_residuals(mod_diamond, "lresid")
ggplot(diamonds2, aes(lcarat, lresid)) + geom_hex(bins = 50)
```



Large diamonds fall below the curve (no price exceeds \$19 000). Residuals no longer depend on carat and show the expected quality order:

```
ggplot(diamonds2, aes(cut, lresid)) + geom_boxplot()
ggplot(diamonds2, aes(color, lresid)) + geom_boxplot()
ggplot(diamonds2, aes(clarity, lresid)) + geom_boxplot()
```



On the $\log_2$ scale, -1 is half the carat-based price, $+1$ twice; relative price rises with quality.

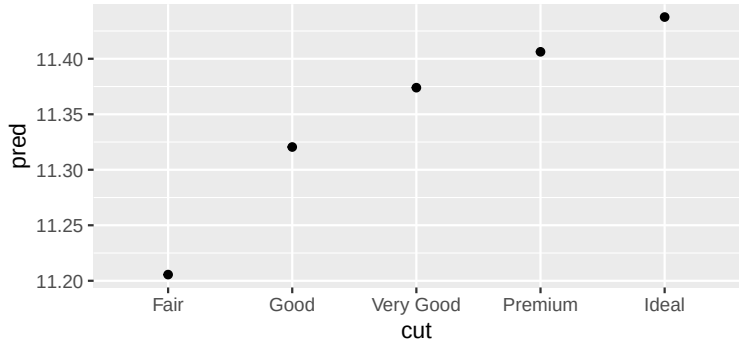
14.2.2 A more complicated model

Add colour, cut and clarity:

```
mod_diamond2 <- lm(lprice ~ lcarat + color + cut + clarity, data = diamonds2)
```

One effect at a time: `data_grid(cut, .model = mod_diamond2)` sets the others to typical values (median or modal level):

```
grid <- diamonds2 %>%
  data_grid(cut, .model = mod_diamond2) %>%
  add_predictions(mod_diamond2)
ggplot(grid, aes(cut, pred)) + geom_point()
```



Some residuals exceed $|r| > 1$ (a factor of two): data errors or bargains?

```
diamonds2 <- diamonds2 %>% add_residuals(mod_diamond2, "lresid2")
diamonds2 %>%
  filter(abs(lresid2) > 1) %>%
  add_predictions(mod_diamond2) %>%
  mutate(pred = round(2 ^ pred)) %>%
  select(price, pred, carat:table, x:z) %>%
  arrange(price)
```

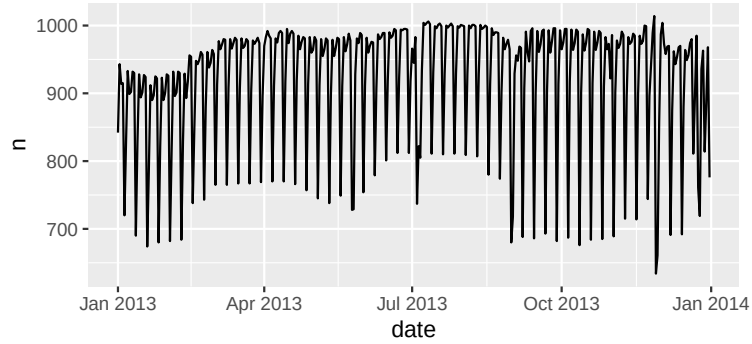
```
## # A tibble: 16 x 11
##   price  pred carat cut     color clarity depth table     x     y     z
##   <int> <dbl> <dbl> <ord>   <ord> <ord>   <dbl> <dbl> <dbl> <dbl> <dbl>
## 1  1013   264  0.25 Fair    F     SI2      54.4    64  4.3   4.23  2.32
## 2  1186   284  0.25 Premium G     SI2      59      60  5.33  5.28  3.12
## 3  1186   284  0.25 Premium G     SI2      58.8    60  5.33  5.28  3.12
## 4  1262  2644  1.03 Fair    E     I1       78.2    54  5.72  5.59  4.42
## 5  1415   639  0.35 Fair    G     VS2      65.9    54  5.57  5.53  3.66
## # i 11 more rows
```

14.3 What affects the number of daily flights?

Motivation. Weekly cycles, seasons and holidays, each seen after removing the last.

Flights per day (365 rows):

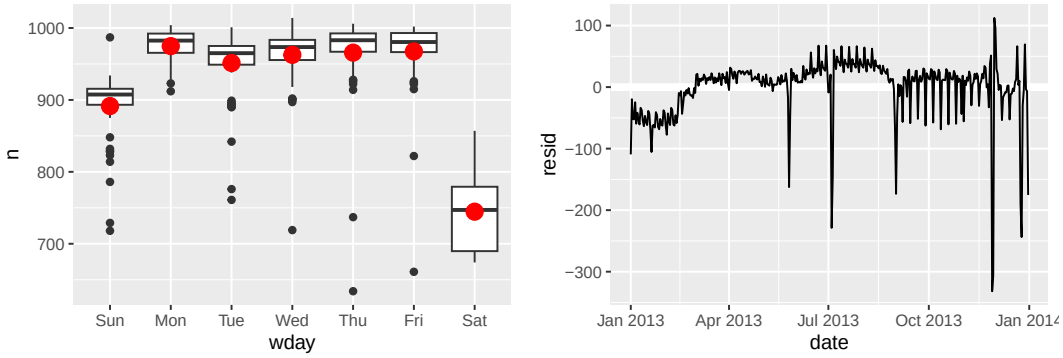
```
daily <- flights %>%
  mutate(date = make_date(year, month, day)) %>%
  group_by(date) %>%
  summarise(n = n())
ggplot(daily, aes(date, n)) + geom_line()
```



14.3.1 Day of week

Business travel: fewer flights at weekends, especially Saturdays. `n ~ wday` predicts weekday means:

```
daily <- daily %>% mutate(wday = wday(date, label = TRUE))
mod <- lm(n ~ wday, data = daily)
grid <- daily %>% data_grid(wday) %>% add_predictions(mod, "n")
ggplot(daily, aes(wday, n)) +
  geom_boxplot() +
  geom_point(data = grid, colour = "red", size = 4)
daily <- daily %>% add_residuals(mod)
daily %>%
  ggplot(aes(date, resid)) +
  geom_ref_line(h = 0) +
  geom_line()
```



Residuals show what it misses:

- from June a regular pattern: Saturdays (coloured by weekday) are badly fit, more in summer, fewer in autumn;
- `filter(resid < -100)`: US holidays (New Year, 4 July, Thanksgiving, Christmas);
- a smooth trend, fewer in January and December, more in summer (`geom_smooth(span = 0.20)`).

```
daily %>% filter(resid < -100)
```

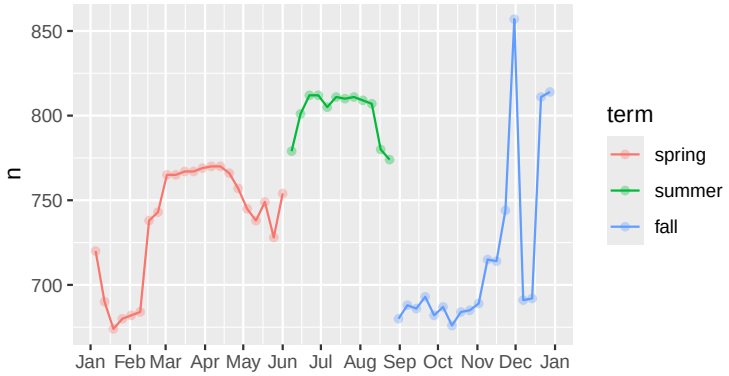
```
## # A tibble: 11 x 4
##   date           n wday  resid
##   <date>     <int> <ord> <dbl>
## 1 2013-01-01   842 Tue   -109.
## 2 2013-01-20   786 Sun   -105.
## 3 2013-05-26   729 Sun   -162.
## 4 2013-07-04   737 Thu   -229.
## 5 2013-07-05   822 Fri   -145.
## # i 6 more rows
```

14.3.2 Seasonal Saturday effect

Saturdays alone suggest school terms (summer-holiday travel); a `term` with breaks read off the plot:

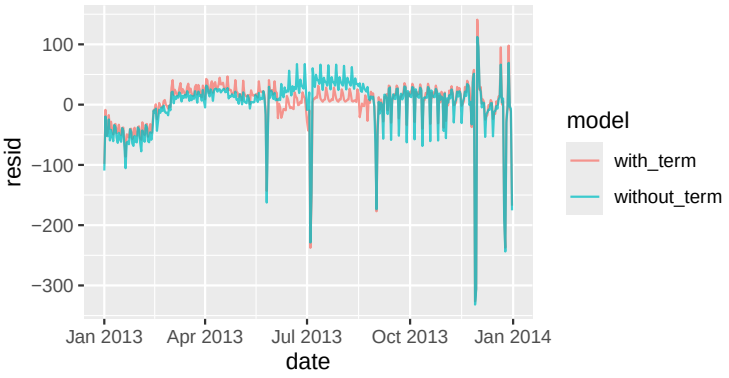
```
term <- function(date) {
  cut(date,
      breaks = ymd(20130101, 20130605, 20130825, 20140101),
      labels = c("spring", "summer", "fall"))
```

```
}
daily <- daily %>% mutate(term = term(date))
daily %>%
  filter(wday == "Sat") %>%
  ggplot(aes(date, n, colour = term)) +
  geom_point(alpha = 1/3) +
  geom_line() +
  scale_x_date(NULL, date_breaks = "1 month", date_labels = "%b")
```



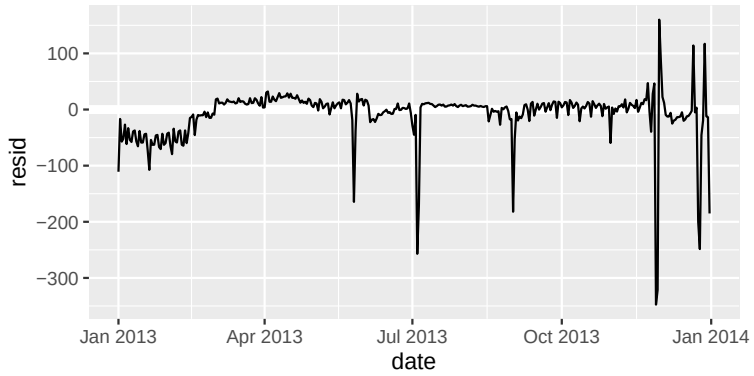
(Labels depend on the locale: a Korean system, as on the slides, returns Korean abbreviations for the filter.) `n ~ wday * term` improves the fit:

```
mod1 <- lm(n ~ wday, data = daily)
mod2 <- lm(n ~ wday * term, data = daily)
daily %>%
  gather_residuals(without_term = mod1, with_term = mod2) %>%
  ggplot(aes(date, resid, colour = model)) +
  geom_line(alpha = 0.75)
```



Cell *means* are pulled by holidays; robust `MASS::rlm()` downweights them, fits the regular pattern better and leaves holidays as large residuals:

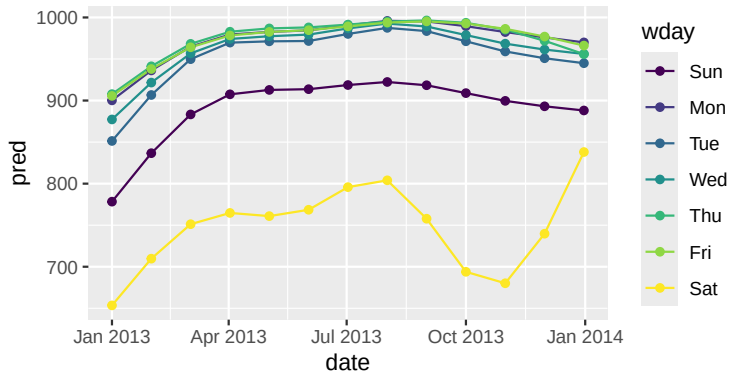
```
mod3 <- MASS::rlm(n ~ wday * term, data = daily)
daily %>%
  add_residuals(mod3, "resid") %>%
  ggplot(aes(date, resid)) +
  geom_hline(yintercept = 0, linewidth = 2, colour = "white") +
  geom_line()
```



14.3.3 Time of year: an alternative approach

A flexible alternative: a natural spline in date, interacting with weekday, a smooth curve per weekday:

```
library(splines)
mod <- MASS::rlm(n ~ wday * ns(date, 5), data = daily)
daily %>%
  data_grid(wday, date = seq_range(date, n = 13)) %>%
  add_predictions(mod) %>%
  ggplot(aes(date, pred, colour = wday)) +
  geom_line() + geom_point()
```



The Saturday pattern reappears unprompted: two approaches, one signal.

Explicit versus flexible models [Supplement]

`term`: a hypothesis in three parameters per weekday, interpretable, but its breaks came from the plot (do not test on the same data). Spline: only smoothness, more parameters, less guesswork. Their agreement suggests the Saturday pattern is real.

14.4 Selected exercises

Motivation. Log–log lines, back-transformed predictions, and too many parameters.

Exercise 14.1 (R4DS Exercise 24.2.3.2)

If $\log_2(\text{price}) = a_0 + a_1 \log_2(\text{carat})$, what does that say about the relationship between price and carat?

Solution.

Step 1. price $= 2^{a_0}\,\text{carat}^{a_1}$, a power law; the base does not affect a_1 ($\log_2 u = \log u / \log 2$ scales both sides).

Step 2. Carat $\times c$ gives price $\times c^{a_1}$ (doubling: 2^{a_1}). $d\log(\text{price})/d\log(\text{carat}) = a_1$: 1% heavier costs about a_1% more (elasticity).

Step 3. If $a_1 > 1$, price per carat $2^{a_0}\text{carat}^{a_1-1}$ rises with size. A $\log_2$ residual r means 2^r times the prediction. ■

Exercise 14.2 (back-transforming predictions)

Suppose $\log_2(\text{price}) = \mu(x) + \epsilon$ with $\epsilon \sim N(0, \sigma^2)$ independent of x. Show that $2^{\mu(x)}$, the back-transformed prediction used in the lecture, is the conditional *median* of price, and that the conditional mean is $2^{\mu(x)} \exp\{(\log 2)^2\sigma^2/2\}$.

Solution [Supplement].

Step 1. $u \mapsto 2^u$ is strictly increasing: $P(\text{price} \le 2^{\mu(x)} \mid x) = P(\epsilon \le 0) = \frac{1}{2}$, the median.

Step 2. Mean: $\mathbb{E}[2^\epsilon] = \mathbb{E}[e^{c\epsilon}]$, $c = \log 2$; completing the square,

$$\begin{aligned}\mathbb{E}[e^{c\epsilon}] &= \int_{-\infty}^{\infty} \frac{1}{\sqrt{2\pi}\sigma} \exp\Big(cu - \frac{u^2}{2\sigma^2}\Big) du \\ &= e^{c^2\sigma^2/2} \int_{-\infty}^{\infty} \frac{1}{\sqrt{2\pi}\sigma} \exp\Big(-\frac{(u - c\sigma^2)^2}{2\sigma^2}\Big) du,\end{aligned}$$

the last integral being 1 (the $N(c\sigma^2, \sigma^2)$ density).

Step 3. So $\mathbb{E}[\text{price} \mid x] = 2^{\mu(x)} e^{(\log 2)^2\sigma^2/2} >$ median: back-transforms underestimate means by this factor but match typical (median) prices. ■

Exercise 14.3 (R4DS Exercises 24.3.5.5 and 24.3.5.6)

(a) What happens if you fit a day-of-week effect that varies by month, `n ~ wday * month`, with `month` a factor? Why is this not very helpful?
(b) What would you expect the model `n ~ wday + ns(date, 5)` to look like?

Solution.

(a)

Step 1. The column space is spanned by the $7 \times 12 = 84$ cell indicators: 84 coefficients for 365 days. As in Exercise 13.2, the RSS separates over cells, so each is fitted by its mean.

Step 2. Cells hold four or five days, so one holiday moves a mean a lot: low bias, high variance, poor generalisation. One parameter per cell also misses the real structure (a Saturday effect changing with term), which `wday * term` expresses in a handful.

(b) Additive: one smooth 5-df seasonal curve shifted by a constant weekday effect, seven parallel curves. The seasonal trend is captured, not the summer Saturday rise, which needs `wday * ns(date, 5)`; Saturday residuals keep the term pattern. ■